

VISION-BASED NAVIGATION PIPELINE FOR AMRs IN A DYNAMIC WAREHOUSE

MR. BOLTON ATHIT DAVIES

A PROJECT SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR
THE DEGREE OF BACHELOR OF ENGINEERING
(ROBOTICS AND AUTOMATION ENGINEERING)
INSTITUTE OF FIELD ROBOTICS
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY THONBURI
2026

Vision-Based Navigation Pipeline for AMRs in a Dynamic Warehouse

Mr. Bolton Athit Davies

A Project Submitted in Partial Fulfillment of the Requirements for
the Degree of Bachelor of Engineering (Robotics and Automation Engineering)
Institute of Field Robotics
King Mongkut's University of Technology Thonburi
2026

Project Committee

..... Chairman of Project Committee
and Project Advisor
(Dr. Rattanachai Ramaithitima)

..... Member
(.....)

..... Member
(.....)

Copyright reserved

Project Title	Vision-Based Navigation Pipeline for AMRs in a Dynamic Warehouse
Project Credits	6
Candidate	Mr. Bolton Athit Davies
Project Advisor	Dr. Rattanachai Ramaithitima
Program	Bachelor of Engineering
Field of Study	Robotics and Automation Engineering
Faculty	Institute of Field Robotics
Academic Year	2026

Abstract

Autonomous mobile robots in warehouses are today navigated with 2-D LiDAR and QR-code infrastructure, which is costly to install and inflexible when the warehouse changes. This project develops and evaluates a vision-based navigation pipeline in which a stereo-inertial visual SLAM front end is combined with a YOLO object detector that removes image features belonging to movable warehouse objects before they enter pose estimation. Two pipelines are compared: ORB-SLAM3 with detector-driven feature rejection, and VINS-Fusion with persistent-track culling feeding an RTAB-Map mapping back end. The work is organised as four experiments, each defined for both a Gazebo simulation of the warehouse and the real robot: a non-visual baseline and estimator comparison, an offline evaluation of the detector, a visual-inertial odometry evaluation with dynamic-environment filtering, and a dense-map reconstruction and map-quality evaluation.

At the reporting cutoff the four simulation experiments have been run and the four real-robot experiments remain planned; one real recording with an independent planar reference, added after the cutoff, is scored provisionally and supports no claim yet. On five static recordings a five-state wheel-plus-gyroscope extended Kalman filter localises to 0.19–0.49 m aligned ATE, ahead of both visual pipelines as configured, but that ordering reverses on every recording once the filter is given the visual systems' own inertial noise model; the noise model, not the architecture, dominates. All three dead-reckoning estimators are overconfident by about two orders of magnitude. The detector reaches a pooled precision of 0.684 and a moving-object recall of 0.591 against projected simulator ground truth, with conditional class accuracy of 0.996. The dynamic-filtering hypothesis remains untested: the VINS-Fusion filter never activated, the ORB-SLAM3 treatment pairs were not input-matched, and the scripted dynamic worlds were found not to constitute a valid testbed; in rebuilt worlds with physically simulated traffic neither front end loses tracking, and the filtering arms again yield no admissible pair. The RTAB-Map map of the small warehouse is geometrically faithful (occupied-cell precision 96.6%, observed-surface recall 89.5%) and drivable along an independent route; the large-warehouse map failed through tracking loss under repetitive structure. The report records these outcomes with their validity limits and sets out the remaining plan.

Keywords: Autonomous Mobile Robot / Dynamic Environment / Extended Kalman Filter / Object Detection / ORB-SLAM3 / RTAB-Map / Visual SLAM / VINS-Fusion / YOLO

หัวข้อโครงงาน	กระบวนการนำทางด้วยภาพสำหรับหุ่นยนต์เคลื่อนที่อัตโนมัติในคลังสินค้าที่เปลี่ยนแปลงตลอดเวลา
หน่วยกิต	6
ผู้เขียน	นายบอลตัน อาทิตย์ เดวิส
อาจารย์ที่ปรึกษา	ดร.รัตนชัย รมัยธิติมา
หลักสูตร	วิศวกรรมศาสตรบัณฑิต
สาขาวิชา	วิศวกรรมหุ่นยนต์และระบบอัตโนมัติ
คณะ	สถาบันวิทยาการหุ่นยนต์ภาคสนาม
ปีการศึกษา	2569

บทคัดย่อ

หุ่นยนต์เคลื่อนที่อัตโนมัติในคลังสินค้าปัจจุบันนำทางด้วยไลดาร์สองมิติร่วมกับคิวอาร์โค้ด ซึ่งมีต้นทุนติดตั้งสูงและปรับเปลี่ยนได้ยากเมื่อผังคลังสินค้าเปลี่ยนไป โครงงานนี้พัฒนาและประเมินกระบวนการนำทางด้วยภาพ โดยผสานระบบระบุตำแหน่งและสร้างแผนที่ด้วยภาพจากกล้องสเตอริโอและหน่วยวัดความเฉื่อยเข้ากับตัวตรวจจับวัตถุ YOLO ซึ่งคัดจุดเด่นบนวัตถุเคลื่อนย้ายได้ออกก่อนการประมาณตำแหน่ง และเปรียบเทียบไปป์ไลน์สองรูปแบบ ได้แก่ ORB-SLAM3 ที่ปฏิเสธจุดเด่นตามผลตรวจจับ และ VINS-Fusion ที่ตัดเส้นทางจุดเด่นคงค้างแล้วส่งต่อให้ RTAB-Map สร้างแผนที่ การประเมินจัดเป็นการทดลองสี่รายการ แต่ละรายการทำทั้งในสถานการณ์จำลองและบนหุ่นยนต์จริง ได้แก่ การเปรียบเทียบตัวประมาณค่าพื้นฐานที่ไม่ใช้ภาพ การประเมินตัวตรวจจับวัตถุแบบออฟไลน์ การประเมินการวัดระยะทางด้วยภาพและความเฉื่อยร่วมกับการกรองวัตถุพลวัต และการประเมินคุณภาพแผนที่

ณ วันตัดยอดรายงาน การทดลองในสถานการณ์จำลองทั้งสี่รายการเสร็จสิ้น ส่วนการทดลองบนหุ่นยนต์จริงยังอยู่ในแผน จากข้อมูลบันทึกฉากคงที่ห้าชุด ตัวกรองคาลมานขยายที่รวมล้อและไจโรสโคปให้ค่าความคลาดเคลื่อนเส้นทางสัมบูรณ์ 0.19 ถึง 0.49 เมตร ดีกว่าไปป์ไลน์ด้วยภาพทั้งสองตามค่าตั้งที่ใช้ แต่ลำดับกลับด้านในทุกชุดข้อมูลเมื่อให้ตัวกรองใช้แบบจำลองสัญญาณรบกวนชุดเดียวกับระบบภาพ แสดงว่าแบบจำลองสัญญาณรบกวนมีผลมากกว่าสถาปัตยกรรม ตัวตรวจจับวัตถุให้ความแม่นยำรวม 0.684 และความครบถ้วนของวัตถุเคลื่อนที่ 0.591 โดยจำแนกประเภทถูกต้อง 0.996 สมมติฐานการกรองวัตถุพลวัตยังไม่ได้รับการทดสอบที่ใช้ได้ เนื่องจากตัวกรองของ VINS-Fusion ไม่ทำงาน คู่ทดลองของ ORB-SLAM3 ได้รับข้อมูลนำเข้าไม่เท่ากัน และโลกจำลองพลวัตยังไม่เป็นสนามทดสอบที่ใช้ได้ แผนที่ RTAB-Map ของคลังสินค้าขนาดเล็กถูกต้องทางเรขาคณิต (ความแม่นยำร้อยละ 96.6 ความครบถ้วนของพื้นผิวที่สังเกตได้ร้อยละ 89.5) และใช้ขับผ่านเส้นทางอิสระได้ ส่วนแผนที่คลังสินค้าขนาดใหญ่ล้มเหลวจากการสูญเสียการติดตามในโครงสร้างที่ซ้ำกัน

คำสำคัญ : การตรวจจับวัตถุ / การระบุตำแหน่งและสร้างแผนที่ด้วยภาพ / คลังสินค้าพลวัต / ตัวกรองคาลมานขยาย / หุ่นยนต์เคลื่อนที่อัตโนมัติ

ACKNOWLEDGEMENTS

[To be written by the author. The template reserves this page for thanks to the project advisor, the examination committee, Gensurv Robotics, the Institute of Field Robotics, and any funding source. Nothing has been drafted here on the author's behalf.]

CONTENTS

PAGE

ABSTRACT ii

THAI ABSTRACT iii

ACKNOWLEDGEMENTS iv

CONTENTS v

LIST OF TABLES viii

LIST OF FIGURES x

LIST OF SYMBOLS xii

LIST OF TECHNICAL VOCABULARY AND ABBREVIATIONS xiii

CHAPTER

1. INTRODUCTION 1
- 1.1 Background and Significance . . . 1
- 1.2 Objectives . . . 1
- 1.3 Expected Benefits . . . 2
- 1.4 Scope . . . 2
- 1.5 Work Plan . . . 2

2. THEORY AND RELATED WORK 4
- 2.1 AMR Navigation Systems . . . 4
  - 2.1.1 Logistics management and the historical bottleneck . . . 4
  - 2.1.2 The era of automated guided vehicles (AGVs) . . . 5
  - 2.1.3 The transition to autonomous mobile robots (AMRs) . . . 5
- 2.2 Visual Simultaneous Localization and Mapping (Visual SLAM) . . . 5
  - 2.2.1 Components of a visual SLAM system . . . 6
  - 2.2.2 ORB-SLAM3 . . . 7
  - 2.2.3 VINS (Visual-Inertial System) . . . 8
  - 2.2.4 RTAB-Map back end and memory management . . . 10
- 2.3 Real-Time Object Detection . . . 11
  - 2.3.1 The role of object detection in semantic understanding . . . 11
  - 2.3.2 The YOLO (You Only Look Once) network architecture . . . 11
  - 2.3.3 Performance evaluation and edge-processor limits . . . 13
- 2.4 RY-SLAM: A Robust YOLO-Based Real-Time Visual SLAM Solution for Dynamic Environments . . . 13
  - 2.4.1 Concept and architecture of RY-SLAM . . . 13
  - 2.4.2 Dynamic feature masking mechanism . . . 13
  - 2.4.3 Relevance to this project . . . 15

3. METHODOLOGY 16
- 3.1 Problem, Requirements and Scope . . . 16

CONTENTS (Cont'd)

PAGE

3.2 System Architecture . . . 17
3.2.1 Control-loop view and data rates . . . 17
3.2.2 Semantic visual SLAM data flow . . . 17
3.2.3 The two navigation pipelines compared . . . 20
3.3 The Pipelines as Implemented . . . 22
3.3.1 Dynamic feature masking . . . 22
3.3.2 ORB-SLAM3 and semantic feature rejection . . . 22
3.3.3 VINS-Fusion front end . . . 23
3.3.4 RTAB-Map mapping and localization back end . . . 25
3.3.5 YOLO detector node, mask construction and synchronisation . . . 26
3.3.6 Navigation and relocalisation integration: the `vslam_navigation` package . . . 27
3.4 Simulation Platform . . . 31
3.4.1 Computing and software environment . . . 31
3.4.2 Simulated sensor rig and platform . . . 31
3.4.3 Simulation fidelity and its limits . . . 35
3.4.4 Coordinate-frame contract . . . 37
3.4.5 Simulation environments and controlled factors . . . 38
3.4.6 Realistic logistics layouts . . . 38
3.4.7 Dataset registry . . . 46
3.5 Real Platform . . . 47
3.5.1 AMR mobile robot testbed . . . 47
3.5.2 Sensors and computing unit . . . 47
3.5.3 Real sensor rig and calibration limits . . . 50
3.5.4 Test site and experimental variables . . . 52
3.6 The Four Experiments . . . 53
3.6.1 Experiment 1: Non-visual baseline and estimator comparison . . . 53
3.6.2 Experiment 2: Offline YOLO detector performance . . . 57
3.6.3 Experiment 3: Visual-inertial odometry with dynamic-environment filtering . . . 59
3.6.4 Experiment 4: Dense map reconstruction and map-quality evaluation 60
3.7 Evaluation Metrics and Analysis . . . 61
3.7.1 Trajectory evaluation . . . 61
3.7.2 Computational performance . . . 62
3.7.3 Collection protocol, validity and stored artifacts . . . 62
3.7.4 Reproducibility controls . . . 64
3.8 Planned Versus Implemented . . . 64

4. EXPERIMENTS AND RESULTS 67

4.1 Experiment 1: Non-Visual Baseline and Estimator Comparison . . . 67
4.1.1 Simulation . . . 67
4.1.2 Real robot . . . 73
4.2 Experiment 2: Offline YOLO Detector Performance . . . 75
4.2.1 Simulation . . . 75
4.2.2 Real robot . . . 79
4.3 Experiment 3: Visual-Inertial Odometry With Dynamic-Environment Filtering . . . 79
4.3.1 Simulation . . . 79
4.3.2 Real robot . . . 89

CONTENTS (Cont'd)

PAGE

4.4 Experiment 4: Dense Map Reconstruction and Map-Quality Evaluation . 96
4.4.1 Simulation 96
4.4.2 Real robot 113
4.5 Cross-Experiment Discussion, Simulation-to-Real Comparison and Limitations 113
4.5.1 Simulation against the real robot 113
4.5.2 Limitations 113

5. CONCLUSION AND RECOMMENDATIONS 115
5.1 Conclusions by Experiment 115
5.1.1 Experiment 1: is vision earning its place? 115
5.1.2 Experiment 2: how good is the detector? 115
5.1.3 Experiment 3: does filtering help? 116
5.1.4 Experiment 4: is the map usable? 116
5.2 Limitations and Threats to Validity 117
5.3 Recommendations and Next Steps 117

REFERENCES 119

CURRICULUM VITAE 121

LIST OF TABLES

TABLE **PAGE**

1.1 Work plan for the 2026 (B.E. 2569) academic year. 3

2.1 Comparison of object-detection model architectures. 12

3.1 The four experiments and the architecture block each one tests. 17
3.2 Implementation status at the reporting cutoff. 18
3.3 Design comparison of the two pipeline architectures, as proposed. . . . 21
3.4 ORB-SLAM3 configuration at the cutoff. 24
3.5 VINS-Fusion simulation configuration at the cutoff, from `vins_fusion_ros2/config/wil_sim/stereo_imu.yaml`. 24
3.6 Navigation parameters for the two platforms. 30
3.7 Tasks provided by the `vslam_navigation` package. 31
3.8 Computing and software environment. 32
3.9 Simulated sensor configuration. 33
3.10 Inertial noise: configured against measured. 34
3.11 Platform geometry and drivetrain limits. 34
3.12 Noise injected by the simulator. 35
3.13 Gazebo physics configuration. 36
3.14 Measured wheel slip on the four recordings. 37
3.15 Configured against injected inertial noise. 37
3.16 Controlled experimental factors and the levels realised. 40
3.17 Zones of the two realistic worlds and the number of object instances each holds. 41
3.18 Dataset registry: the eleven recordings in use. 47
3.19 Specification of the AC-IMX390-H190 camera. 49
3.20 Summary of the hardware technical specification. 51
3.21 Real sensor rig, and the calibration status of each quantity. 51
3.22 Sensor noise and scale measured from each recording. 54
3.23 Implemented non-visual estimators. 55
3.24 Parameters of the detection evaluation. 58
3.25 Principal stored artifacts. 63
3.26 Planned-versus-implemented deviations at the reporting cutoff. 65
3.27 Original objectives (section 1.2) against the work reported here. 66

4.1 Status of the eight result slots at the reporting cutoff. 67
4.2 Recordings of Experiment 1. 68
4.3 Estimator comparison on the four realistic recordings. 69
4.4 Relative pose error at $\Delta t = 1\,\mathrm{s}$ on the four realistic recordings. 70
4.5 Wheel+gyro dead reckoning against the optical tracker. 74
4.6 Detector scores per recording and pooled. 75
4.7 Recall against truth-box size. 76

LIST OF TABLES (Cont'd)

PAGE

4.8 What the unmatched predictions are. . . . 76
4.9 Structure of the misses. . . . 77
4.10 Score-threshold sweep. . . . 77
4.11 Misses and false positives by class. . . . 77
4.12 Unfiltered arms on the four realistic-warehouse recordings. . . . 82
4.13 Realistic world: onset of sustained error, unfiltered runs. . . . 84
4.14 Real-time and resource measurements on the mapping recordings. . . . 86
4.15 Realv6: every surviving run against the cleaned tracker truth. . . . 90
4.16 Realv6: raw versus cleaned tracker export. . . . 90
4.17 Realv6: ORB-SLAM3 drift while turning versus on straights. . . . 91
4.18 Realv6: the VINS-Fusion run. . . . 92
4.19 Realv7: the plane of every ORB-SLAM3 estimate as written in the live log. . . . 93
4.20 Realv7: every run after the frame repair, with its initialisation history. . . . 94
4.21 Realv7: the VINS-Fusion runs. . . . 95
4.22 Conditions for the dense-map evaluation. . . . 97
4.23 Stored contents of `map/map_tiny.db`. . . . 98
4.24 Occupied-cell geometry of `map/map_tiny.db` against `small_warehouse_static_tinymap.world` at 0.25 m tolerance. . . . 98
4.25 Drivability of `map/map_tiny.db` under the true 0.83×0.50 m nav2 footprint, evaluated against two different routes. . . . 99
4.26 Occupied-cell geometry of the four realistic databases. . . . 103
4.27 Drivability of the realistic-warehouse databases under the true nav2 footprint, against independent routes. . . . 104
4.28 Stored contents and build parameters of the realistic-warehouse databases. 104
4.29 Limitations of the results reported in this chapter, and what each one prevents. . . . 114

LIST OF FIGURES

FIGURE **PAGE**

1.1 Scope of the project within the AMR navigation loop. 2
1.2 Top view of the mock warehouse test field on the first floor of the Gensurv building. 3

2.1 The eight principal wastes of lean manufacturing. 4
2.2 Three warehouse robots with different navigation technologies: the orange robot uses QR-code navigation, the pallet truck in the centre uses SLAM navigation without floor infrastructure, and the white robot uses magnetic-tape guidance. 5
2.3 An autonomous driverless transport vehicle navigating precisely in the BMW Group press plant at Regensburg. 6
2.4 The indirect (feature-based) method common to SLAM systems: a front end producing pseudo-measurements and loop-closure constraints, and a back end performing geometric SLAM by factor-graph optimisation [2]. 6
2.5 Main components of ORB-SLAM3: the tracking, local mapping and loop and map-merging threads around the Atlas multi-map representation and the DBoW2 keyframe database [4]. 7
2.6 Main components of monocular VINS: measurement preprocessing, initialisation, sliding-window visual-inertial odometry with relocalization, and global pose-graph optimisation and reuse [5]. 8
2.7 Main components of RTAB-Map: sensor synchronisation, short-term and working memory, long-term memory, loop-closure and proximity detection, graph optimisation and global map assembly [7]. 10
2.8 Architecture of RY-SLAM: a YOLOv8-seg dynamic-object detection stage feeding a semantic mask into the ORB-SLAM3 tracking thread ahead of the Atlas multi-map system [8]. 14
2.9 Detail of the semantic-segmentation framework in RY-SLAM and its multi-threaded operation: ONNX Runtime inference producing the dynamic mask, the neighbourhood test that deletes feature points, and the viewer outputs [8]. 14

3.1 Implemented experimental architecture. 18
3.2 Pipeline 1: YOLO26 with ORB-SLAM3, as designed. 20
3.3 Pipeline 2: YOLO26 with VINS-Fusion and RTAB-Map, as designed. . 21
3.4 ORB-SLAM3 pipeline and the semantic-mask insertion point. 23
3.5 VINS-Fusion pipeline and its two semantic-mask insertion points. . . . 25
3.6 Simulated Ackermann platform, drawn to scale from the model definition. 33
3.7 Coordinate-frame contract used by the composed RTAB-Map pipeline. . 38
3.8 The four realistic recordings as scenes. 39
3.9 Plan of the U-shape realistic world. 42
3.10 Plan of the I-shape realistic world. 43

LIST OF FIGURES (Cont'd)

PAGE

3.11 Baked occupancy truth of the two realistic worlds. . . . 45
3.12 The two mapping recordings as scenes. . . . 46
3.13 Dimensions of the mock AMR. . . . 48
3.14 Hardware components of the mock AMR. . . . 48
3.15 Dimensions of the AC-IMX390 camera family. . . . 49
3.16 The Neousys NRU-51V computing unit. . . . 50
3.17 Dimensions of the Neousys NRU-51V in millimetres. . . . 50

4.1 Estimator comparison on every realistic recording, world frame. . . . 71
4.2 Wheel+gyro dead reckoning against the tracker. . . . 74
4.3 Recall against truth-box size. . . . 78
4.4 What the unmatched predictions are. . . . 78
4.5 Unfiltered estimates on the four realistic recordings. . . . 81
4.6 Aligned ATE and jump-free RPE of the unfiltered arms. . . . 82
4.7 Timelines, static U-layout recording. . . . 84
4.8 Timelines, static I-layout recording. . . . 85
4.9 Timelines, U-layout recording with traffic. . . . 85
4.10 Timelines, I-layout recording with traffic. . . . 88
4.11 Real runs against the cleaned tracker truth. . . . 90
4.12 ORB-SLAM3 planar error over the run. . . . 91
4.13 VINS-Fusion resets and IMU sample use. . . . 92
4.14 The twelve realv7 estimates as written in the live log. . . . 94
4.15 The twelve realv7 runs after the frame repair. . . . 95
4.16 Dense-map evaluation of `map/map_tiny.db`. . . . 100
4.17 The same database scored against the run that built it, `dataset/dataset_map_tiny`. . . . 101
4.18 Placement overlay of `map/map_realistic` (U-Oct). . . . 105
4.19 Placement overlay of `map/dataset_allsensor_realistic_map` (U-Nov), fitted in the -180° basin. . . . 106
4.20 Dense-map evaluation of `map/map_realistic` (U-Oct) against the independent probe. . . . 107
4.21 Dense-map evaluation of `map/dataset_allsensor_realistic_map` (U-Nov) against the independent probe. . . . 108
4.22 Dense-map evaluation of `map/map_realistic_01` (I-Oct) against the independent probe. . . . 111
4.23 Dense-map evaluation of `map/dataset_allsensor_realistic_01_map` (I-Nov) against the independent probe. . . . 112

LIST OF SYMBOLS

SYMBOL		UNIT
a, a_x	longitudinal acceleration measured by the IMU	$\mathrm{m\,s^{-2}}$
B	wheel track (lateral wheel separation)	m
B_k	the k-th detected bounding box $\{x_k, y_k, w_k, h_k, c_k, P_k\}$	–
b_a, b_g, b_ω	accelerometer, gyroscope and yaw-rate biases	$\mathrm{m\,s^{-2}}$, $\mathrm{rad\,s^{-1}}$
Δ	relative-pose-error interval	s
$\mathbf{e}$	estimation error vector $[\Delta x, \Delta y, \Delta\theta]$	m, m, rad
E_i	relative pose error transform at pose i	–
$\mathbf{F}$, $\mathbf{G}$	state-transition and noise-input Jacobians	–
$\mathscr{F}_{\text{static}}$	set of image features surviving the dynamic mask	–
f_x, f_y, c_x, c_y	pinhole intrinsics	pixel
g	gravitational acceleration	$\mathrm{m\,s^{-2}}$
$\mathbf{H}$	measurement matrix	–
$\mathbf{K}$	Kalman gain	–
L	wheel base	m
n	number of experimental units (recordings)	–
$\mathbf{P}$	state covariance	–
P, Q	estimated and reference pose sequences (RPE definition)	–
p_x, p_y	planar position	m
$\mathbf{Q}$, $\mathbf{R}$	process- and measurement-noise covariances	–
R, T	rotation matrix and rigid transform	–
r	wheel radius	m
σ	standard deviation of the quantity it subscripts	as subscript
θ, θ_m	heading and midpoint heading	rad
τ	bias correlation time	s
v	longitudinal speed	$\mathrm{m\,s^{-1}}$
ω	yaw rate	$\mathrm{rad\,s^{-1}}$
$\mathbf{x}$	EKF state $[p_x, p_y, \theta, v, b_\omega]^\top$	–
χ	VINS sliding-window state vector	–

LIST OF TECHNICAL VOCABULARY AND ABBREVIATIONS

AGV	= Automated Guided Vehicle
AMR	= Autonomous Mobile Robot
ANEES	= Average Normalised Estimation Error Squared
AP, mAP	= Average Precision, mean Average Precision
ATE	= Absolute Trajectory Error
BA	= Bundle Adjustment
CPU, GPU	= Central / Graphics Processing Unit
DoF	= Degrees of Freedom
EKF	= Extended Kalman Filter
FOV, HFOV	= (Horizontal) Field of View
FN, FP, TP	= False Negative, False Positive, True Positive
GT	= Ground Truth
IMU	= Inertial Measurement Unit
IoU	= Intersection over Union
IQR	= Interquartile Range
KLT	= Kanade–Lucas–Tomasi optical-flow tracker
LC	= Loop Closure
LiDAR	= Light Detection and Ranging
LTM, STM, WM	= Long-Term, Short-Term and Working Memory (RTAB-Map)
MAP	= Maximum a Posteriori
NEES	= Normalised Estimation Error Squared
NMS	= Non-Maximum Suppression
QoS	= Quality of Service (ROS 2 middleware)
RMSE	= Root-Mean-Square Error
ROS	= Robot Operating System
RPE	= Relative Pose Error
RSS	= Resident Set Size
RTAB-Map	= Real-Time Appearance-Based Mapping
SfM	= Structure from Motion
SGBM	= Semi-Global Block Matching
SLAM	= Simultaneous Localization and Mapping
TF	= ROS transform tree
VIO	= Visual-Inertial Odometry
VINS	= Visual-Inertial System (VINS-Mono / VINS-Fusion)
YOLO	= You Only Look Once object detector

CHAPTER 1 INTRODUCTION

1.1 Background and Significance

Autonomous mobile robots (AMRs) play a central role in raising the efficiency of warehouse logistics. Gensurv Robotics currently relies on 2-D LiDAR sensing combined with QR-code navigation. That solution carries business limitations: LiDAR sensors are expensive, and the approach is inflexible because the company must spend money and time modifying the warehouse infrastructure to install artificial reference markers. Moving from LiDAR to vision-based navigation, which uses lower-cost camera sensors and yields richer information about the environment, is therefore an option that directly addresses the company's cost problem.

Bringing cameras and simultaneous localization and mapping from images (visual SLAM) into real warehouse use nevertheless raises a substantial academic challenge. Most standard visual SLAM algorithms were developed under the assumption that the environment is static. Saputra et al. [1] state clearly that when a SLAM system is deployed in a dynamic environment, such as a warehouse in which goods and obstacles are continually moved, its accuracy degrades severely, because the system draws features from moving objects into its pose computation. Cadena et al. [2] further stress that the greatest challenges facing SLAM today are robustness and lifelong mapping: the robot must be able to separate permanent structure from dynamic objects.

To address navigation in a dynamic environment, this project proposes the development of a vision-based navigation pipeline centred on integrating the YOLO26 deep-learning model to detect and filter dynamic objects out of the localization system. The project develops and compares the performance of two navigation-system architectures: (1) operation with the ORB-SLAM3 framework, and (2) operation with VINS combined with RTAB-Map, in order to find the most robust approach for a warehouse.

In addition, replacing an active sensor (LiDAR) with a passive one (a camera) may make the system sensitive to indoor lighting conditions. The project therefore designs an experiment that simulates the environment and the light intensity at different times of day, so that data can be collected and it can be assessed empirically whether warehouse lighting significantly affects the performance of the visual SLAM system. That assessment will be important input to the company's long-term development and improvement of its navigation system.

1.2 Objectives

1. Develop a visual-inertial odometry localization system with lifelong mapping, so that the warehouse map is updated in real time.
2. Develop image enhancement for low-light environments with the EnlightenGAN model.
3. Develop perception of obstacles and moving objects (dynamic objects) through deep-learning object detection.
4. Develop lane detection with PolyNET and lane tracking with a PID controller.
5. The integrated system must be able to build a 2-D cost map and localize the robot with an error of less than 10 cm under differing light intensities.

1.3 Expected Benefits

1. Help the company reduce the cost of modifying warehouse infrastructure for AMR navigation.
2. Obtain a navigation pipeline that is robust to a dynamic environment.
3. Allow the pipeline to be plugged into the control and motion-planning systems for real use on warehouse robots.

1.4 Scope

1. The main tests are carried out only on a small mobile robot computing on an NRU-51V processing unit and using AC-IMX390-H190 cameras.
2. Within the AMR navigation system, the author's work is framed only within the visual SLAM, object prediction and lane-tracking controller blocks (red frame) shown in figure 1.1.
3. The integrated system is tested only in a mock warehouse arena with changing mock goods, a walkway with a 5° slope, five stacks of goods of four boxes each, and a starting point at the grey square at the lower right of figure 1.2. The positions of the mock warehouse contents are changed between tests to simulate a dynamic environment.

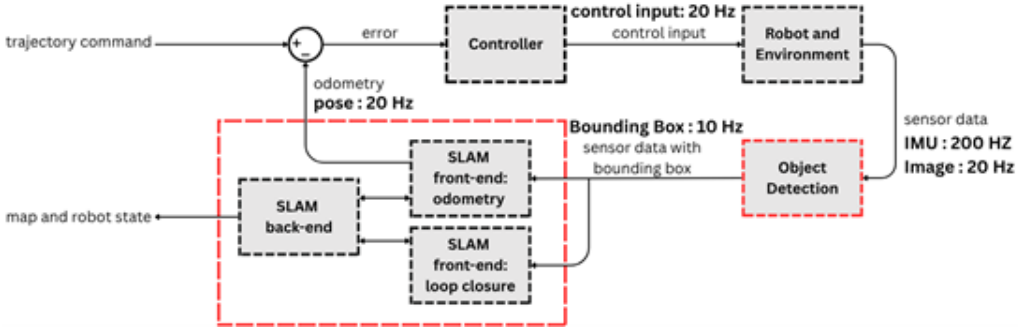


Figure 1.1 Scope of the project within the AMR navigation loop. The red dashed frame encloses the blocks this project develops: the SLAM front end (odometry and loop closure), the SLAM back end (map and robot state) and the object-detection block that supplies bounding boxes. The controller and the robot are existing components. Rates are those of the target system: control input 20 Hz, odometry pose 20 Hz, bounding boxes 10 Hz, IMU 200 Hz, images 20 Hz.

1.5 Work Plan

Table 1.1 reproduces the project work plan of the proposal. Task numbering follows the original plan, which has no item 5.

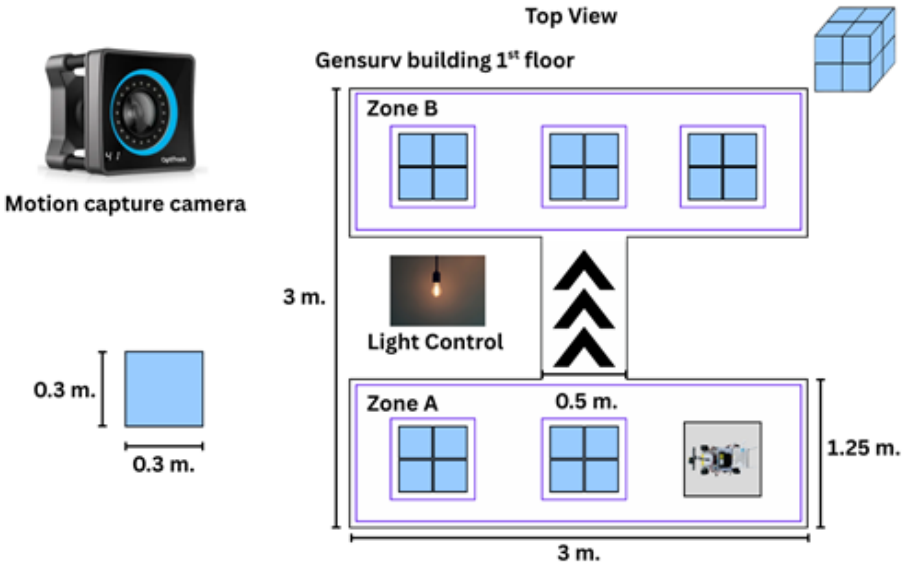


Figure 1.2 Top view of the mock warehouse test field on the first floor of the Gensurv building. Zone A and Zone B are each 3 m square and are joined by a walkway; the robot starts at the grey square in Zone A. Goods are 0.3 m cubes stacked in piles, a light-control fixture sets the illumination, and motion-capture cameras around the field provide the ground-truth reference.

Table 1.1 Work plan for the 2026 (B.E. 2569) academic year. Shaded cells mark the months allocated to each task.

Task	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1. Study the concepts and theory of vision-based navigation	■						
2. Search and analyse information on visual SLAM algorithms, low-light image enhancement (EnlightenGAN), object detection (YOLO26) and lane tracking (PolyNET)	■						
3. Design the structure and software pipeline that joins the hardware and software systems		■					
4. Specify and connect the AC-IMX390-H190 camera sensors with the NRU-51V edge computing unit on the mobile robot		■					
6. Integrate all subsystems under the AMR navigation framework			■				
7. Prepare the mock warehouse area (Zone A and Zone B) with a 5° slope and arrange mock goods to create a dynamic environment			■				
8. Run the integrated system experiment (live test) with the robot moving at 1.5 m s^{-1} under scheduled changes in lighting				■			
9. Collect the test data and evaluate system performance on the defined metrics: trajectory error (ATE, RPE), system latency, and detection accuracy (IoU, mAP)					■		
10. Summarise the work, discuss the results and produce the complete engineering project report						■	■

CHAPTER 2 THEORY AND RELATED WORK

In carrying out the project "Vision-Based Navigation Pipeline for AMRs in Dynamic Warehouse", the author's aim is to develop and evaluate only the navigation pipeline, within the scope defined in section 1.4. The project does not set out to build a complete autonomous robot from scratch; it integrates navigation software into the existing architecture of an autonomous mobile robot (AMR). To make that development effective, the author studied and collected theory, documents and research on the following principal topics.

2.1 AMR Navigation Systems

Autonomous mobile robots are among the technologies transforming industry in the digital era, particularly in manufacturing, warehousing and materials handling. With their capacity to learn, their flexibility and their high precision, AMRs reduce cost and raise efficiency by a large step. This section reviews the properties, technology and use cases of AMRs to make the potential of the innovation clear.

2.1.1 Logistics management and the historical bottleneck

In a traditional warehouse, most wasted expenditure arises in the logistics of moving material from one point to another, work that exhausts staff who must walk while continuously carrying loads. These movements produce "motion waste", one of the eight principal wastes of lean manufacturing (figure 2.1). To stay competitive and cut logistics costs, smart warehouses have begun to bring autonomous robots in to work alongside people and reduce that time and waste.



Figure 2.1 The eight principal wastes of lean manufacturing. Source: https://teachme-biz.com/th/blog/reduce-waste/.

2.1.2 The era of automated guided vehicles (AGVs)

Research and development of autonomous mobile robots began in the 1950s with automated guided vehicles. AGV systems have an important limitation: the vehicle depends on infrastructure installed in the warehouse beforehand for navigation, such as magnetic tape, QR codes or artificial landmarks (figure 2.2). AGVs can also only follow fixed, pre-planned routes, so the system lacks spatial flexibility, and if the warehouse environment changes the AGV cannot adapt immediately.

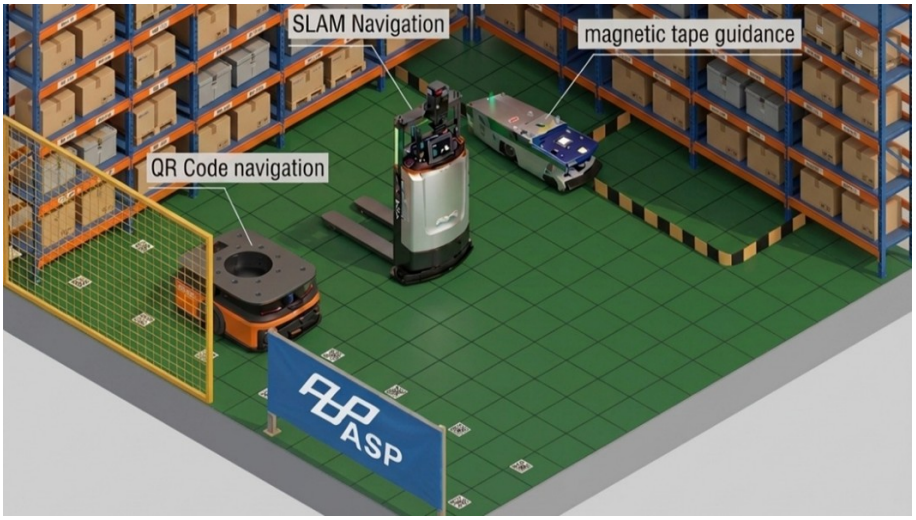


Figure 2.2 Three warehouse robots with different navigation technologies: the orange robot uses QR-code navigation, the pallet truck in the centre uses SLAM navigation without floor infrastructure, and the white robot uses magnetic-tape guidance. Source: https://www.linkedin.com/pulse/agv-amr-navigation-technologies-which-one-drives-your-abdul-rahman-6pjyc.

2.1.3 The transition to autonomous mobile robots (AMRs)

AMRs were developed to overcome the limitations of AGVs. The essential difference is that an AMR can make its own decisions and does not need pre-installed external infrastructure to navigate. Recent hardware advances, particularly in imaging sensors (2-D/3-D cameras and LiDAR) together with high-performance processors, let an AMR perceive and assess its surroundings accurately. The distinguishing capability that lets AMRs replace AGVs is operation in a dynamic environment: when an AMR detects an obstacle on its route it can compute alternative routes to avoid the obstacle or another robot automatically, without the user redesigning the route.

With this capacity for independent decision-making and navigation, large industrial users have moved to AMRs widely. One example is the deployment of autonomous driverless transport vehicles in the press plant of the BMW Group at Regensburg [3], whose vehicles navigate and manoeuvre precisely through busy, complex areas of a real factory (figure 2.3), showing that AMRs have fully overcome the limits of the older navigation systems.

2.2 Visual Simultaneous Localization and Mapping (Visual SLAM)

In the operation of an AMR, localization is the heart of what lets the robot move accurately. The widely accepted and used method is SLAM (simultaneous localization and mapping).



Figure 2.3 An autonomous driverless transport vehicle navigating precisely in the BMW Group press plant at Regensburg. Source: https://www.press.bmwgroup.com/global/video/detail/PF0009726/.

A SLAM navigation system extracts features present in the environment to build a map; the software gathers information from the robot's current position and the positions it has passed through to estimate a probability distribution and predict the robot's most likely position.

2.2.1 Components of a visual SLAM system

This project concentrates on visual SLAM, mapping and localization with image sensors, to reduce the cost of LiDAR sensors. Following the project's own data-flow diagram, a visual SLAM architecture has three main parts (figure 2.4):

1. **SLAM front end (visual odometry)**: receives camera images and extracts features to estimate the robot's relative motion and pose over short intervals.
2. **SLAM back end**: refines and corrects the accumulated drift of the front-end estimate using graph optimisation, to obtain the most accurate map and pose.
3. **Loop closure**: the process by which the system recognises a place the robot has visited before; when the robot returns, the current image is matched to a past one and the whole map is corrected to reduce accumulated error.

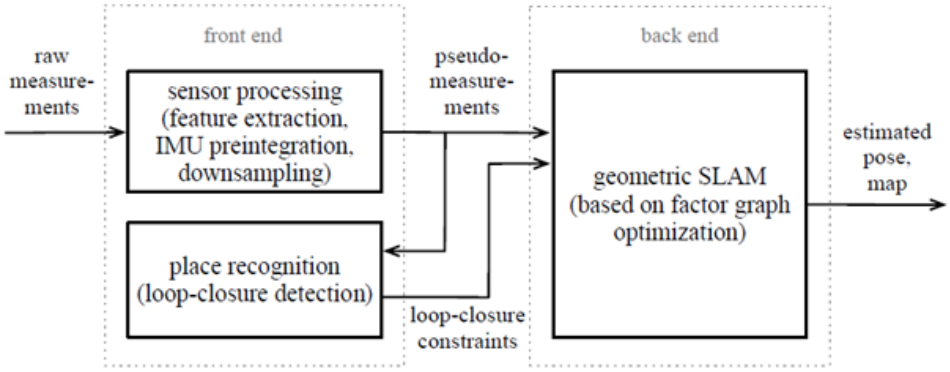


Figure 2.4 The indirect (feature-based) method common to SLAM systems: a front end producing pseudo-measurements and loop-closure constraints, and a back end performing geometric SLAM by factor-graph optimisation [2].

2.2.2 ORB-SLAM3

ORB-SLAM3 [4] is an open-source SLAM system that builds on the success of ORB-SLAM and ORB-SLAM2. It is the first framework to support visual, visual-inertial and multi-map SLAM with a range of camera types, monocular, stereo and RGB-D, and with both pinhole and fisheye lens models (figure 2.5). The parts of its architecture and mechanisms relevant to this project are as follows.

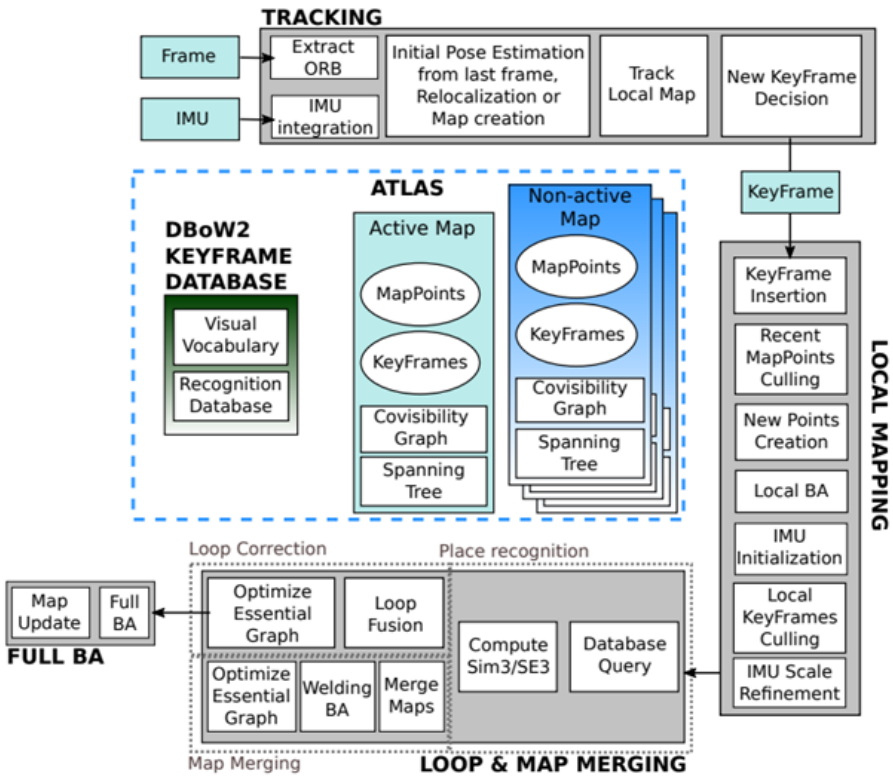


Figure 2.5 Main components of ORB-SLAM3: the tracking, local mapping and loop and map-merging threads around the Atlas multi-map representation and the DBoW2 keyframe database [4].

Tightly-coupled visual-inertial SLAM

One of the most important advances of ORB-SLAM3 is its visual-inertial SLAM (VI-SLAM), which fuses data tightly by maximum-a-posteriori (MAP) estimation both during initialisation and in normal operation, unlike earlier systems that separated the computations. Its mechanisms are:

- **Fast IMU initialisation.** The main problem of typical VI-SLAM systems is the delay in computing the IMU's initial parameters. ORB-SLAM3 proposes a MAP uncertainty computation that recovers the map scale, the gravity direction and the gyroscope and accelerometer biases within about two seconds of start-up, so the robot obtains a correct pose with a physically true distance scale immediately.
- **Joint local bundle adjustment.** While moving, the system solves for the robot pose together with the 3-D landmarks and the IMU states (velocity and biases) simultaneously within a local window, compensating the accumulated drift of the images and the inertial sensor effectively.

Atlas: multi-map system and place recognition

To support lifelong mapping in an environment that changes or has visual disturbances, such as a dynamic warehouse, ORB-SLAM3 introduces the Atlas map management architecture, holding maps in two states:

- **Active map**: the local sub-map the robot is currently using for tracking and mapping.
- **Non-active maps**: the set of past sub-maps the system has recorded and kept in memory.

When the robot passes through severe occlusion, or the lighting changes so far that the camera cannot follow the previous features (tracking lost), an ordinary SLAM system usually fails and stops. ORB-SLAM3 instead moves the map that lost tracking into the non-active set and immediately starts a new active map to continue localizing. When the robot later returns to a previously mapped area, the place-recognition algorithm, with its improved recall, detects the overlap and triggers map merging of the current active map with the earlier non-active map, correcting the error by loop closure. This mechanism permanently reduces the risk of the robot becoming lost in the warehouse.

Abstract camera representation for wide-angle lenses

Because this project selects the AC-IMX390-H190 camera with an ultra-wide 186° field of view (fisheye lens), ORB-SLAM3 is technically well suited: its architecture places an abstract representation layer between the feature extractor and the camera projection model, so the system can switch to the Kannala–Brandt fisheye model directly without affecting the back-end SLAM algorithms. The robot can therefore process wide-angle images correctly and without distortion in the localization result.

2.2.3 VINS (Visual-Inertial System)

VINS (VINS-Mono, VINS-Fusion) [5] is a high-performance open-source state estimator developed by the Aerial Robotics Group (HKUST). Its extended versions can fuse GPS, but for this project's indoor dynamic warehouse VINS is used as a local 6-DoF visual-inertial odometry (VIO) estimator (figure 2.6). Its main mechanisms are as follows.

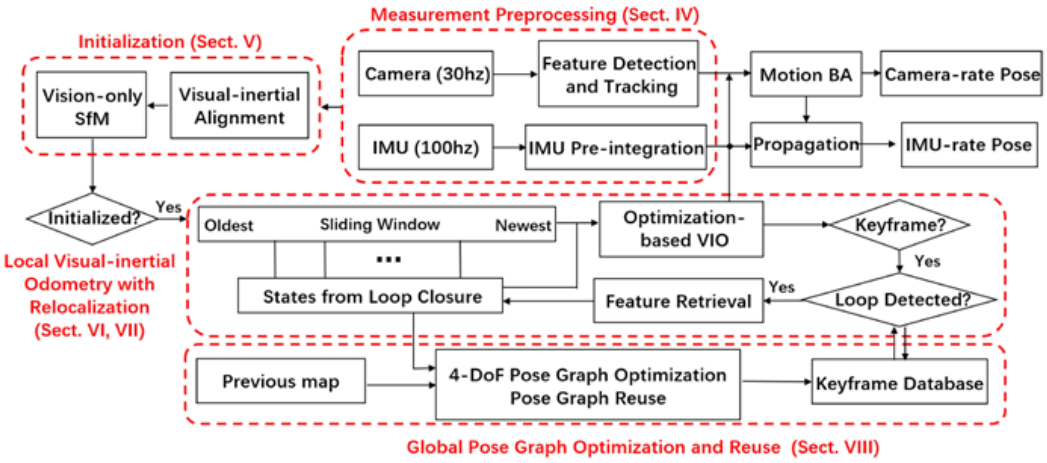


Figure 2.6 Main components of monocular VINS: measurement preprocessing, initialisation, sliding-window visual-inertial odometry with relocalization, and global pose-graph optimisation and reuse [5].

IMU pre-integration

In a fused sensor system the accelerometer and gyroscope deliver data at a very high rate (100–200 Hz) while the camera delivers images at a lower one (20–30 Hz). If a SLAM system re-integrated the IMU data every time a pose was adjusted, the computational load would be enormous. VINS solves this with IMU pre-integration: the accelerations and angular rates between two consecutive image frames are integrated in the robot's local body frame rather than the global frame, so the pre-integrated measurements do not depend on the initial state and need not be recomputed when the back end corrects the robot's pose. The system also propagates covariance and corrects gyroscope and accelerometer bias through a first-order Taylor expansion, giving a physically accurate pose estimate.

Robust estimator initialisation

A central challenge of visual-inertial SLAM is initialisation, because monocular or stereo images carry no true metric scale at the outset and the IMU carries accumulated bias. VINS establishes a tightly-coupled initialisation in two steps:

- **Local structure from motion (SfM).** Features are extracted and tracked over a window of time to build a 3-D structure and the relative camera poses first.
- **Visual-inertial alignment.** The SfM trajectory is set against the IMU pre-integration results to recover three initial quantities: the metric scale, the gravity direction relative to the robot, and the initial gyroscope bias. The AMR thereby perceives true metric distances in the warehouse immediately after it begins to move.

Tightly-coupled sliding-window optimisation

At the heart of the VINS pipeline is a sliding window that limits the number of image frames and robot states that must be solved for. VINS forms a nonlinear least-squares cost function, solved with Ceres Solver, in which three residuals are tightly coupled:

$$\min_{\chi}\Big\{\big\|r_p - H_p\chi\big\|^2 + \sum_{k\in\mathscr{B}}\big\|r_{\mathscr{B}}\big(\hat{z}^{b_k}_{b_{k+1}},\chi\big)\big\|^2_{P^{b_k}_{b_{k+1}}} + \sum_{(l,j)\in\mathscr{C}}\big\|r_{\mathscr{C}}\big(\hat{z}^{c_j}_{l},\chi\big)\big\|^2_{P^{c_j}_{l}}\Big\}, \tag{2.1}$$

where

- the **marginalisation residual** r_p is the prior obtained from past frames removed from the sliding window;
- the **IMU residual** $r_{\mathscr{B}}$ is the error between the robot's true state and the IMU pre-integration; and
- the **visual re-projection residual** $r_{\mathscr{C}}$ is the re-projection error of feature points on the stereo image plane.

Marginalisation for real-time load control

To keep the edge processor (NRU-51V) on the robot running in real time, VINS removes frames from the sliding window under two conditions:

- **Marginalise old (`MARGIN_OLD`).** If the newest frame is judged a keyframe, the oldest frame in the window is removed, and its visual and inertial information is converted into a prior factor by the Schur complement so that important past parameters are not lost.

- **Marginalise second newest (`MARGIN_SECOND_NEW`).** If the newest frame is judged a non-keyframe (insufficient parallax), the second-newest frame is discarded outright without forming a prior. This bounds the density of the matrices and saves a great deal of computation when the AMR is stationary or moving slowly in the warehouse.

Online temporal calibration

A distinctive feature of VINS [6] is its treatment of temporal misalignment between the camera and the IMU. In practice the camera trigger and the IMU signals suffer transmission and triggering delays, so the two sensors' timestamps disagree, an important cause of VIO failure when the robot moves fast or turns. VINS adds the time offset as a state variable estimated jointly in the back-end optimisation, computing the temporal shift of the image frames against the IMU angular rate. The pipeline can thereby compensate the latency of the AC-IMX390-H190 image stream on the processing board precisely, keeping tracking consistent even when the robot vibrates on the 5° warehouse slope.

2.2.4 RTAB-Map back end and memory management

Following Labbé and Michaud [7], RTAB-Map is an open-source SLAM library designed to avoid failure through computational and memory explosion when a robot operates over a large area or for a long time (lifelong mapping). RTAB-Map receives odometry and point-cloud data from VINS and performs graph SLAM in the back end through three main mechanisms (figure 2.7).

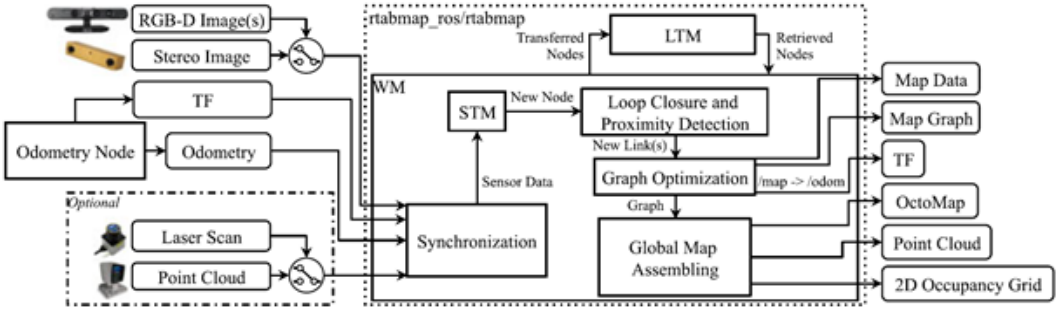


Figure 2.7 Main components of RTAB-Map: sensor synchronisation, short-term and working memory, long-term memory, loop-closure and proximity detection, graph optimisation and global map assembly [7].

Memory partitioning modelled on human memory

RTAB-Map does not keep and compute over every map node in RAM; it organises the data on three levels:

- **Short-term memory (STM)** receives the latest image and odometry from VINS, extracts features and creates a new node. Nodes in STM are not yet used for loop-closure computation.
- **Working memory (WM)** is the main RAM area in which the algorithm detects loop closures and performs graph optimisation. The WM size is held constant by a time threshold so that the computation time never exceeds the real-time limit (for example <30 ms).

- **Long-term memory (LTM)** is the database written to permanent storage (SSD or hard disk). Its contents do not enter the ordinary graph optimisation, conserving RAM and CPU.

Memory transfer and weighting heuristics

To lock the processing time at a constant ($O(1)$ time complexity) regardless of how large the warehouse map becomes, RTAB-Map moves data between WM and LTM as follows:

- **Node weighting.** When the robot traverses a previous route and a loop closure occurs, the nodes there gain weight; frequently traversed nodes (such as the main warehouse aisles) carry high weight.
- **WM-to-LTM transfer.** When WM processing time approaches the set limit, the algorithm forces the "oldest and least-weighted" nodes out of RAM into LTM temporarily.
- **LTM-to-WM retrieval.** If the robot returns to an old area and feature extraction finds a link to a past node, the algorithm automatically retrieves the neighbouring nodes from LTM back into WM to connect the map structure and correct accumulated drift.

Memory-constrained graph optimisation

Unlike ordinary SLAM systems, which must optimise the graph over the whole map and stall when run for a long time, RTAB-Map optimises the graph (through g2o or GTSAM) only over the nodes remaining in working memory. The NRU-51V edge processor can therefore run the navigation system and update the warehouse map smoothly over its whole service life without loss of speed.

2.3 Real-Time Object Detection

2.3.1 The role of object detection in semantic understanding

In traditional geometric visual SLAM the algorithm extracts geometric features without knowing what kind of object they belong to. For a warehouse that changes continually, geometry alone is not enough: the system needs semantic understanding to distinguish permanent structure from movable dynamic obstacles (goods, pallets, people). This project therefore integrates object detection into the front end to produce bounding boxes and pass the coordinates of dynamic objects to the SLAM system, which masks out the unstable features.

2.3.2 The YOLO (You Only Look Once) network architecture

For the system to respond in real time while the SLAM system is running, the project concentrates on the YOLO family, a one-stage detector distinguished by its processing speed. Unlike two-stage models (such as Faster R-CNN) that first generate region proposals and then classify them, YOLO predicts bounding-box regression and class probabilities for the whole image in a single forward pass, which makes YOLO (for example YOLO26) well suited to deployment on a resource-limited edge processor such as the NRU-51V board. Table 2.1 compares the main detector architectures.

Table 2.1 Comparison of object-detection model architectures.

Architecture	Category	Main strengths	Limitations	Suitable use
Faster R-CNN	Two-stage detector	Excels in classification and localization, achieving high mAP and IoU with a low overall error rate and high spatial accuracy	The multi-stage process slows inference	Work requiring high accuracy without real-time processing
YOLO series	Single-stage detector	Casts detection as a single-stage regression problem tuned for speed, balancing speed and accuracy well, with high precision and recall and a low false-positive rate	Although now well refined, early versions had trouble with small or densely clustered objects compared with two-stage models	Real-time work requiring high processing speed and low latency
SSD (Single Shot Detector)	Single-stage detector	Removes the region-proposal step and performs multi-scale detection within one network	Sometimes slightly slower than YOLO, but still very efficient	Applications needing a reliable balance of throughput and accuracy
MobileNet / EfficientNet	Lightweight model	Uses techniques such as depthwise-separable convolutions to cut parameters and computation substantially without reducing learning capacity	Overall accuracy may be below large models such as Faster R-CNN or the larger YOLO versions	Edge devices with limited memory and compute: smartphones, IoT devices, drones

2.3.3 Performance evaluation and edge-processor limits

In bringing a YOLO model into a visual SLAM pipeline, the essential point is the trade-off between accuracy and inference time (latency): too large a model creates a bottleneck and makes the SLAM system fall behind the camera frame rate. The evaluation of the YOLO model on a simulated warehouse dataset, through a platform such as Roboflow, therefore uses the following main metrics:

- **Intersection over union (IoU)**: the overlap ratio between the predicted box and the true object box, measuring localization accuracy.
- **Precision**: the fraction of dynamic-object predictions that are correct, out of all dynamic-object predictions (accuracy of the prediction).
- **Recall**: the fraction of dynamic objects correctly predicted, out of all dynamic objects actually present in the image (coverage of the search).
- **F1 score**: the harmonic mean of precision and recall, measuring the model's balance.
- **Average precision (AP) and mean average precision (mAP)**: the overall accuracy reflecting the model's combined performance across all object types under differing confidence thresholds.

Choosing the model with the highest mAP that still keeps latency within a safe budget (for example $<$15–20 ms) is the key to a complete integration of YOLO and SLAM.

2.4 RY-SLAM: A Robust YOLO-Based Real-Time Visual SLAM Solution for Dynamic Environments

2.4.1 Concept and architecture of RY-SLAM

Chen et al. [8] present RY-SLAM (Robust YOLO SLAM), an advanced visual SLAM system designed to solve navigation failure in dynamic environments. RY-SLAM integrates the ORB-SLAM3 framework with a YOLO-family semantic detection and segmentation network (for example YOLOv8s-seg). Its architecture relies on multi-threaded parallel processing with GPU acceleration: the semantic-segmentation thread is separated from the SLAM tracking thread, so the system can analyse dynamic-object coordinates and estimate the camera pose at the same time without losing real-time throughput (figure 2.8).

2.4.2 Dynamic feature masking mechanism

The heart of RY-SLAM's localization accuracy is its dynamic feature masking, in three technical steps:

- **Semantic mask generation.** The YOLO model analyses the input frame to detect dynamic objects (such as people or vehicles) and generates a mask around each object's area.
- **Outlier elimination.** In the ORB-SLAM3 feature-extraction step the system checks the position of each ORB feature; any feature falling inside a dynamic-object mask is deleted immediately.
- **Static pose estimation.** Only features extracted from the permanent structure of the environment (static background) are fed to bundle adjustment, reducing accumulated drift and tracking loss significantly compared with standard ORB-SLAM3.

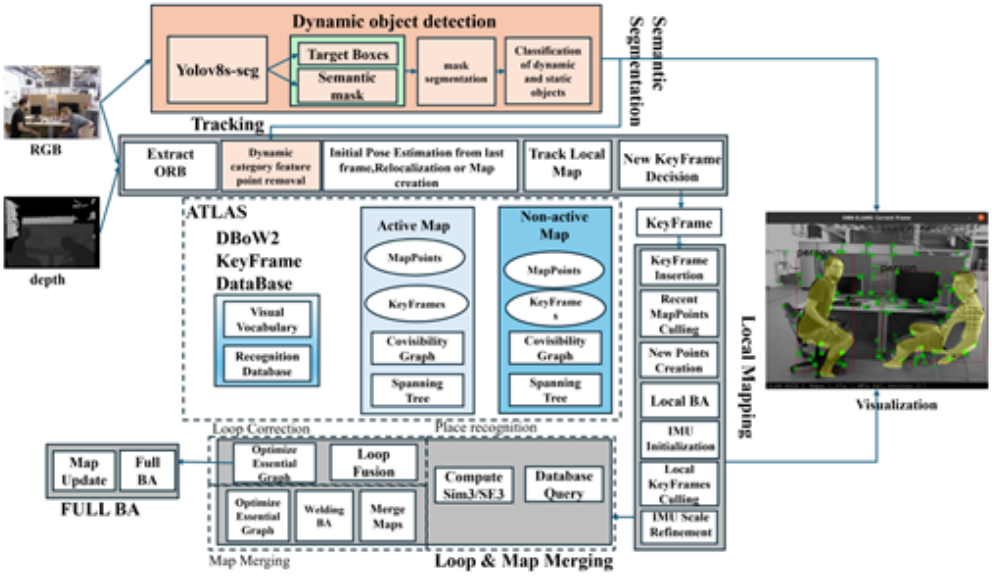


Figure 2.8 Architecture of RY-SLAM: a YOLOv8-seg dynamic-object detection stage feeding a semantic mask into the ORB-SLAM3 tracking thread ahead of the Atlas multi-map system [8].

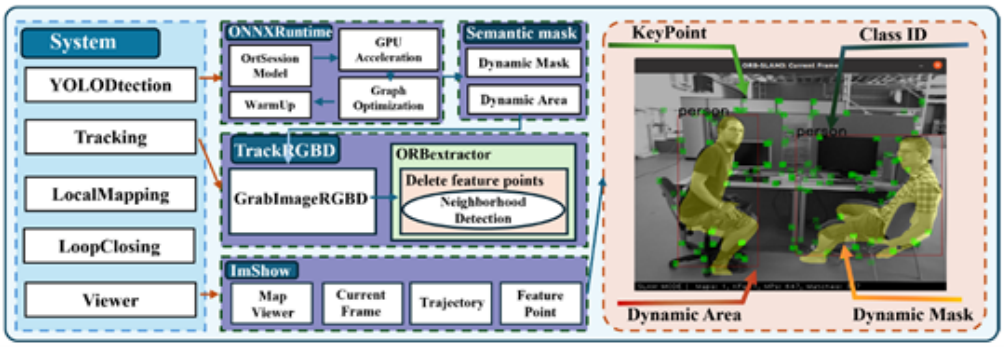


Figure 2.9 Detail of the semantic-segmentation framework in RY-SLAM and its multi-threaded operation: ONNX Runtime inference producing the dynamic mask, the neighbourhood test that deletes feature points, and the viewer outputs [8].

2.4.3 Relevance to this project

RY-SLAM is important academic evidence for the effectiveness and soundness of combining a YOLO-family model with ORB-SLAM3. For this project the author adopts the basic concept and architecture of RY-SLAM and develops it further in pipeline 1 (YOLO26 + ORB-SLAM3), raising system performance by choosing the newer YOLO26 model to filter obstacles and mock goods in the warehouse in real time, together with the Atlas multi-map architecture of ORB-SLAM3 on the NRU-51V edge processor, so as to build a navigation system that is robust both to close-proximity dynamic objects and in long-term (lifelong) mapping, as the objectives require.

CHAPTER 3 METHODOLOGY

To let the system solve the problem set out in chapter 1 and be deployed on an edge AI computer, this project integrates a neural object detector (YOLO26) into the core of a visual SLAM framework, a combination referred to as semantic visual SLAM. This chapter describes the problem and its scope, the system architecture, the two navigation pipelines as they were actually built, the simulation and real platforms, the four experiments that structure the evaluation, the metrics, and the points at which the implemented system departs from the method originally proposed.

Reporting cutoff. Every implementation claim in this chapter describes the repository at 7 November 2026. Components are labelled **implemented**, **partial**, **blocked** or **planned**; nothing planned is described in the present tense as though it existed.

3.1 Problem, Requirements and Scope

The engineering problem is to localize a warehouse AMR with cameras and an IMU, without floor infrastructure, in an environment whose contents move. The company's present LiDAR-and-QR-code navigation is accurate but expensive to install and inflexible (section 1.1); a camera-based replacement is attractive only if it stays accurate when goods, pallets and people move through the field of view, which is exactly the case standard visual SLAM assumes away.

Performance targets. The project states two trajectory targets for the integrated system at the robot's working speed of $1.5\,\mathrm{m\,s^{-1}}$: an absolute trajectory error of at most 0.05 m, and a relative pose error over 1 s of at most 1.5 m. The original objective list (section 1.2) states the localization requirement as an error below 10 cm under differing illumination. Both figures are carried into chapter 4 and every trajectory result is read against them.

Site. The real tests take place on the first floor of the Gensurv building, in a mock warehouse of roughly 10 m by 7 m divided into two operating zones, Zone A and Zone B, each about 3 m square and joined by a walkway (figure 1.2). The walkway carries a 5° slope. Goods are 0.3 m cubes in stacks whose positions are changed between runs, and a light-control fixture sets the illumination to a schedule. A motion-capture system provides the ground-truth reference (section 3.5.4).

Two domains. Every experiment is defined once and run twice, in a Gazebo simulation of the warehouse and on the real robot. The simulation supplies exact ground truth, exact calibration and repeatable dynamic scenes; the real robot supplies the sensors, the lighting and the physics the project is ultimately about. The two domains are never pooled.

What is excluded. Path planning and motion control are outside the project: the pipeline delivers a pose and a map to an existing planner and controller (figure 1.1). Of the five original objectives, this report addresses objectives 1, 3 and 5 (visual-inertial localization with mapping, dynamic-object detection, and the integrated localization requirement). Objective 2 (EnlightenGAN low-light enhancement) and objective 4 (PolyNET lane detection with PID lane tracking) were not pursued within the reporting period and are recorded as deviations in section 3.8.

3.2 System Architecture

3.2.1 Control-loop view and data rates

Figure 1.1 places the project inside the AMR's control loop. The controller issues control input at 20 Hz; the SLAM front end returns an odometry pose at 20 Hz; the object detector returns bounding boxes at 10 Hz on the real robot; the IMU streams at 200 Hz and the cameras at 20 Hz. In simulation the cameras run at 30 Hz and detections are produced per frame, and the wheel odometry of the simulated platform is published at 50 Hz (table 3.11). The four experiments of section 3.6 are each located on one block of this loop:

Table 3.1 The four experiments and the architecture block each one tests.

#	Experiment	Block under test
1	Non-visual baseline and estimator comparison	the robot's own odometry against the SLAM output: is vision earning its place?
2	Offline YOLO detector performance	the object-detection block, in isolation
3	Visual-inertial odometry with dynamic-environment filtering	the SLAM front end, with and without the detector's mask
4	Dense map reconstruction and map-quality evaluation	the SLAM back end and the map it delivers

3.2.2 Semantic visual SLAM data flow

The architecture is organised in two parts, front-end sensor processing and back-end SLAM and mapping.

- **Front-end processing pipeline.** *Sensor data ingestion* receives raw images from the wide-angle stereo cameras (Sony IMX390, 186° field of view) together with acceleration and angular rate from the IMU, through ROS 2 nodes. The *dynamic object detection module* passes each image to the YOLO26 model to detect, in real time, movable warehouse objects (goods, pallets, people) and emits their bounding boxes. The *dynamic feature masking protocol* compares those boxes with the geometric keypoints extracted by the SLAM front end (ORB or FAST corners); any keypoint that falls inside a dynamic object's box is masked and discarded, so that only features of the permanent warehouse structure remain.
- **Back-end SLAM and mapping.** The filtered features enter odometry and graph optimisation to estimate the 6-DoF robot state, update the 3-D point map of the permanent structure, and build the 2-D occupancy grid the navigation system consumes.

Figure 3.1 shows the data flow of the system as it exists.

As shown in figure 3.1, the current work contains two related but separate evaluation paths. The first measures front-end trajectory behaviour with and without semantic feature filtering. The second studies map reconstruction and external global correction through RTAB-Map. RTAB-Map's `map` → `odom` transform does not modify the internal feature map, state history, or previously recorded trajectory of ORB-SLAM3 or VINS-Fusion. A unified architecture for relocalization, global optimization, and multi-session map merging is **planned**, not implemented. Path planning is outside the present research scope.

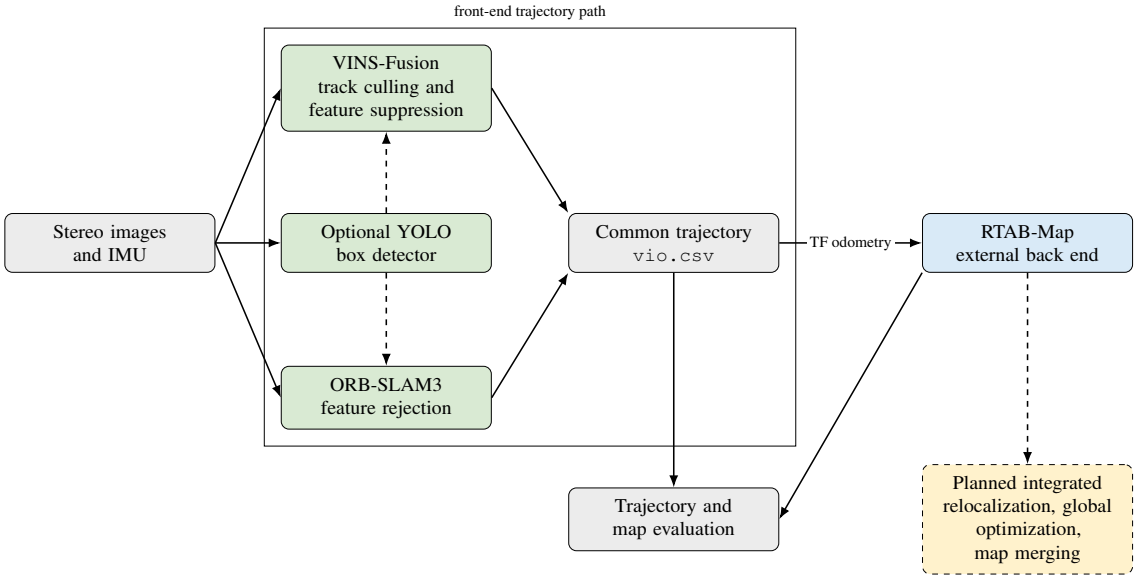


Figure 3.1 Implemented experimental architecture. Green blocks are implemented project components, blue denotes the external RTAB-Map back end, and the dashed yellow block is future work. The diagram deliberately does not show a feedback connection from RTAB-Map into either front end because that integration has not been implemented.

Table 3.2 Implementation status at the reporting cutoff.

Component	Status	Scope of evidence
Warehouse simulation and controlled worlds	**Implemented; verified**	The two realistic layouts and their static twins are generated by `script/make_realistic_world.py` and `make_static_world.py`, each with baked occupancy truth.
ORB-SLAM3 ROS 2 stereo-inertial wrapper	**Implemented; verified in both domains**	Live topics, worker-thread tracking, TF and trajectory output. The fork carries fifteen patches, among them five out-of-bounds reads in the mapping thread that caused the real-rig crashes.
ORB-SLAM3 GPU front end	**Implemented; verified**	The feature extractor and stereo matcher run on the GPU through a run-time-compiled kernel and need no CUDA toolkit; selected by configuration and measured in table 4.14.
ORB-SLAM3 semantic filtering	**Implemented; mechanism verified; untested**	Detector messages reach the tracker and masked candidates are rejected before octree distribution, but no filtered run exists in the current dataset set.

Component	Status	Scope of evidence
VINS-Fusion stereo-inertial estimator	**Implemented; verified in simulation**	Simulation and real-rig configurations, odometry publication, TF and trajectory output. On the real rig it has no valid run; two delivery and configuration defects are diagnosed in section 4.3.2.
VINS-Fusion semantic filtering	**Implemented; mechanism verified; untested**	Existing KLT tracks are culled and new features suppressed in detected regions; as above, no filtered run exists.
Offline YOLO evaluation	**Implemented; evaluated**	Ground-truth projection, IoU matching, per-frame records and overlays, applied to the four realistic recordings with the four-class checkpoint (section 4.2.1).
RTAB-Map construction and saved-map localisation	**Implemented; partially verified**	Five databases scored for geometry and drivability (section 4.4.1). Localisation against a closure-free prior does not work until the loop-closure threshold is restored, which is fixed; arbitrary-start aliasing rejection is untested.
RTAB-Map extend and multi-session merging	**Implemented at launch level; experimentally unverified**	The mode exists; no multi-session experiment has been run.
Live VINS-Fusion and RTAB-Map co-execution	**Implemented; verified**	The combined workload runs on the current computer with active cooling.
Trajectory and compute evaluation	**Implemented**	ATE and RPE, front-end, back-end, local-mapping and loop-closing timing, delivery and state counters, process CPU and resident memory are logged for every run and reported in table 4.14. External-process monitoring and a repeated hardware matrix remain planned.
Non-visual baseline, real robot	**Implemented; provisional**	Wheel-plus-gyroscope dead reckoning with a fitted speed scale (section 3.6.1); no filter, because the rig publishes no steering measurement.

The repository state used for this draft is superproject commit `32baaed876cabbddc19b0d89eb14a69bb6932d7e`. The checked-out submodules are VINS-Fusion `247b640eee04ecf74fa64b42f3696a53ad040d5b`, RTAB-Map ROS `b06adfa8cfd4434ca2a6b47c4dfd23e388372273`, the warehouse environment `601c2fd3bf3bd6766602c719240e6f316941f9ac` and the ORB-SLAM3 core `a639fa462d064587dff867335503362dde9fb20b`. Every run records the superproject commit in its manifest, so a result traces to the code that produced it.

3.2.3 The two navigation pipelines compared

The project compares two pipeline architectures built on that front end.

Pipeline 1: YOLO26 + ORB-SLAM3 (Atlas multi-map system). A tightly-coupled visual-inertial SLAM that performs both odometry and map construction through one graph. Lifelong mapping relies on ORB-SLAM3's Atlas framework: when tracking is lost because the environment changed, the system opens a new active map and keeps the old one as non-active; when the robot returns to a known area the maps are merged and joined by loop closure across maps. Its strength is high accuracy from tight image–IMU fusion, suited to areas with clear geometric features (figure 3.2).

Pipeline 2: YOLO26 + VINS + RTAB-Map (decoupled VIO and memory management). A decoupled architecture that separates high-rate front-end odometry from long-term back-end memory management. The front end runs VINS sliding-window visual-inertial odometry on the static features from YOLO26, giving low-latency odometry that remains stable when the robot vibrates or drives the 5° slope. Lifelong mapping is handled by RTAB-Map, which holds its working memory at a constant size to bound computation time and moves old nodes to long-term memory on disk. Its strength is stable use of hardware resources: it prevents RAM exhaustion on a small processing board during long continuous operation (figure 3.3).

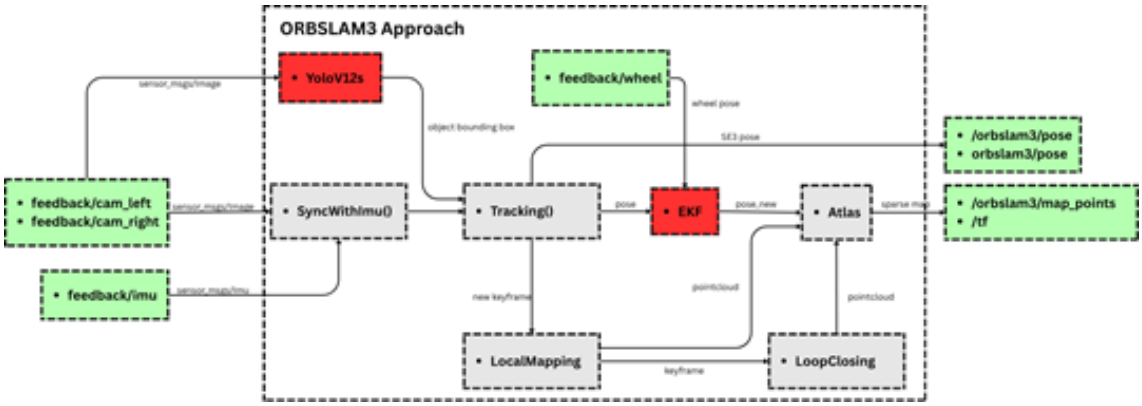


Figure 3.2 Pipeline 1, YOLO26 + ORB-SLAM3, as designed: stereo images and IMU are synchronised, tracked and passed to local mapping, loop closing and the Atlas map; the detector's boxes enter the tracking stage. The EKF block that fuses the SLAM pose with wheel feedback is a planned integration point and is not exercised in this report.

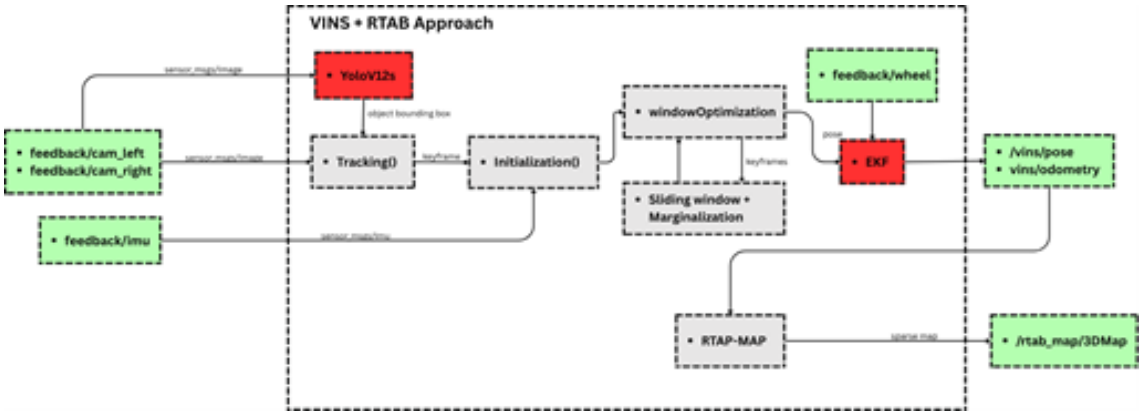


Figure 3.3 Pipeline 2, YOLO26 + VINS + RTAB-Map, as designed: VINS tracking, initialisation and sliding-window optimisation produce odometry, RTAB-Map builds the map from that odometry and the stereo images. As in figure 3.2, the EKF fusion with wheel feedback is planned.

Table 3.3 Design comparison of the two pipeline architectures, as proposed.

Aspect	Pipeline 1: YOLO26 + ORB-SLAM3	Pipeline 2: YOLO26 + VINS + RTAB-Map
System structure	tightly coupled (centralised)	decoupled (odometry separate from back end)
Odometry	ORB-SLAM3 tracking thread; tightly-coupled MAP estimation over visual and IMU terms	VINS-Fusion local VIO; sliding-window optimisation
Lifelong-mapping mechanism	Atlas multi-map system and map merging (Sim(3))	RTAB-Map memory management (WM / LTM node transfer)
Computation-time bound	grows with the number of keyframes and sub-maps in Atlas	held at a constant $O(1)$ ceiling in working memory
RAM / CPU use	grows with accumulated map size	bounded through the processing-time threshold $T_{\max}$
Robustness to vibration	moderate to high; relies on ORB features and IMU pre-integration	very high; online temporal and spatial calibration in VINS

3.3 The Pipelines as Implemented

3.3.1 Dynamic feature masking

In a dynamic warehouse a conventional SLAM algorithm accumulates drift if it uses features on moving objects to compute distance. The project therefore designs a shared front end in which the detector's output filters the geometric features before they reach the back end. When the stereo camera delivers frame I_t at time t, the image is passed to the YOLO26 network, which returns a set of bounding boxes

$$\mathscr{B} = \{B_1, B_2, \ldots, B_K\}, \qquad B_k = \{x_k, y_k, w_k, h_k, c_k, P_k\}, \tag{3.1}$$

where (x_k, y_k) is the box centre, (w_k, h_k) its width and height, c_k the class label and P_k the confidence. Boxes with $P_k < \tau_{\text{conf}}$ are rejected to limit false positives. Meanwhile the SLAM tracking thread extracts the frame's feature set $\mathscr{F} = \{f_1, \ldots, f_N\}$ with $f_i = (u_i, v_i)$. A feature is labelled dynamic when

$$\exists B_k \in \mathscr{B} : \left(x_k - \tfrac{w_k}{2} \le u_i \le x_k + \tfrac{w_k}{2}\right) \wedge \left(y_k - \tfrac{h_k}{2} \le v_i \le y_k + \tfrac{h_k}{2}\right), \tag{3.2}$$

and is removed immediately, leaving the static feature set

$$\mathscr{F}_{\text{static}} = \{\, f_i \in \mathscr{F} \mid f_i \notin B_k \ \forall k \,\}, \tag{3.3}$$

which is passed to feature matching and re-projection-error minimisation. The following subsections state where in each estimator that test is actually applied, because the two front ends re-use features differently and a single insertion point is not sufficient for both.

3.3.2 ORB-SLAM3 and semantic feature rejection

The project provides a purpose-built ROS 2 Humble wrapper around the pinned ORB-SLAM3 core. Approximate-time synchronization pairs left and right images with a 0.02 s slop. Raw and compressed transports are supported. Image callbacks place stereo pairs in a one-frame queue; a worker thread calls `TrackStereo()` so blocking visual tracking does not starve the 200 Hz IMU subscription. If tracking falls behind, the older pending image is discarded and the drop count is reported.

With `filter:=false`, no detection subscription is created and an empty mask passes through the patched interfaces. With filtering enabled, a bounded buffer is searched for the detection closest in source-image time, accepted only within 0.15 s. Boxes are expanded by eight pixels, clipped to the image, and rasterized to an 8-bit mask in which 255 retains a feature and zero rejects it. A mask covering more than 0.8 of the frame is discarded as a safety measure.

The mask passes through `System`, `Tracking`, and `Frame` to the ORB extractor. Candidate keypoints are tested before octree distribution, so the per-level feature quota can be redistributed across surviving static regions. At each pyramid level, the implemented boundary rule rejects a candidate when at least five samples in a 3×3 neighbourhood are dynamic. Only left-camera features are masked because ORB-SLAM3 establishes stereo matches from left keypoints to right candidates.

The current simulation YAML requests 1500 ORB features, eight pyramid levels, a 1.2

scale factor, initial FAST threshold 20, and fallback threshold 7; table 3.4 collects these. An earlier draft of this section reported 1000 and attributed 1500 to a superseded engineering note. That was backwards: `ORBextractor.nFeatures` is 1500 in the committed configuration, and no uncommitted change touches it. No controlled feature-count or FAST-threshold sweep is complete.

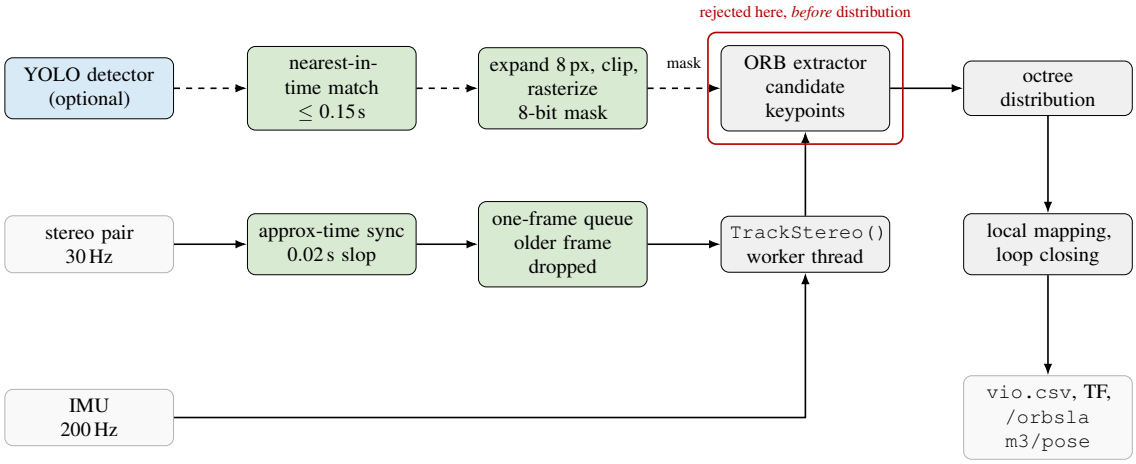


Figure 3.4 ORB-SLAM3 pipeline and the semantic-mask insertion point. Green is built by this project, grey is the pinned upstream core, blue the optional detector path. The mask is applied to candidate keypoints *before* octree distribution, which is what allows the per-level feature quota to redistribute onto surviving static regions rather than simply shrink. A candidate is rejected when at least five samples of its 3×3 neighbourhood are dynamic, and only the left camera is masked, because ORB-SLAM3 forms stereo matches outwards from left keypoints.

The wrapper publishes `/orbslam3/odometry`, `/orbslam3/pose`, and TF after valid tracking. It writes the common 11-column trajectory format directly and flushes each row. Repository evidence is located in `orbslam3_ros2/src/orbslam3_node.cpp`, `orbslam3_ros2/src/slam_wrapper.cpp`, `orbslam3_ros2/src/trajectory_writer.cpp`, and `orbslam3_ros2/config/wil_sim/stereo_imu.yaml`.

3.3.3 VINS-Fusion front end

VINS-Fusion uses stereo features and inertial measurements in a sliding-window nonlinear estimator. The ROS wrapper synchronizes the raw image topics and uses separate callback groups for images, IMU, feature messages, and detections. The simulation configuration processes every image (`image_skip: 1`), uses two cameras and IMU, fixes simulated extrinsics, targets 25 Hz feature publication, and uses 10-pixel keyframe parallax. Image queues were increased to 100 and the IMU queue to 2,000 to reduce losses during initialization and backlog.

VINS cannot use only the ORB strategy because it propagates existing features with pyramidal KLT optical flow and detects new features only for replenishment. A feature attached to an object before detection could otherwise persist after masking. The implementation initializes `FeatureTracker::setMask()` from the YOLO mask. Tracks in zero-valued regions fail the retention test, while the same mask is passed to `goodFeaturesToTrack()` to suppress new points. The count of persistent tracks removed in dynamic regions is accumulated as a mechanism diagnostic.

Table 3.4 ORB-SLAM3 configuration at the cutoff. Wrapper values are from `orbslam3_ros2/src/`; extractor and IMU values from `config/wil_sim/stereo_imu.yaml`. Noise densities are given in table 3.10 and are not repeated here.

Parameter	Value	Role
Wrapper (built here)		
Stereo sync slop	0.02 s	approximate-time pairing
Image queue depth	1 frame	older frame dropped, drop count logged
Detector match window	0.15 s	nearest in source-image time
Box expansion	8 px	before rasterizing the mask
Mask reject threshold	0.8 of frame	safety cut-out on runaway coverage
Boundary rule	≥ 5 of 3×3	candidate rejected when dynamic
Masked camera	left only	stereo matches form from left keypoints
Extractor and core		
`ORBextractor.nFeatures`	1500	per-frame feature budget
`ORBextractor.nLevels`	8	pyramid levels
`ORBextractor.scaleFactor`	1.2	inter-level scale
`ORBextractor.iniThFAST`	20	initial corner threshold
`ORBextractor.minThFAST`	7	fallback threshold
`Stereo.ThDepth`	90.0	close/far stereo split
`IMU.Frequency`	200 Hz	preintegration rate
`loopClosing`	1	comment in the same file says "set to 0 deliberately"

Table 3.5 VINS-Fusion simulation configuration at the cutoff, from `vins_fusion_ros2/config/wil_sim/stereo_imu.yaml`. Inertial noise densities are in table 3.10.

Parameter	Value	Role
`num_of_cam` / `imu`	2 / 1	stereo-inertial
`image_skip`	1	every image processed
`max_cnt`	150	tracked-feature budget
`min_dist`	30 px	minimum feature separation
`freq`	25 Hz	target feature publication
`keyframe_parallax`	10 px	keyframe decision
`F_threshold`	1.0	RANSAC fundamental-matrix gate
`max_solver_time`	0.08 s	Ceres budget per update
`max_num_iterations`	100	Ceres iteration cap
`estimate_extrinsic`	0	simulated extrinsics held fixed
`estimate_td`	0	camera–IMU delay held at 0 s
`g_norm`	9.8 $\mathrm{m\,s^{-2}}$	matches the world's gravity
Image queue	100	raised to survive initialization backlog
IMU queue	2000	raised for the same reason

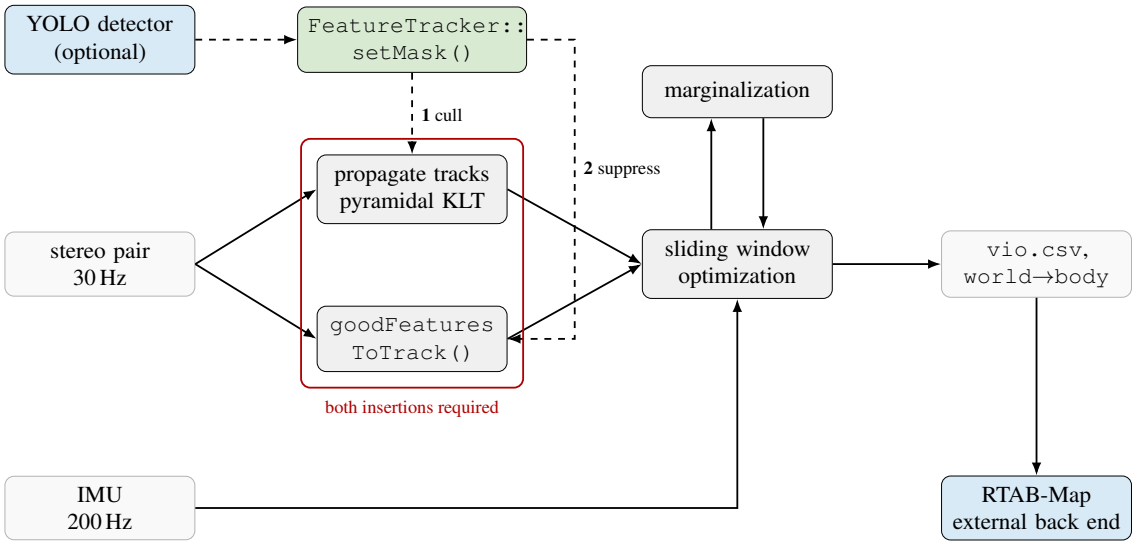


Figure 3.5 VINS-Fusion pipeline and its two semantic-mask insertion points. Because the front end propagates existing KLT tracks and only detects new features to replenish them, suppressing new detections alone is insufficient: a track that attached to an object before it was detected would survive masking. Tracks in masked regions must therefore also be culled (1) while the same mask suppresses replenishment (2). Contrast figure 3.4, where re-extraction every frame makes a single insertion sufficient. The count of persistent tracks removed is logged as a mechanism diagnostic — the counter that read zero in all twelve filtered runs.

VINS writes the optimized state after each estimator update to `vio.csv` and broadcasts `world` → `body` using configurable frame names. The ROS `output_path` parameter overrides the YAML path, recursively creates missing directories, and reports an explicit error if `vio.csv` cannot be opened. This behaviour is an uncommitted worktree change at the cutoff. Evidence is in `vins_fusion_ros2/src/vins_estimator.cpp`, `vins_fusion_ros2/vins/src/featureTracker/feature_tracker.cpp`, and `vins_fusion_ros2/vins/src/estimator/estimator.cpp`.

3.3.4 RTAB-Map mapping and localization back end

RTAB-Map receives front-end odometry through TF and stereo imagery through `rtab_camera_shim.py`. The shim reconstructs `CameraInfo`, decompresses the pair, and publishes validated static camera transforms from the common simulation calibration. TF was selected over an odometry topic because adding high-rate odometry to multi-topic approximate synchronization starved the lower-rate images. The cost is that odometry covariance is not supplied to graph edge weighting.

The back end uses stereo SGBM (`Stereo/DenseStrategy: 1`), height-based ground separation, ray tracing, spatial noise filtering, and odometry-derived gravity links. Its measured default depth range is 10 m. Because the featureless floor provides essentially no stereo ground points, ray tracing supplies free-space evidence between the sensor and observed obstacles. These settings affect occupancy assembly; visual-word loop closure is separate.

Three modes are provided by one launch file:

construct starts a new incremental database;
extend opens an existing database, places prior nodes in working memory, and permits

new nodes; and

localize disables incremental memory, loads existing nodes, serves the saved map, and can enable `Mem/LocalizationReadOnly`.

Localization still extracts features from current stereo images and matches stored RTAB-Map signatures. Read-only mode prevents database modification; it does not disable perception. The outputs are an external `map` → `odom` correction and `/localization_pose`. Neither RTAB-Map features nor corrected poses are injected into ORB-SLAM3's Atlas or VINS-Fusion's sliding window.

The composed localization launch raises `RGBD/OptimizeMaxError` from 3.0 to 10.0 because the default rejected all loop closures proposed with the observed VINS drift. This is effective operationally but is not yet proven safe against perceptual aliasing in repeated warehouse bays. Occupancy creation is disabled in localization mode because the stored map is reused. Live VINS-Fusion and RTAB-Map co-execution nevertheless remains blocked on the current machine. Recorded `vio.csv` can be replayed as `odom` → `vio_body` for a deterministic back-end test, but this is not proof of real-time co-execution.

Evidence is in `rtabmap_ros/rtab_sim.launch.py`, `rtabmap_ros/rtab_camera_shim.py`, `aws-robomaker-small-warehouse-world/launch/localization_rtabmap.launch.py`, and `script/gt_tf_publisher.py`.

3.3.5 YOLO detector node, mask construction and synchronisation

The semantic front end is a standalone Python ROS 2 node using Ultralytics. The project checkpoint is `weight/best.pt`, a custom detector for `bin`, `box`, and `bucket`, fine-tuned from `yolo26m.pt` using 640-pixel inputs. It produces bounding boxes, not segmentation masks. The stock `yolo26n.pt` and `yolo26m.pt` files are not project-trained models and are not evidence of custom training.

At startup, the node resolves requested classes against checkpoint metadata, warms the model with three dummy images, and selects CUDA when available. Current defaults are confidence 0.35, NMS IoU 0.5, and image size 640. Only the left image is processed. Each `Detection2DArray` preserves the source image header, so nearest-timestamp selection measures sensor-time correspondence rather than inference-completion time. Empty arrays distinguish a valid frame with no detections from detector failure.

The detector intentionally remains outside stereo synchronization. A three-input image/image/detection synchronizer would discard pairs whenever inference lagged. The implemented design continues without filtering if no recent detection exists. Filtered runs write `yolo_mask.csv` with timestamp, box count, coverage, match state, detection age, and rejection status.

Offline evaluation in `script/yolo_eval.py` transforms Gazebo model poses and collision-mesh vertices into the left-camera frame to form projected truth boxes. Object speed over $0.05\,\mathrm{m\,s^{-1}}$ defines the moving subset. Predictions are greedily matched by class at IoU 0.5. Heavily occluded truth boxes and predictions covered by known but unlabeled box-like scenery are ignored at stated thresholds. Outputs are per-frame `yolo_eval.csv`, per-detection `yolo_detections.csv`, and overlays. Overlays are inspected before aggregate metrics because invalid pose stamps, centimetre mesh units, or malformed

camera transforms can produce plausible but invalid numbers.

The current semantic rule assumes all selected-class detections are dynamic. No optical-flow, stereo-depth, or world-motion gate is implemented, and axis-aligned boxes also remove background pixels. These are explicit method limitations.

3.3.6 Navigation and relocalisation integration: the `vslam_navigation` package

The bring-up of the three components above is collected into one ROS 2 package, `vslam_navigation`, the instrument used to answer the second half of Experiment 4's question: whether the reconstructed map is usable by a route planner. The two front ends and the mapping back end are separate processes with different frame conventions, different image transports and, on the real rig, a different camera model; operating them by hand produced several invalid runs. Path planning remains outside the scope of this project (section 1.4): the planner appears only as the consumer that makes the map's usability measurable, and no navigation result is reported in chapter 4.

Pose by transform rather than by topic

The back end is not told which front end produced its pose. RTAB-Map is given an `odom_frame_id` and reads the sensor pose from the transform tree rather than from an odometry message, logging `subscribe_odom = false` accordingly. One back-end configuration therefore accepts ORB-SLAM3 or VINS-Fusion unchanged. Each front end is told which frame names to broadcast, which avoids bridging three naming conventions with static identity transforms. The resulting chain has exactly one publisher per edge in both domains (section 3.4.4).

Manufacturing the stereo input

RTAB-Map's stereo pipeline requires a rectified image pair, both camera information messages and a static transform to the optical frames, and the recordings carry only compressed images. `vslam_navigation/scripts/rtab_camera_shim.py` constructs the missing inputs from the same calibration files the estimators read, so a single calibration remains the source of truth. The two domains need different treatment:

- **Simulation.** A Gazebo camera is an ideal pinhole: every distortion coefficient is zero and the two cameras share intrinsics, so the raw frames already satisfy what rectification would produce and are republished unaltered. This shortcut is valid only in simulation.
- **Real rig.** The cameras are approximately 180° Kannala–Brandt fisheyes (section 3.5.3), and RTAB-Map's stereo path has no equidistant model. Every pair is therefore remapped onto a virtual rectified pinhole pair of stated horizontal field of view. The rectification rotations come from `cv2.fisheye.stereoRectify`, but the projection matrices are constructed explicitly, because OpenCV's own projection degenerates towards zero focal length for a lens of this field of view. The remap is validated on-line by an epipolar statistic: the fraction of cross-checked ORB matches whose rectified rows agree to within two pixels at positive disparity.

Three memory modes, and why a prior must be built without closures

The back end is operated in one of three modes: *construct* writes a new database, *extend* appends a session to an existing one, and *localize* runs read-only against a prior with RTAB-Map's incremental memory disabled. Priors from the two mapping recordings are built *closure-free*: both detectors that can add a non-neighbour link are shut, the appearance-based one by raising its acceptance threshold beyond reach and the proximity one by disabling it. The reason is measured: on those recordings the accepted closures were false, and displaced an otherwise 0.15 m map by several metres (section 3.6.4). Visual words are still extracted, so the database remains usable for localisation; the cheaper alternative of disabling feature extraction would yield a database that can never relocalise.

RTAB-Map restores its parameter set *from the database*, so a closure-free prior returns with both detectors still disabled even when the launch requests closures. Relocalising against a prior *is* the acceptance of a loop closure against it, so in localize mode the effect is fatal and silent: the node starts, publishes a localisation pose, and never localises. The symptom is a covariance trace pinned at RTAB-Map's no-information value of 3×10^4, against roughly 0.02 for a successful fix. The launch now sets both parameters explicitly, since a run-time parameter overrides the stored one. Every localisation run made against a closure-free prior before 6 November 2026 is void and is excluded from this report.

Gating the planner on the localisation

A supervisor, `vslam_navigation/scripts/pose_gate.py`, holds the planner's lifecycle nodes inactive until three conditions hold together: an initialisation interval has elapsed, the correction between the map and odometry frames has moved less than a stated step over a number of consecutive samples, and the pose lies inside the prior graph's node bounds plus a margin. The gate exists because nothing in the localisation path reports its own failure, and a planner acting on a wrong pose is worse than one that does not act. It activates the stack through the lifecycle manager and pauses it again if the conditions stop holding. The same boolean is consumed by the command interface, so a lost localisation zeroes the robot's commands rather than merely stopping replanning.

Planner, controller and vehicle model

The planning layer is Nav2 [9] with three components chosen for a car rather than for a differential-drive robot: a kinematic search planner, a pursuit controller, and a velocity smoother that preserves path curvature. None is a contribution of this project; the configuration is reported because it decides whether a plan on the reconstructed map is one the vehicle can follow.

One constraint governs every value in table 3.6. A car cannot turn on the spot, so for a kinematic bicycle of wheel base L at steering angle δ the rear axle traces a circle of curvature

$$\kappa = \frac{\tan\delta}{L}, \qquad \kappa_{\max} = \frac{\tan\delta_{\max}}{L}, \tag{3.4}$$

and the bound is independent of speed. The planned frame is not the rear axle, however, so the radius that the planner must respect is the larger $R_b = \sqrt{R^2 + d^2}$, with $R = 1/\kappa_{\max}$ and d the longitudinal offset of `base_link` from the rear axle. Confusing the two radii produces plans the controller cannot track, and the steering plugin does not reject an infeasible command: it clamps the radius internally and the vehicle silently under-turns.

The planner is the Hybrid-A* variant distributed with Nav2, which searches the vehicle's kinematic state space rather than a grid of free cells and then refines the result [10]. Its motion primitives are Dubins curves [11], so every expansion is a path the vehicle could drive, and the search is forward-only. Reverse primitives are available but not enabled: the rig carries a forward-facing stereo pair, so reversing would move the robot in the one direction its sensors do not observe. The planner's minimum turning radius is set above the kinematic floor rather than at it, leaving the controller authority to correct tracking error; planning at the exact limit makes any lateral error unrecoverable.

The controller is Regulated Pure Pursuit [12], which bounds the commanded curvature geometrically rather than by tuning. It steers towards a carrot at lookahead distance L_d, emitting

$$\kappa = \frac{2y}{L_d^2}, \qquad |y| \le L_d \quad \Longrightarrow \quad |\kappa| \le \frac{2}{L_d}, \tag{3.5}$$

so a lookahead of at least $2R_b$ guarantees that no command exceeds what the vehicle can steer. That is why the lookahead distances in table 3.6 are long, and why shortening them to tighten cornering is counterproductive: it raises the curvature the controller may ask for. Rotate-to-heading is disabled because it commands a yaw rate at zero speed, which the bicycle model turns into full steering lock at a standstill. The velocity smoother scales all velocity components by one factor, so reducing speed leaves $\kappa = \omega_z / v_x$ and therefore the demanded steering angle unchanged.

Both costmaps carry the prior map and an inflation layer, and no live obstacle layer in either domain. The simulated rig has no lidar and the real one has none either, and the back end's local occupancy grids are disabled in localize mode because the dense stereo cost starved the front end. Anything that moved into the environment after the map was built is therefore invisible to the planner, which is a limitation of the instrument and not a property of the map (section 3.6.4).

The forward collision check is enabled on the real rig and disabled in simulation, and that asymmetry is deliberate but only half justified. In simulation it was measured to cause paralysis: with the lookahead of table 3.6 the check vetoed the controller's own corner-cutting against the inflation ring 155 times in one run and held the robot near $0.1\,\mathrm{m\,s^{-1}}$ with metres of real clearance. The real rig keeps it on, untested, because its prior map is the less trustworthy of the two and no run yet exists to show which failure matters more there.

Command interface

The two domains differ structurally, not by tuning. The simulated robot's steering plugin consumes a velocity command directly, so the planner's output reaches it unchanged. The real rig accepts a speed and a steering angle on separate topics, so `vslam_navigation/scripts/ackermann_bridge.py` inverts the bicycle model,

$$\delta = \arctan\left(\frac{L\,\omega_z}{v_x}\right), \qquad |\delta| \le \delta_{\max}, \tag{3.6}$$

with L the wheel base and $\delta_{\max}$ the steering limit. Three behaviours in that node are consequences of the platform rather than choices. The standstill case of section 3.3.6 is guarded a second time here: steering is forced to zero below a speed threshold. Both outputs are zeroed when no command has arrived within a timeout, so that a planner crash

Table 3.6 Navigation parameters for the two platforms, with the quantity each is derived from. Values are those in `params/nav2_sim.yaml` and `params/nav2_real.yaml` at the reporting cutoff.

Parameter	Simulation	Real rig	Derived from
Wheel base L [m]	0.42	0.60	measured
Steering limit $\delta_{\max}$ [°]	35	20	measured
Rear-axle bound $\kappa_{\max}$ [m^{-1}]	1.667	0.607	equation (3.4)
Planned-frame radius R_b [m]	0.636	1.754	$\sqrt{R^2 + d^2}$ [a]
`minimum_turning_radius` [m]	0.75	1.85	above R_b, for tracking margin
Motion model	Dubins	Dubins	forward-only
`angle_quantization_bins`	72	72	5° per bin
Lookahead, min to max [m]	1.30–2.00	3.55–4.50	$\geq 2R_b$, equation (3.5)
`desired_linear_vel` [$\mathrm{m\,s}^{-1}$]	1.50	1.50	project target speed
Yaw-rate limit [$\mathrm{rad\,s}^{-1}$]	1.00	0.91	$\kappa_{\max} v$
Acceleration, braking [$\mathrm{m\,s}^{-2}$]	0.80, −1.50	0.50, −1.00	chosen, not measured
Footprint, longitudinal [m]	−0.31 to +0.52	−0.52 to +0.13	measured extent
Footprint, lateral [m]	±0.25	±0.21	measured extent
`inflation_radius` [m]	0.80	0.80	exceeds padded R_b of the footprint
Live obstacle layer	none	none	no exteroceptive sensor
Forward collision check	off	on	asymmetric; see below

[a] The offset d differs by platform: `base_link` sits mid-wheelbase on the simulated robot and at the front axle on the real rig, so the two radii are not computed from the same lever arm.

mid-goal cannot leave the last command executing. The published speed is also capped in the robot's own command units, which are not metres per second and whose scale is known only at a single operating point (the speed scale is provisional; section 3.5).

Table 3.7 Tasks provided by the `vslam_navigation` package, one entry point per task per domain. Status is at the reporting cutoff.

Task	Entry point	Produces	Status
Build a prior	`mapping_sim` `mapping_real`	front end and back end together; a database and the optimised graph	implemented
Relocalise	`localize_sim` `localize_real`	read-only fix against a prior, and a per-sample record of pose, correction step and whether the pose is inside the prior's extent	implemented
Plan and drive	`navigate_sim` `navigate_real`	the above plus the planner, the localisation gate and the command interface	sim **untested**; real **blocked**

Status at the cutoff

The mapping and relocalisation tasks are **implemented** and are the ones the reported Experiment 4 results rest on. The planning task is **untested** in simulation: its planner, controller and behaviour tree have been exercised offline against a served prior grid and a stationary transform tree, which confirms that the configuration loads and that the commanded path curvature stays inside the chassis bound, but no closed-loop run in Gazebo has been completed. On the real robot it is **blocked**: the platform has been unavailable since the navigation layer was written, and the one command-unit scale that the bridge needs is fitted from a single operating point. Relocalisation on the real rig has been demonstrated once, against a prior that does contain closures, and is reported with that provenance (section 4.4.2).

3.4 Simulation Platform

3.4.1 Computing and software environment

One machine produced every simulation result in this report. Table 3.8 records it. Each entry was re-read from the running system on 20 September 2026 rather than copied forward from the engineering log, and all agreed.

Two consequences follow from the table rather than from any experiment. The proposed Jetson-versus-laptop study has not been instrumented or executed, so every timing, throughput and CPU figure in chapter 4 describes this machine alone and none supports a hardware comparison. And the detector runs a different OpenCV build from the one the estimators link against, which is unremarkable for two separate processes but would matter if image handling were ever moved across that boundary.

3.4.2 Simulated sensor rig and platform

The simulated Ackermann robot carries two ideal pinhole cameras and an IMU on a forward mast, and is driven through a front-steering, rear-driven four-wheel chassis. Figure 3.6

Table 3.8 Computing and software environment. A single laptop produced every result reported here; there is no second platform, so no entry below supports a cross-hardware comparison.

Component	Version	Role
Platform		
Operating system	Ubuntu 22.04.5 LTS	
Middleware	ROS 2 Humble	
CPU	16 logical cores	
Memory	15 GB, no swap	no swap, so memory pressure fails outright rather than degrading
GPU	NVIDIA RTX 3070 Laptop	detector inference only
Build toolchain		
GCC	11.4.0	
CMake	3.22.1	
OpenCV (system)	4.5.4	links both estimators
Pangolin	0.9.1	ORB-SLAM3 built out of tree
Detector environment		
PyTorch	2.5.1+cu121	
torchvision	0.20.1+cu121	
Ultralytics	8.4.138	YOLO training and inference
NumPy	1.26.4	
OpenCV (Python)	4.11.0	differs from the 4.5.4 the estimators link
Mapping back end		
`ros-humble-rtabmap`	0.23.7	apt core library
RTAB-Map ROS wrappers	project checkout	only packages through `rtabmap_slam` are built

gives its dimensioned plan and elevation, and tables 3.9 and 3.11 list the sensor and platform constants. Every value in the figure and both tables is read from `aws-robomaker-small-warehouse-world/models/ackermann_robot/model.sdf`, so the drawing is to scale rather than illustrative.

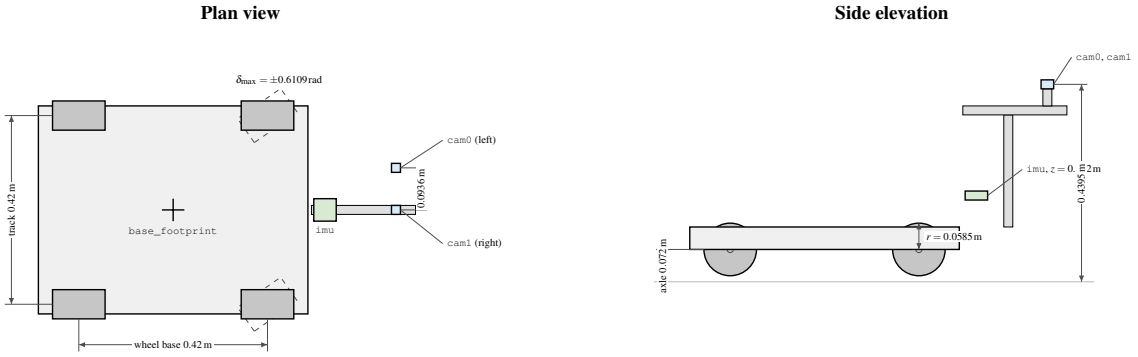


Figure 3.6 Simulated Ackermann platform, drawn to scale from the model definition. Distances are in metres relative to `base_footprint`. The stereo pair is mounted asymmetrically: `cam1` sits on the vehicle centreline and `cam0` is offset 0.0936 m to its left, so the optical centre of the pair is not the chassis centreline. The wheel base and track are equal at 0.42 m, which is what makes the rear-wheel differential a poor yaw-rate estimator (section 3.6.1).

Table 3.9 Simulated sensor configuration. Intrinsics are analytic rather than calibrated: the simulator renders an exact pinhole camera, so $f_x = f_y$ follow from the image width and the horizontal field of view and the distortion coefficients are identically zero. The stereo baseline was independently checked from positive disparity in a recorded bag rather than taken from the model alone.

Quantity	`cam0` (left)	`cam1` (right)	`imu`
Resolution	1280×720	1280×720	–
Rate	30 Hz	30 Hz	200 Hz
Horizontal FOV	$\pi/2$ rad (90°)	$\pi/2$ rad (90°)	–
$f_x = f_y$	640	640	–
(c_x, c_y)	$(640, 360)$	$(640, 360)$	–
Distortion	zero	zero	–
Position from `base_footprint` (m)	$(0.496, 0.0936, 0.4395)$	$(0.496, 0, 0.4395)$	$(0.338, 0, 0.192)$
Position from IMU body (m)	$(0.158, 0.0936, 0.2475)$	$(0.158, 0, 0.2475)$	$(0, 0, 0)$
Stereo baseline	0.0936 m		–

The fixed camera rotation converts the ROS body convention (forward, left, up) to the optical convention (right, down, forward). The VINS simulation configuration fixes the extrinsics (`estimate_extrinsic: 0`) and disables temporal-offset estimation (`estimate_td: 0`); the assumed inertial noise model is given in table 3.10, and the same values are mapped to the ORB-SLAM3 configuration.

The real platform's wheel base is 0.60 m (section 3.5) against the simulated 0.42 m, and the two are not interchangeable. Under each platform's own steering limit the simulated car turns about a 0.64 m radius at `base_link` and the real one about 1.75 m, so a route that fits the simulated robot need not fit the real one and the planner is configured separately for each (section 3.3.6). The real rig also publishes no steering measurement, so the bicycle model cannot be formed on it (section 3.6.1).

Table 3.10 Inertial noise parameters supplied to the visual-inertial estimators, beside the values measured from the four recordings in table 3.22. The configured values are inherited estimator defaults, not measurements of this simulated IMU.

Parameter	Configured (VINS, ORB)	Measured from the recordings	Units
`acc_n`	0.02	0.015–0.024	$\mathrm{m\,s^{-2}/\sqrt{Hz}}$
`gyr_n`	0.002	0.00030–0.00036	$\mathrm{rad\,s^{-1}/\sqrt{Hz}}$
`acc_w`	0.001	not measurable	$\mathrm{m\,s^{-3}/\sqrt{Hz}}$
`gyr_w`	0.0001	not measurable	$\mathrm{rad\,s^{-2}/\sqrt{Hz}}$

Table 3.11 Platform geometry and drivetrain limits used by the wheel-odometry and EKF baselines. `AckermannSteering` exposes no noise parameter, so the simulated encoders and odometry are noise free at source.

Quantity	Symbol	Value	Source field
Wheel base (front to rear axle)	L	0.42 m	`wheel_base`
Track (wheel separation)	B	0.42 m	`wheel_separation`
Kingpin width	–	0.42 m	`kingpin_width`
Wheel radius	r	0.0585 m	`wheel_radius`
Wheel width	–	0.065 m	wheel `<length>`
Steering limit	$\delta_{\max}$	± 0.6109 rad	`steering_limit`
Chassis mass	–	10.1 kg	`base_footprint` `<mass>`
Maximum speed	–	$10\,\mathrm{m\,s^{-1}}$	`max_velocity`
Maximum acceleration	–	$3\,\mathrm{m\,s^{-2}}$	`max_acceleration`
Odometry publish rate	–	50 Hz	`odom_publish_frequency`
Joint-state publish rate	–	1 kHz	measured from bag

3.4.3 Simulation fidelity and its limits

Every simulation result in this report rests on how faithfully the simulator reproduces the sensors and the contact physics. This subsection states what is modelled, what is idealised, and in which direction each idealisation biases the comparison between the visual and proprioceptive estimators.

Inertial measurement. Table 3.10 reports the noise measured from the recorded stream and queries it, because no recording contains a stationary interval long enough for a sound estimate: the longest is 2.16 s. That query can be closed without a new recording, because the noise is *declared* in the robot model rather than emergent, so the simulator's own configuration is the ground truth the measurement was trying to recover.

Table 3.12 collects what the simulator actually injects, together with the measured values and the EuRoC reference. The measured gyroscope range brackets the declared value, which corroborates both. The measured accelerometer range does not: it is four to six times the declared figure, and that discrepancy is the estimator's rather than the sensor's, because the residual on the driving axis carries real longitudinal acceleration that a window of two seconds cannot separate from noise. Prefer the declared values wherever a number is needed.

Table 3.12 Noise injected by the simulator, from the `<imu>` and `<camera>` sensor blocks of the robot model. Densities are per-sample σ divided by $\sqrt{f_s}$ at 200 Hz, in the channel's own units per $\sqrt{\text{Hz}}$. "Measured" is the range over the four recordings of table 3.22. The EuRoC column is the ADIS16448 of that benchmark, on which both visual estimators were published.

Channel (units of σ)	Declared σ	Density	Measured	EuRoC
Gyroscope, per axis ($\text{rad}\,\text{s}^{-1}$)	0.0048	3.394×10^{-4}	3.0×10^{-4}–3.6×10^{-4}	1.70×10^{-4}
Accelerometer, per axis ($\text{m}\,\text{s}^{-2}$)	0.0566	4.002×10^{-3}	1.5×10^{-2}–2.4×10^{-2}	2.00×10^{-3}
Camera, per pixel (normalised intensity)	0.005	—	not measured	—
Bias, first-order Gauss–Markov: σ_b and correlation time τ				
Gyroscope bias ($\text{rad}\,\text{s}^{-1}$)	3.8785×10^{-5}	$\tau = 1000\,\text{s}$	not measurable	—
Accelerometer bias ($\text{m}\,\text{s}^{-2}$)	0.006	$\tau = 300\,\text{s}$	not measurable	—

Two readings follow. The simulated IMU is *twice as noisy* as the EuRoC reference on both channels, so it is not an optimistic sensor. The camera, at about 1.3 grey levels out of 255 and with no distortion terms, is much cleaner than any real one. Both bias correlation times exceed the 80–125 s length of the four shorter recordings, so within a run the bias behaves as a slowly varying constant and its instability is never exercised; only the two mapping runs, at 288–315 s, approach the accelerometer's 300 s correlation time.

Three further effects are absent: chassis vibration, which on a real platform is frequently the dominant error; scale-factor, cross-axis and g-sensitivity errors; and temperature drift. Finally, the message's `orientation` field carries no noise block at all and is therefore ground truth; no estimator in this project consumes it, and none should (section 3.6.1).

Wheel encoders. `JointStatePublisher` reports the true solver angle. Slip con-

sequently appears as a *discrepancy* rather than an error: the joint angle stays true, so encoder-integrated distance over-reads whenever traction breaks, which is exactly a real encoder's behaviour. What is idealised is resolution. A physical 1024 CPR encoder quantises to 6.1×10^{-3} rad, setting a floor on low-speed velocity estimation that the simulation does not have. `script/encoder_sim.py` exists to impose that floor downstream and has not been applied to any result in this report.

Physics engine and solver. Table 3.13 records the simulation's physics configuration, which is identical across all 28 world files. Two entries deserve comment because they are easy to misread.

The `<physics>` element declares `type="ignored"`. That is not a misconfiguration: in Gazebo Fortress the engine is selected by the physics system plugin, not by that attribute, and the attribute is disregarded. No world passes an `<engine>` parameter to the plugin, so the engine is Fortress's default, `dartsim`. This matters for the friction discussion below, and it is the reason `slip1` and `slip2` have no effect.

No solver block is declared anywhere: no iteration count, no SOR parameter, no constraint-force mixing or error-reduction terms. Every one of those therefore takes the engine default. A 1 ms step is a fine integration step for a 1.5 m s^{-1} ground vehicle, but the untuned contact solver is the part of this setup least able to support a quantitative claim about contact behaviour.

Table 3.13 Gazebo physics configuration, identical in all 28 world files. Entries marked "engine default" are not declared anywhere in the project and are therefore whatever `dartsim` chooses.

Setting	Value	Note
Engine	`dartsim`	Fortress default; no world sets `<engine>`
`<physics type>`	`"ignored"`	disregarded in Fortress; the plugin selects the engine
`max_step_size`	1 ms	1 kHz integration
`real_time_factor`	1	no deliberate slow-down
`real_time_update_rate`	engine default	not declared
Solver type and iterations	engine default	no `<solver>` block anywhere
CFM / ERP	engine default	no `<constraints>` block anywhere
Gravity	-9.8 m s^{-2}	against ORB-SLAM3's hard-coded 9.81, a 0.1 % difference
`slip1`, `slip2`	0 on all grounds	an ODE feature; `dartsim` does not implement it, so inert at any value

Achieved real-time factor was not logged during recording, so the table states the requested factor rather than the realised one; the simulator in fact ran below real time, which is why a recording's ground-truth span is shorter than its wall-clock duration (table 3.18). Replay rate, which is logged, is a separate control and is 1.0 for every run in this report.

Contact friction, and why the encoder is flattered. The wheels declare $\mu = 1$, $\mu_2 = 1$ and every ground model declares $\mu = 100$, $\mu_2 = 50$. Both sets sit inside `<friction><ode>` elements, which is where SDF places them irrespective of engine, and

`dartsim` consumes them (table 3.13). How it combines two differing coefficients is not established here, so the effective value is not asserted; what is measured is the outcome in table 3.14.

Table 3.14 Measured wheel slip on the four recordings of Experiment 1. The ratio is encoder-integrated distance over true path length: above one is slip, below one is a rolling-radius error and cannot be slip. Measured by `script/check_slip.py`.

Recording	Encoder [m]	Truth [m]	Ratio	Reading
`realistic_000`	171.36	171.35	1.0001	no measurable slip
`realistic_001`	132.56	133.04	0.9964	radius error, not slip
`realistic_002`	102.81	102.68	1.0012	slip, 0.12 %
`realistic_003`	157.82	158.01	0.9988	radius error, not slip

Slip is below 0.4 % everywhere and on two recordings the encoder under-reads, which cannot be slip. Against a wheel-odometry final drift of 2.2–5.9 % of route length, slip is therefore not the dominant error; the yaw-rate geometry is (section 4.1.1).

Noise models supplied to the estimators. The visual estimators are not told the truth about the IMU, as table 3.15 shows.

Table 3.15 What the visual estimators are configured to expect, against what the simulator injects. Both estimators use identical values. Inflating noise above datasheet figures is standard practice in visual-inertial odometry and the configuration records that intent; the ratios are given so the size of the margin is visible. Since 6 November 2026 the EKF baseline is run on the same configured values, so that the two estimator families share one inertial model (section 3.6.1).

Parameter	Configured	Injected	Ratio	Quantity
`gyr_n`	0.002	3.394×10^{-4}	5.9×	gyroscope noise density
`acc_n`	0.02	4.002×10^{-3}	5.0×	accelerometer noise density
`gyr_w`	0.0001	1.735×10^{-6}	58×	gyroscope bias random walk
`acc_w`	0.001	4.899×10^{-4}	2.0×	accelerometer bias random walk

The injected bias random walks are the Gauss–Markov driving densities $\sigma_b\sqrt{2/\tau}$ implied by table 3.12.

3.4.4 Coordinate-frame contract

Standalone VINS-Fusion publishes `world` → `body`. Standalone ORB-SLAM3 publishes an estimator-specific world frame. For RTAB-Map integration, both publish `odom` → `vio_body`; a measured static transform connects `vio_body` → `base_footprint` by $(-0.338, 0, -0.192)$ m. RTAB-Map then publishes the external correction `map` → `odom` (figure 3.7).

The two independent paths to `cam0_optical` provide a runtime calibration check: `tf2_echo cam0_undis cam0_optical` must return zero translation and the identity quaternion. This check has been measured successfully. It is needed because an earlier OpenCV YAML parsing error yielded a plausible translation but a non-rigid rotation and invalidated a map-quality campaign.

The standalone simulation ORB launch still labels the estimated IMU-body pose as `base_footprint`. The composing RTAB-Map launch corrects this by using

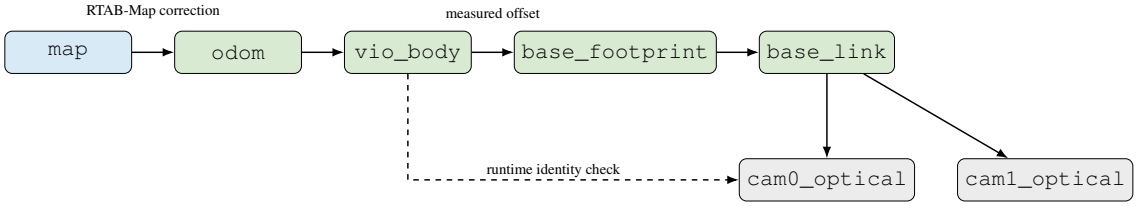


Figure 3.7 Coordinate-frame contract used by the composed RTAB-Map pipeline. The dashed connection is a calibration consistency check, not a second pose publisher.

`vio_body`; standalone consumers must not assume that the default label is the chassis origin. Constant offsets are mostly absorbed by rigid trajectory alignment but remain material for map placement and robot-footprint tests.

3.4.5 Simulation environments and controlled factors

Three simulated worlds carry every result in this report, all generated from the AWS RoboMaker small-warehouse model and all held in one directory, `aws-robomaker-small-warehouse-world/worlds/small_warehouse/`:

- `small_warehouse_static_tinymap.world`, the original small-warehouse shell at its native size. It is the world `map/map_tiny/map_tiny.db` was built in and scored against, and the only world small enough that one recording covers it.
- `small_warehouse_dynamic_realistic_nofloortexture.world` and its static twin `small_warehouse_static_realistic_nofloortexture.world`, the **U-shape** logistics layout.
- `small_warehouse_dynamic_realistic_nofloortexture_01.world` and its static twin `..._01.world`, the **I-shape** layout.

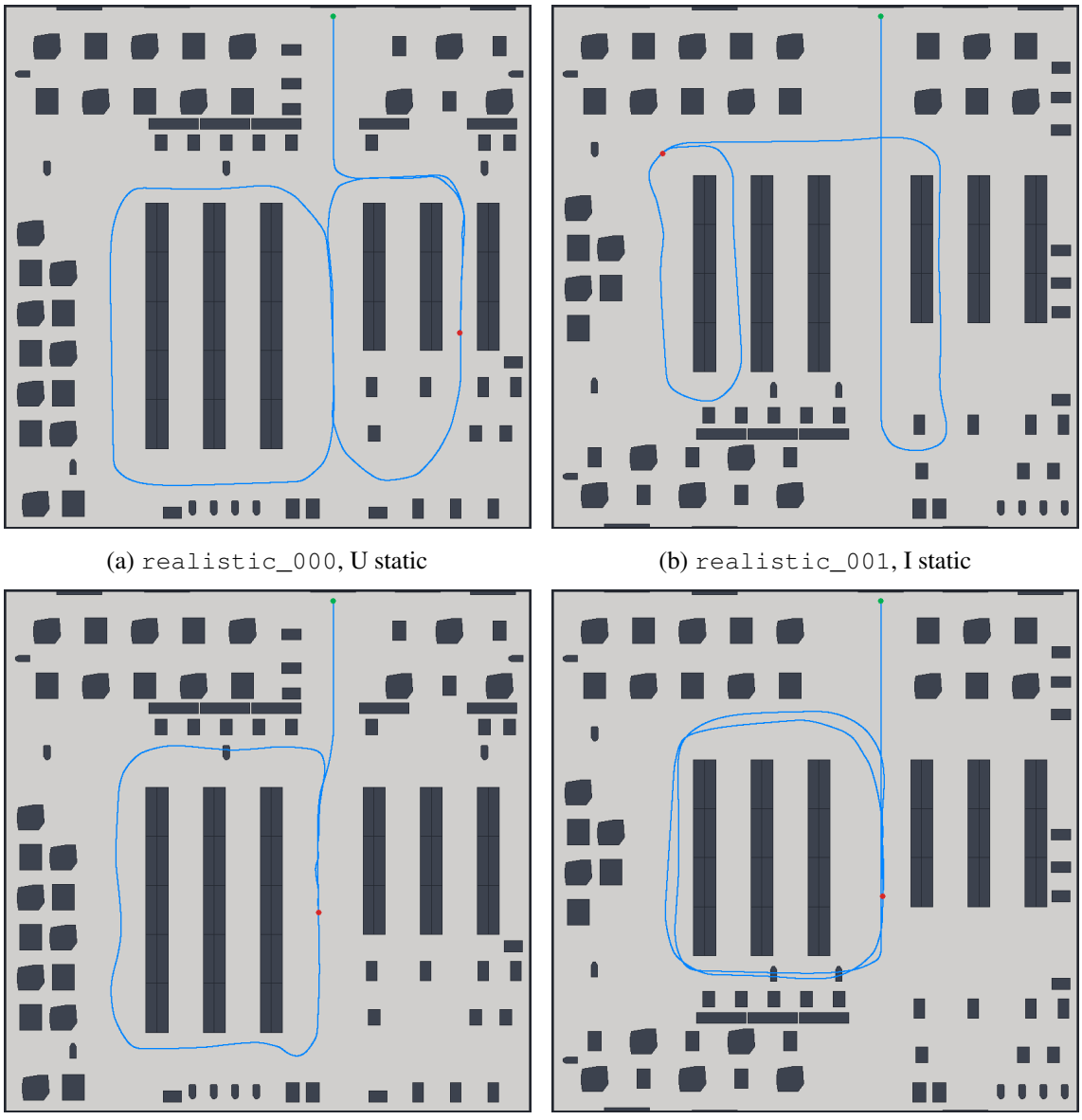
The two realistic layouts are the subject of section 3.4.6, which gives their zones, traffic model and baked occupancy truth. They are generated by `script/make_realistic_world.py` and `script/make_static_world.py` and are never hand-edited, so a layout can be rebuilt exactly from its generator.

Earlier work used three further arrangements of the rescaled warehouse, with scripted props moving on fixed segments, and a textured `bayline` floor variant. Those worlds still exist in the same directory, but the 6 November 2026 dataset decision removed every recording made in them, so no result in this report comes from them and no claim rests on the textured floor. Every recording in use is on the featureless floor, which is the harder condition for a visual front end and the one the real site resembles.

Table 3.16 states which experimental factors the simulator can vary, which levels the recordings in use actually realise, and which are available but unexercised. Figure 3.8 shows the four recordings of the realistic family as scenes.

3.4.6 Realistic logistics layouts

Status: implemented 6 October 2026; since the dataset decision of 6 November 2026 the simulation slots of Experiments 1, 3 and 4 in chapter 4 use these worlds and no other. The earlier object arrangements (section 3.4.5, retired with their recordings on 6 November 2026) were built as controlled stimuli: props distributed over the floor,



(a) `realistic_000`, U static

(b) `realistic_001`, I static

(c) `realistic_002`, U with traffic

(d) `realistic_003`, I with traffic

Figure 3.8 The four recordings of the realistic family: obstacle footprints at robot height over the featureless floor, with the ground-truth route drawn. The static twin and the traffic recording of a layout differ in traffic and in route, which is why traffic is reported as a condition of a recording and not as a tested factor.

Table 3.16 Controlled experimental factors in the simulation environment, the levels the recordings in use realise, and the experiment that reads each. A factor with one realised level is held constant, not studied: it bounds what the results can be attributed to.

Factor	Levels realised	Read by
Layout	U-shape, I-shape	Exp. 1, 3, 4; never pooled
Environment dynamics	static twin, warehouse traffic	Exp. 3, as a condition of a recording, not a factor: no two recordings share a route
Floor texture	featureless only	held constant; the textured variant is unexercised
Route	one per recording	confounded with layout and traffic; a second route per layout is the first thing a repeat needs
Estimator	ORB-SLAM3, VINS-Fusion	Exp. 1, 3
Front end	CPU, GPU feature extractor	Exp. 3
Loop closer	active, disabled	Exp. 1, 3
Dynamic filter	off only	Exp. 3: no filtered run exists in the current dataset set, so the hypothesis is untested
Replay rate	1.0 only	held constant; it is the deployment condition
Compute platform	one laptop	held constant; no hardware comparison is possible

props confined to the wall region, and props shuttling back and forth along fixed segments. None of them is laid out as a warehouse is laid out, and none of their dynamic props does anything a warehouse vehicle does. Two further worlds were therefore generated from the warehouse-design literature collected under `docs/references/warehouse/`, whose common prescription is a chain of zones—receiving, storage, picking, packing, dispatch—served by cross-aisles, with the fast-moving goods nearest the docks, bulk storage at the back or side, and offices and equipment charging out of the flow [13, 14]. The two canonical flow shapes of that literature are both provided:

- **U-shape flow**, `worlds/small_warehouse/small_warehouse_dynamic_realistic_nofloortexture.world`: receiving and dispatch docks on the same (north) wall with the management office between them, the fast pick faces nearest the docks, rack storage in north–south aisles, picking and packing east, block storage west, and the charging area, empty-pallet store and general office along the south wall.
- **I-shape (through) flow**, `..._realistic_nofloortexture_01.world`: receiving docks on the north wall and dispatch docks on the south wall, rack storage in the middle, the pick faces between storage and dispatch, packing beside dispatch, block storage west and both offices on the east wall, so that goods cross the building in one direction.

Both are written by `script/make_realistic_world.py` (`-layout u` or `i`) from parametric tables and are never edited by hand. Everything that is not the object arrangement is held fixed: the same 41.28 m shell, the same plain (`nofloortexture`) floor, lights, physics configuration (table 3.13) and robot spawn. The only change to

the shell is a regenerated wall model carrying dock-door openings, since the stock wall's geometry is frozen in its mesh. The controlled factors of table 3.16 are therefore unchanged by the shell, and these two layouts are the only levels of the layout factor the report realises. As explained below, they also make scene dynamics a within-layout condition for the first time.

Zones and contents. Table 3.17 lists what each zone holds. Aisles are 3.0 m between zones and 2.8 m between rack rows; pallet pitches are 3.2–4.0 m, widened from the 2.4 m of a stock bay so that a loaded pallet jack has the 2.6 m of swing room it needs to turn. The dock doors are 3.6 m by 4.5 m openings in the wall; a closed door is a roll-up panel in the wall thickness, an open door shows a docked 13.6 m semi-trailer and a yard slab, so that a camera looking out through it sees a trailer interior and tarmac rather than the sky colour. The U-shape world has five doors on the north wall (two open), the I-shape five on the north and three on the south (three open in all).

Table 3.17 Zones of the two realistic worlds and the number of object instances each holds, counted from the generated world files. "Inside" excludes the dock doors, trailers and yard, which lie in or beyond the wall line. The four carriers are the pallet jacks the traffic plugin drives; each layout holds ten pallet jacks in all, four carriers, four parked at charging and one in each staging zone. The generator reproduces the U world exactly but no longer reproduces the I world: it would now emit eight dock-line items instead of thirteen and two more clutter instances. The committed world file is the recorded environment and is the source of these counts.

Zone	Contents	U-shape	I-shape
Receiving staging	pallet loads in two rows, one parked jack	11	17
Dispatch staging	pallet loads in two rows, one parked jack	7	11
Pick faces	east–west shelves with totes in front	13	8
Storage, medium rotation	three double rack rows	30	24
Picking	three double rack rows	18	18
Packing	four desks, totes, bin	8	8
Block storage	floor-stacked pallets, one empty cell per job	13	6
Management office	two desks, bin	3	3
General office	desks, bin	4	3
MHE charging	parked pallet jacks; the bin is in the U layout only	5	4
Empty pallets	stacks	2	2
Carriers (traffic)	pallet jacks on jobs	4	4
People on foot (actors)	order pickers and dock checkers; not objects, not counted below	4	4
Inside the walls		118	108
Dock doors, trailers, yard		8	13
Total object instances		126	121
Baked occupancy (% of cells)		20.7	19.1

Traffic as jobs, not shuttles. The dynamic props of the earlier family move back and forth along a segment. In the realistic worlds the moving objects are four pallet jacks on logistics jobs, driven by a second world-level Gazebo system, `gz_kinematic_trajectory/LogisticsScheduler.cc`: a carrier drives to a load, takes it onto its forks (the load's

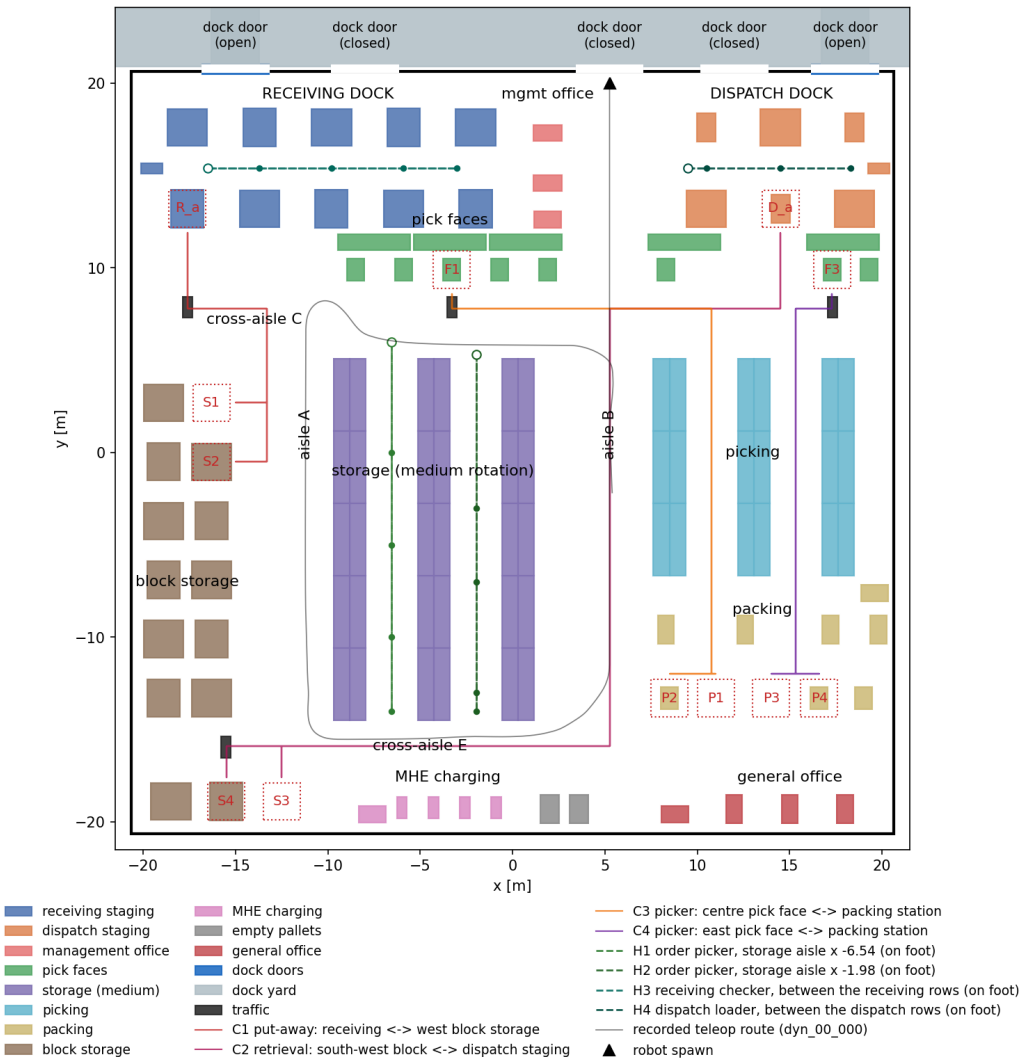


Figure 3.9 Plan of the U-shape realistic world, drawn by the generator from the same tables that write the world file. Filled rectangles are object footprints coloured by zone; dotted red boxes are the exchange slots the carrier jobs pick from and drop on (labels `R_a`, `S1...P4`); the coloured polylines are the four carrier routes; the dashed green lines are the routes of the four people on foot, with a dot at each place they stop and an open circle where they start; the grey trace is the teleop route of the earlier campaign that the main aisles A and B and cross-aisles C and E were placed to keep drivable; that recording has since been retired, and the routes actually used are those of figure 3.8. The band above the north wall is the dock yard with the two docked trailers.

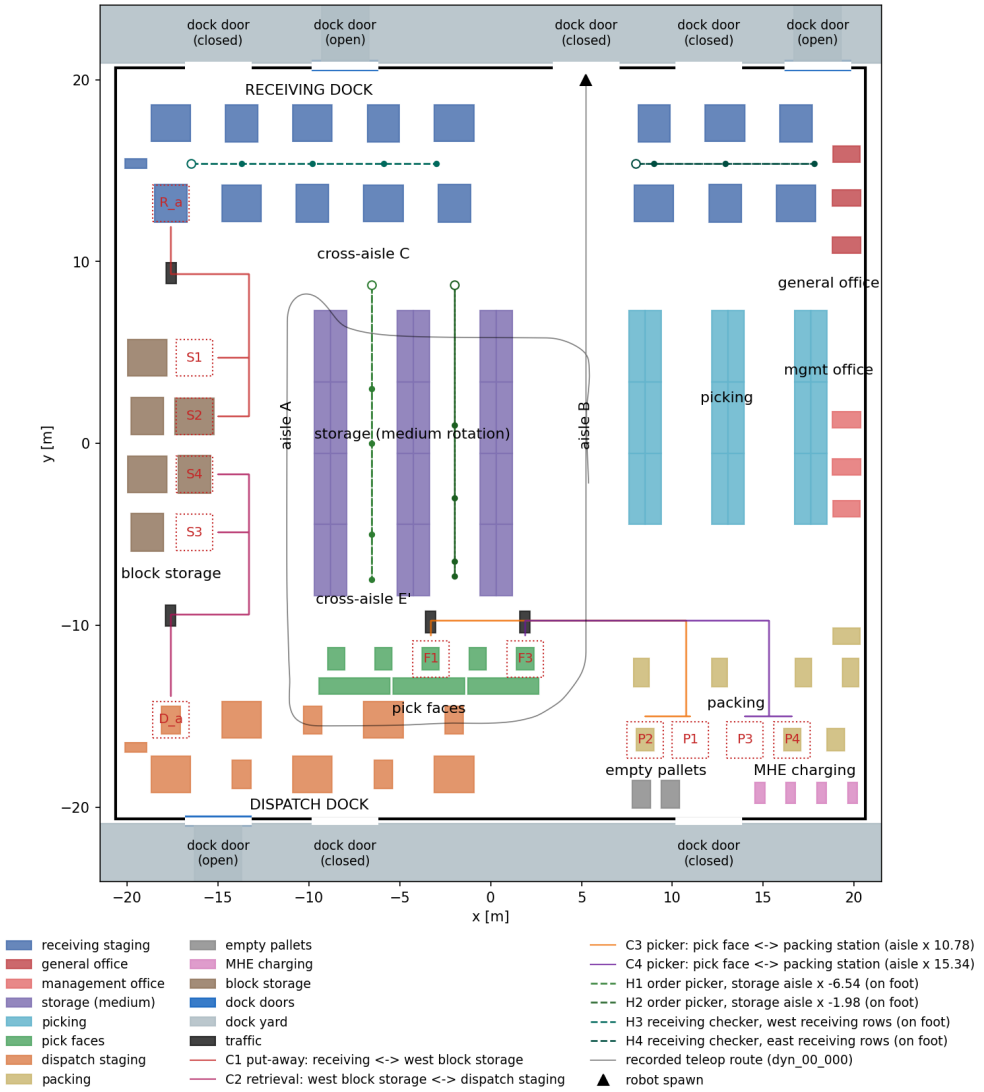


Figure 3.10 Plan of the I-shape realistic world, with the conventions of figure 3.9. Receiving docks are on the north wall and dispatch docks on the south wall; the pick faces sit between the storage racks and dispatch, and both offices are on the east wall because with docks on opposite walls the office cannot lie between reception and dispatch. The recorded teleop route (grey) crosses the storage racks here and is *not* drivable in this layout.

pose relative to the carrier is captured at pick-up and held rigidly through every turn), carries it through the aisles, sets it down on a target slot and goes for the next. Two jacks move pallets between receiving, block storage and dispatch; two move totes between the pick faces and the packing desks. Each job is a six-move rotation over two loads and three slots that returns to its starting state, so it loops indefinitely and no drop ever lands on an occupied slot. Carriers move at 1.2–1.3 $\mathrm{m\,s^{-1}}$, turn in place at 1.5 $\mathrm{rad\,s^{-1}}$, spend 4–5 s on each pick or drop, and carry the load 1.3 m ahead of the body. Slots are entered head-on and left in reverse, as a real jack does; the first design spun the loaded jack on the spot and swung the pallet into the neighbouring loads. The mechanism is the same kinematic pose command used by the earlier dynamic props, so the fidelity caveat of section 3.4.3 is unchanged: the vehicles cost no physics and offer no collision response if the robot drives into them.

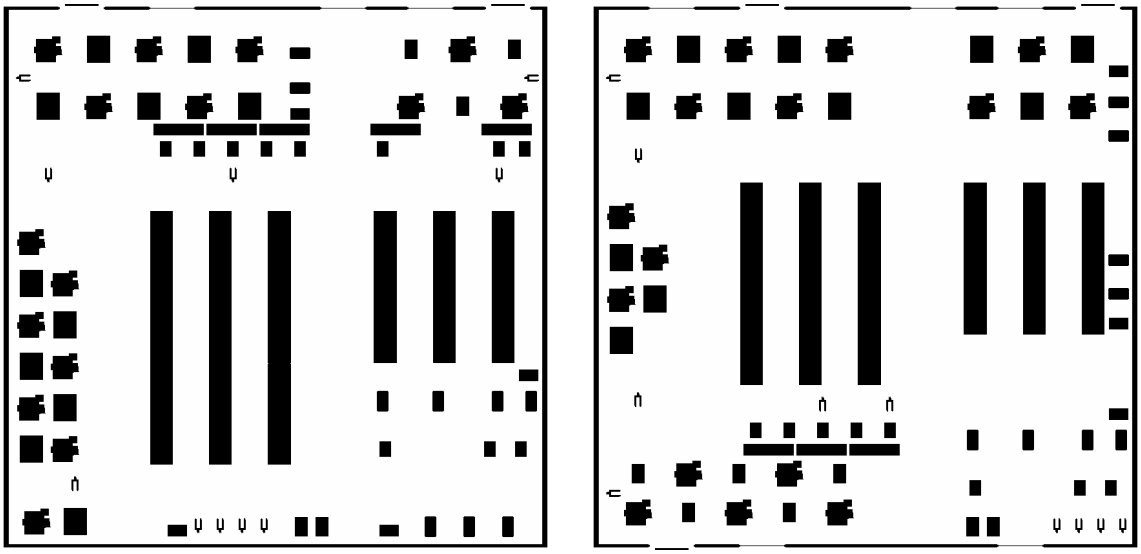
Before a world is written the generator proves it: every static pair is at least 0.25 m apart (rack runs excepted), every job's slot bookkeeping is simulated, and for every leg and every turn the swept box of the carrier plus its load is checked against every object not in that carrier's own job. Both worlds pass with every clearance positive, the tightest being 0.10 m at a packing-desk turn in each. Carriers can still cross one another briefly at an aisle intersection; that is a visual overlap, not a path conflict, and it is accepted.

People on foot. A warehouse floor also has people on it, and they are the dynamic objects a visual pipeline finds hardest: articulated, self-occluding and close to the camera. Each world therefore carries four walking people (added 9 October 2026), written as Gazebo *actors*: the stock Fuel walking-person skin, packaged as `models/aws_robomaker_warehouse_Person_01`, animated with a walk clip between stops and a stand clip at them, on a scripted closed route with one spline per leg so that legs stay straight in the aisles. The roles follow the same references as the layout and the Eaton case study [15]: two order pickers, one per storage aisle, who walk the aisle, stop at bays for six seconds and turn at the ends; and two dock checkers who walk the lane between the staging rows and stop at loads—in the U-shape one at receiving and one at dispatch, in the I-shape one on each receiving group. Walking speed is 1.1–1.2 $\mathrm{m\,s^{-1}}$, starts are staggered, and the number was kept small deliberately: four people on 1704 $\mathrm{m^2}$ with four vehicles is a working floor, not a crowd. People keep out of the vehicle lanes by construction. The generator sweeps a 0.7 m person box along every leg and checks it against the statics (at least 0.25 m), the carrier corridors and turn sweeps (no overlap) and the other people; the tightest clearances are 0.59 m to a static and 0.34 m to a carrier sweep. The pickers turn at the aisle mouths, which places them in view of a robot driving the cross-aisle. Actors have no collision geometry and no physics: nothing can bump into them, the occupancy bake does not see them, and the static twins omit them. One limitation matters for scoring: a scripted actor's animated pose is computed on the rendering side and is not written back to the entity state, so `/world/default/pose/info` reports a person at its initial pose throughout. The script is deterministic, so the pose is instead evaluated from the world file and the simulation clock by `script/people_script.py`, which the 2-D viewer uses to draw the people (linear between the 2 m waypoints, within 0.2 m of the rendered spline). Projecting a detector ground truth from it is not yet implemented, so Experiment 2 cannot yet score the people and Experiment 3 sees them only as unlabelled motion.

Static twins. For each layout a static world, `worlds/small_warehouse/small_warehouse_static_realistic_nofloortexture.world` and its `..._01.`

`world` counterpart, is derived by `script/make_static_world.py`: every object is marked static and the traffic plugin removed, with nothing else changed. This is the methodological point of the family. In the earlier worlds the static arrangement (`_01`) and the dynamic arrangements (`_00` to `_02`) differ in geometry as well as in motion, so a static-versus-dynamic comparison on them is confounded by the arrangement. A realistic world and its twin share every footprint and differ only in whether four vehicles move.

Occupancy truth and routes. The baked grids (figure 3.11) rasterise the moving carriers and their loads at their $t = 0$ pose, which is where the static twin holds them permanently. The closed dock doors appear as a one-cell line in the wall; the open ones are gaps onto the yard. The U-shape world keeps the main aisles A ($x \approx -11.2$ m) and B ($x = 5.25$ m, the dock lane the robot spawns in) and the cross-aisles C and E drivable; the I-shape world fitted none of the routes recorded before it existed, so both layouts were driven again. The six recordings of the family, four for Experiments 1 and 3 and two longer mapping runs, are those of table 3.18.



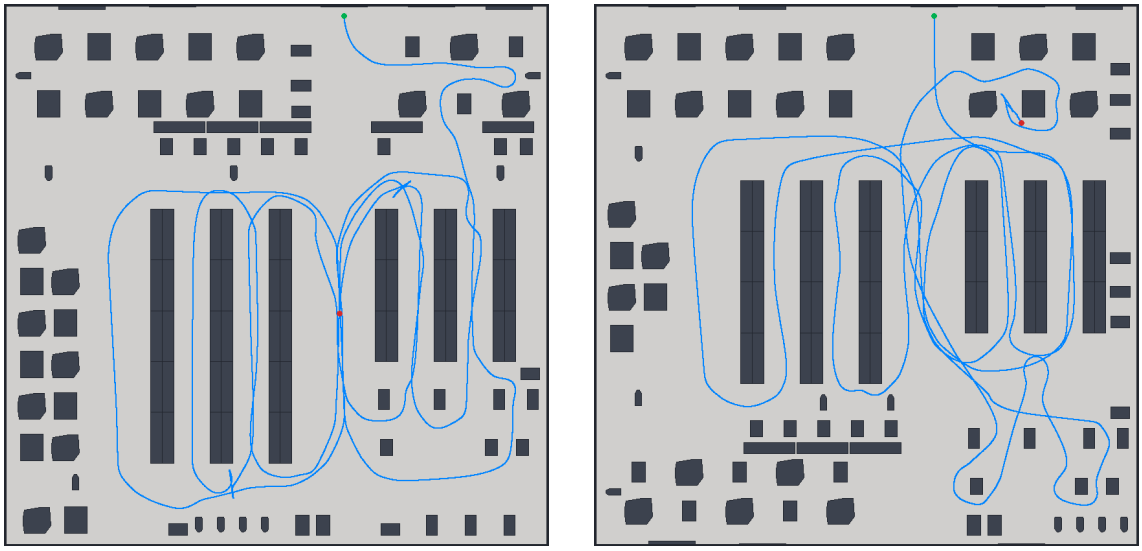
(a) U-shape, 20.7 % occupied

(b) I-shape, 19.1 % occupied

Figure 3.11 Baked occupancy truth of the two realistic worlds: 42.2 m square at 0.05 m per cell, origin $(-21.1, -21.1)$ m. Carriers and loads are baked at their $t = 0$ pose. Closed dock doors are the one-cell segments in the north (and, in (b), south) wall; open doors are the gaps.

Recordings in the family. Six recordings were made in these worlds, all listed in table 3.18. Four are the `dataset_allsensor_realistic` set of Experiments 1 and 3: a static twin and a traffic recording in each layout, 80–125 s of simulated time over 103–171 m. Two are longer mapping runs in the static twins, `dataset_allsensor_realistic_map` (U) and `dataset_allsensor_realistic_01_map` (I), which build the databases of Experiment 4 and carry the repeated runs that measure latency and resources in Experiment 3.

The mapping runs are the only recordings that reach the speed the project targets. Both drive a serpentine that enters every aisle (figure 3.12), 374.3 m over 288.2 s in the U layout and 384.6 m over 315.2 s in the I layout, at a median 1.49 and 1.48 $\mathrm{m\,s^{-1}}$ against the 1.5 $\mathrm{m\,s^{-1}}$ of section 3.1. The four shorter recordings run well below that. Both mapping runs start at the same spawn, $(5.25, 20.0)$ m in the dock lane, and both carry the proprioceptive topics, so the non-visual baseline of section 3.6.1 could be computed on them as well.



(a) U layout, 374.3 m (b) I layout, 384.6 m

Figure 3.12 The two mapping recordings, drawn with the conventions of figure 3.8: obstacle footprints at robot height over the featureless floor, with the `/ground_truth/odometry` route in blue from the green start to the red end. Both routes enter every aisle, which is what a map build needs and what the four shorter recordings do not do.

3.4.7 Dataset registry

The dataset set was fixed on 6 November 2026 and the report draws every result from the eleven recordings of table 3.18. Earlier recordings remain on the storage drive but are used by nothing in this report: the five `allsensor` and `static_nofloortexture` bags of the first simulation campaign, and the `realv2` to `realv5` real recordings, which were deleted when the drive was rebuilt. Results computed from them were moved to `trash/output/` on the same day and are cited nowhere.

All recordings live on an internal partition mounted at `/media/ambushee/New Volume/wil_rescue/`, to which every bag was copied by `rsync` on 3 November 2026 after the removable drive that had held them began to fail; that drive is retired as a source. The workspace tree `dataset/` keeps only what is in active use. Ground truth is held separately under `dataset/gt/<recording>/`: for the simulation recordings a copy of the bag's own `/ground_truth/odometry` resampled to the scoring convention, and for the real recordings the optical tracker's planar export. Integrity was checked by opening each SQLite store and counting rows against the count its `metadata.yaml` declares rather than by trusting the metadata, because the two faults seen on the old drive, a truncated store and a stub file, both leave the metadata intact; all eleven pass.

What the real recordings carry, and what they do not. The real rig publishes no wheel odometry in metric units and no joint states. In place of them each bag carries `/feedback/velocity` and `/feedback/steering` at 50 Hz as `std_msgs/Float64`: the velocity is in a firmware unit whose scale is not documented, and the steering is a command echo that takes two values on every recording, so it carries no steering angle. The consequence for the non-visual baseline is worked out in section 3.6.1. The inertial stream is a WitMotion HWT101CT at 200 Hz on the inherited topic name `/hwt101ct_yaw_publisher`; its orientation field is an identity quaternion and must never be consumed

Table 3.18 Dataset registry at the reporting cutoff: the eleven recordings every result in this report is drawn from. Span is the ground-truth time span for the simulation recordings, which is shorter than the rosbag wall-clock duration because the simulator ran below real time, and the rosbag duration for the real recordings. Route is the ground-truth path length, from the simulator odometry or the tracker export. "Prop." marks the proprioceptive group (`/model/ackermann_robot_001/joint_state` and `.../odometry`) in simulation and the wheel-speed and steering feedback on the real rig. Stereo pairs are `cam0` messages; inertial messages are `/imu` in simulation and `/hwt101ct_yaw_publisher` on the real rig.

Recording	Role	Layout	Span [s]	Route [m]	Stereo	IMU	Truth	Prop.	Size [MB]
`allsensor_realistic_000`	Exp. 1, 3; Exp. 4 probe	U static	125.5	171.3	3 805	25 112	6 278	full	1 529
`allsensor_realistic_001`	Exp. 1, 3; Exp. 4 probe	I static	96.9	133.0	2 937	19 388	4 847	full	1 159
`allsensor_realistic_002`	Exp. 1, 3	U traffic	79.5	102.7	2 409	15 903	3 975	full	1 017
`allsensor_realistic_003`	Exp. 1, 3	I traffic	117.0	158.0	3 547	23 409	5 852	full	1 520
`allsensor_realistic_map`	Exp. 4 build; Exp. 3 resources	U static	288.2	374.3	8 725	57 645	14 411	full	3 736
`allsensor_realistic_01_map`	Exp. 4 build; Exp. 3 resources	I static	315.2	384.6	9 552	63 040	15 760	full	3 835
`map_tiny`	Exp. 4 build (tiny world)	tiny	60.6	70.1	1 837	12 127	3 032	—	726
`realv6_000`	Exp. 1, 3 (real)	Gensurv	220.1	37.3	6 601	43 903	277	feedback	5 422
`realv7_000`	Exp. 1, 3 (real)	Gensurv	90.7	21.6	2 722	18 107	524	feedback	2 642
`realv7_001`	Exp. 1, 3 (real)	Gensurv	126.3	31.7	3 790	25 204	669	feedback	3 621
`realv7_002`	Exp. 1, 3 (real)	Gensurv	97.7	23.4	2 930	19 486	540	feedback	2 795

as an attitude measurement.

Ground truth and clocks. The simulation recordings carry `/ground_truth/odometry` at 50 Hz from the simulator, which is the pose of the robot model's origin and is moved to the body point the estimators report before any comparison (section 3.7.1). The real recordings have no onboard reference: an optical tracker exports a planar position at 1.7–6 Hz on its own clock, so the offset between tracker and robot is estimated per recording by cross-correlating the tracker's speed with the wheel-speed feedback. The `realv7` exports are the first that pass every validity rule of section 3.7.1; the `realv6` export runs at 1.7 Hz and fails them, which is why its values are reported as diagnostics.

3.5 Real Platform

3.5.1 AMR mobile robot testbed

The system is installed and tested on a Gensurv Robotics mobile-robot base designed to reproduce the behaviour of an industrial warehouse AMR (figures 3.13 and 3.14). Its maximum speed is set to $1.5\,\mathrm{m\,s^{-1}}$, the limit of the SLAM stability tests. The drivetrain is a rear brushless motor (Flipsky 6384) through a BLDC drive with a Dynamixel RX-64 steering servo, a GigaDevice GD32407V microcontroller on a CAN link to the NRU-51V, an LM2596 regulator from a lithium-polymer pack, and a Wi-Fi router for tele-operation from a smartphone. The faculty template describes the base as a differential-drive robot; the hardware diagram (a single traction motor with a steering servo) and the simulated model of figure 3.6 are Ackermann. The description here follows the hardware and should be confirmed by the author.

3.5.2 Sensors and computing unit

Wide-angle stereo cameras (AC-IMX390-H190). Two AC-IMX390-H190 modules form the stereo pair (figure 3.15); table 3.19 lists the specification.

Inertial measurement unit. A six-axis IMU measures linear acceleration and angular

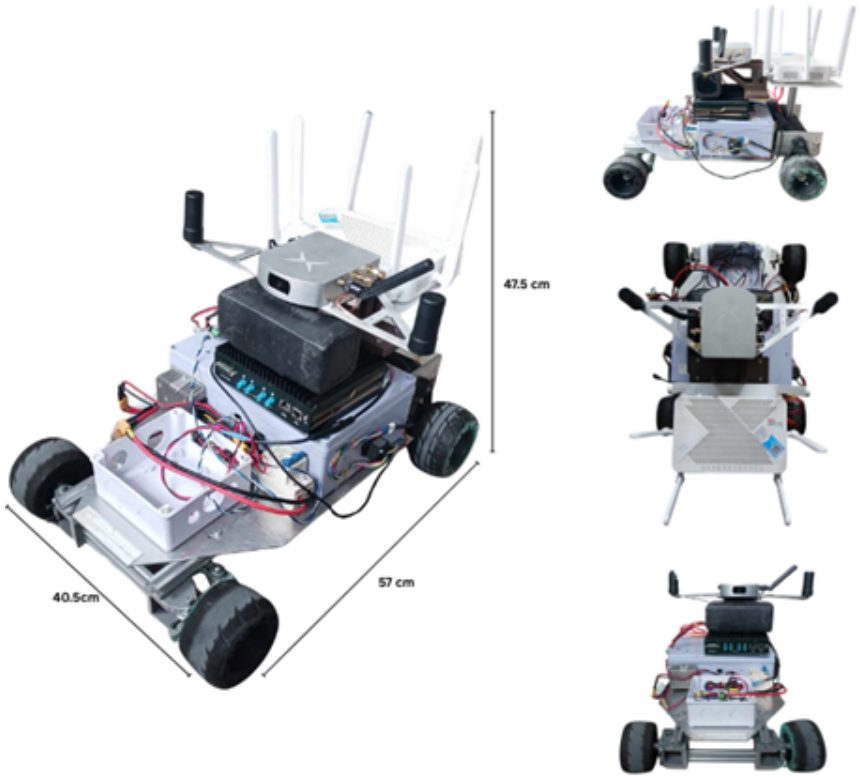


Figure 3.13 Dimensions of the mock AMR used to test the navigation system: 57 cm long, 40.5 cm wide and 47.5 cm high to the top of the sensor mast.

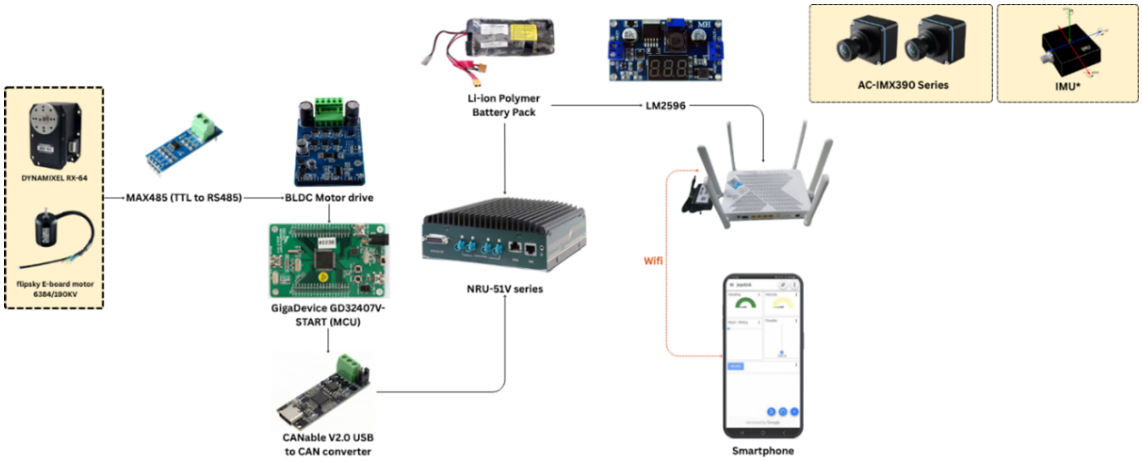


Figure 3.14 Hardware components of the mock AMR: steering servo and traction motor, BLDC drive and microcontroller on CAN, the NRU-51V computer, power supply, the AC-IMX390 stereo pair and IMU, and the Wi-Fi link to the smartphone joystick.

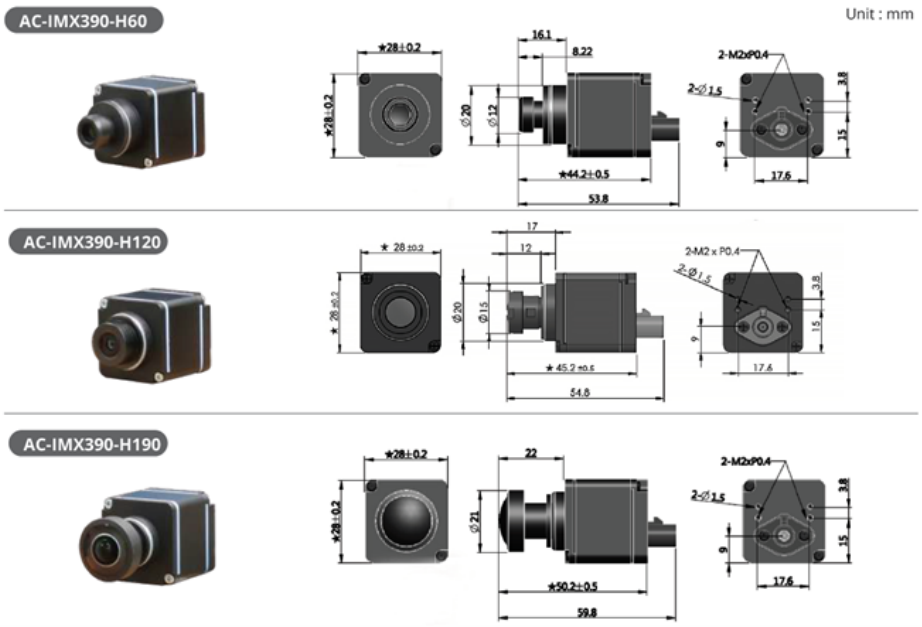


Figure 3.15 Dimensions of the AC-IMX390 camera family in millimetres; the H190 variant at the bottom is the one used.

Table 3.19 Specification of the AC-IMX390-H190 camera.

Item	Detail
Sensor and ISP	Sony IMX390 CMOS sensor with GEO Semiconductor GW5200 ISP
Resolution and frame rate	1920 × 1080 (2 MP) at 30 fps
Horizontal field of view	186°
Video output	uncompressed YUV422 (UYVY)
Dynamic range	120 dB high dynamic range (HDR)
Special features	LED flicker mitigation (LFM), 5-axis active alignment
Environmental rating	IP67 and IP69K (lens side)
Operating temperature	−40–85 °C
Connector	male FAKRA

rate at 100–200 Hz. Its data enter IMU pre-integration and are fused with the camera images to support the odometry estimate while the camera moves fast or the image is momentarily blurred. On the recorded real datasets the inertial stream is the `/hwt101ct_yaw_publisher` topic (table 3.21).

Edge AI computing unit: Neousys NRU-51V (NVIDIA Jetson Xavier NX). To run the deep-learning model (YOLO26) and the visual-inertial SLAM on the robot without a central computer, the project uses the Neousys NRU-51V, a compact industrial edge computer designed for vision and autonomous-vehicle systems (figures 3.16 and 3.17). Table 3.20 summarises the hardware.



Figure 3.16 The Neousys NRU-51V (NVIDIA Jetson Xavier NX), showing the four FAKRA camera ports.

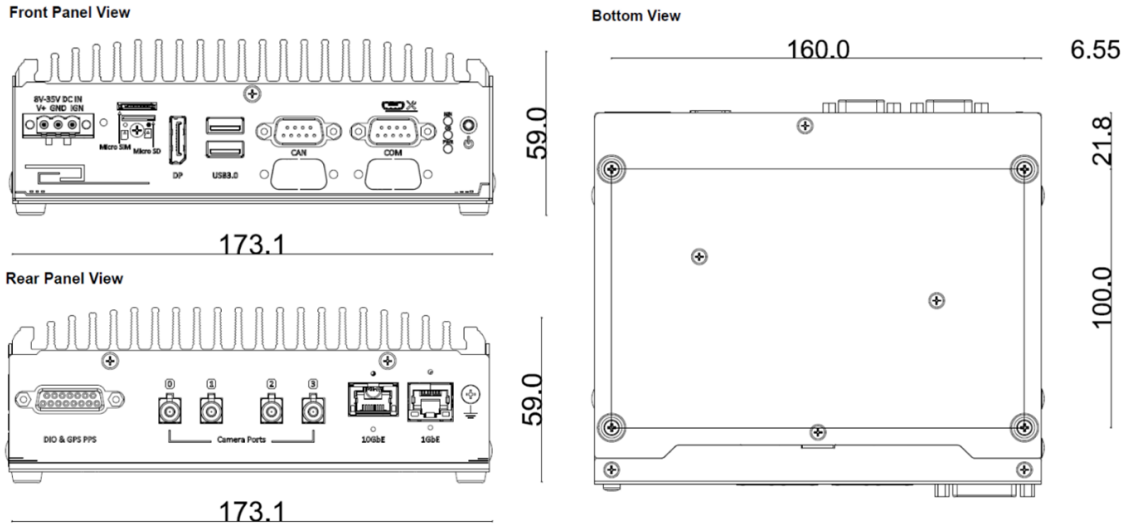


Figure 3.17 Dimensions of the Neousys NRU-51V in millimetres.

3.5.3 Real sensor rig and calibration limits

Table 3.21 gives the real sensor rig and the calibration status of each quantity. The real rig's calibration is materially weaker than the simulated rig's, and the weaknesses are load bearing rather than cosmetic, although two of them have been closed since the first real recordings.

Two calibration quantities have been closed. The inertial lever arm, a zero placeholder in the first real configuration, was tape-measured in the camera optical frame and is now carried in `IMU.T_b_c1`. The stationary accelerometer magnitude was re-measured on this

Table 3.20 Summary of the hardware technical specification.

Device / sensor	Model / specification	Role in the system
Edge processing unit	Neousys NRU-51V (NVIDIA Jetson Xavier NX, 21 TOPS)	runs YOLO26, the SLAM system and the ROS 2 navigation stack
Stereo camera module	AC-IMX390-H190 (Sony IMX390 sensor)	records 186° stereo imagery for object detection and feature extraction
Inertial sensor (IMU)	6-DoF industrial IMU (100–200 Hz)	measures acceleration and angular rate to support pose estimation (VIO)
Mobile testbed	AMR platform (speed 1.5 $\mathrm{m\,s^{-1}}$, slope 5°)	robot base for online navigation tests in the mock environment

Table 3.21 Real sensor rig, and the calibration status of each quantity at the reporting cutoff. The two entries that changed since the first real recordings are the inertial lever arm, now measured, and the gravity magnitude, now re-measured on this rig.

Quantity	Value	Status
Stereo resolution	1920×1080 fisheye	measured
Stereo baseline	0.0916 m	measured 3 November 2026
Camera order	`cam1` is the left camera	since the 19 October 2026 remount; encoded in the launch defaults
IMU stream	`/hwt101ct_yaw_publisher`, 200 Hz	WitMotion HWT101CT; orientation field is identity and is never consumed
Camera model (ORB-SLAM3)	`KannalaBrandt8`	configured
Camera model (VINS)	project fisheye files	configured
Second-camera intrinsics	reuse of the first camera	approximation
IMU-to-camera rotation	observational support	partial
IMU-to-camera translation	0.26 m below and 0.065 m behind `cam0`	tape-measured 4 November 2026
Stationary $\|a\|$	9.8858 $\mathrm{m\,s^{-2}}$	measured at rest on this rig
Inertial noise (ORB-SLAM3)	`NoiseGyro` 0.1, `NoiseAcc` 1.0	ten times the simulated values, by experiment
Extrinsic estimation (VINS)	`estimate_extrinsic: 1`	online
Time-offset estimation (VINS)	`estimate_td: 1`	online
Marker-to-IMU lever arm	not measured	placeholder; bounds every real accuracy figure

rig as $9.8858\,\mathrm{m\,s^{-2}}$, replacing a $9.2642\,\mathrm{m\,s^{-2}}$ figure inherited from an earlier RealSense unit; VINS-Fusion takes the measured value as `g_norm` and the ORB-SLAM3 launch rescales acceleration by $9.81/9.8858$ rather than the old $9.81/9.2642$, which had been feeding the estimator $10.47\,\mathrm{m\,s^{-2}}$ and preventing inertial initialisation altogether.

Three weaknesses remain and bound what any real result can claim. The second camera reuses the first camera's intrinsics, so the stereo rectification is approximate. The camera–IMU rotation has observational support but no formal calibration. And the transform from the tracked marker to the body point the estimators report is unmeasured, so every real trajectory is scored as though marker and body coincide. Real trajectories can therefore be compared with an independent planar reference, which the `realv6` and `realv7` recordings now provide, but every metric computed from them is provisional until that lever arm is measured (sections 4.1.2 and 4.3.2).

3.5.4 Test site and experimental variables

Spatial layout. The test area is divided into two operating zones, Zone A and Zone B, separated by and connected through a walkway (each zone about 3 m square, walkway 1.25 m wide; figure 1.2). The division reproduces the traffic pattern of a warehouse AMR that shuttles continuously between a pick-up and a drop-off point. The robot's linear speed is limited to $v_{\max} = 1.5\,\mathrm{m\,s^{-1}}$ throughout.

Ground truth by motion capture. Evaluating SLAM error (ATE and RPE) needs a reference accurate to millimetres. A motion-capture camera system is installed around the test area and tracks reflective markers placed at the robot's centre of mass at a rate above 100 Hz.

Coordinate-frame alignment. The SLAM pose P_{slam} is expressed in the local odometry frame fixed at start-up, whereas the motion-capture pose P_{gt} is expressed in the room's global frame. Before errors are computed the two trajectories are aligned by a rigid-body transformation; the alignment actually used for every result in this report, and its treatment of scale, is defined in section 3.7.1.

Dynamic-environment variable. Each time the robot completes a zone transfer (for example arrives in Zone B from Zone A), the operator randomly re-positions and re-counts the stacks of goods in the opposite zone before the robot returns. This forces the SLAM system to face a scene that no longer matches its map (map obsolescence), to test whether ORB-SLAM3's Atlas or RTAB-Map's memory management can resolve the conflicting spatial evidence without tracking loss.

Illumination variable. Data are collected along the same route at three times of day representing different light levels and directions: morning (08:00), oblique natural light casting shadows across the test area; midday (12:00), maximum brightness, with possible high contrast or over-exposure in places; and evening (18:00), low light relying mainly on the building's artificial lighting, with raised image noise. The variable tests both the camera hardware (Sony IMX390 HDR) and the robustness of the two SLAM frameworks in keeping enough inlier features for odometry under variable light.

Real-dataset status. The four real recordings in use are listed in table 3.18: `dataset_realv6_000` and `dataset_realv7_000` to `_002`, each with inertial data at 200 Hz and a planar optical-tracker reference (section 4.3.2). Varied illumination is still absent, so the illumination variable of this section is unexercised, and the real slots of Experiments 2

and 4 remain planned for want of annotated frames and a surveyed reference grid.

3.6 The Four Experiments

Each experiment below is defined by its objective, its process, its evaluation metrics, its constraints and the validity rule that decides whether a run counts. Each is run in both domains; chapter 4 reports each domain in its own slot.

3.6.1 Experiment 1: Non-visual baseline and estimator comparison

Objective. Establish a dead-reckoning baseline to determine whether the visual pipelines (ORB-SLAM3 and VINS-Fusion) genuinely outperform the robot's own wheel odometry, its IMU, and a wheel-plus-IMU extended Kalman filter, and whether the uncertainty each estimator reports describes the error it makes.

Process. Wheel odometry, IMU dead reckoning and the EKF are run offline on the same recordings the visual estimators consumed, from proprioceptive topics only. A calibration pass measures each sensor's noise and scale against ground truth first; the estimators then run and are scored on the same interval as the visual runs.

Evaluation metrics. Aligned and anchored ATE, final drift as a fraction of distance, final heading error, RPE at 1 s, and the covariance consistency measures NEES and $k\sigma$ coverage.

Validity rule. A run counts only if the planar assumption holds (maximum tilt below 1°), the IMU orientation field is not consumed, and the number of wheel corrections equals the number of wheel-odometry messages.

Three non-visual estimators are implemented in `script/proprio_estimator.py` to supply the dead-reckoning floor: wheel odometry, IMU dead reckoning, and an extended Kalman filter fusing the two. They consume only proprioceptive topics and no images.

Implementation form and why it is offline

All three read the recorded bag directly and are deterministic functions of it. Run as live ROS nodes they would instead depend on replay rate, QoS delivery and CPU contention, none of which is a property of the estimator being compared. The cost is a stated asymmetry: the visual figures they are compared against were produced by live runs, so any timing-induced error is present on one side of the comparison only.

Sensor model derived from data

Every covariance field in the source bags is identically zero: of the 25 112 inertial messages in `dataset_allsensor_realistic_000`, none carries a non-zero orientation, angular-velocity or linear-acceleration covariance. Gazebo Fortress does not populate them and `AckermannSteering` exposes no noise parameter, so the noise model is measured rather than read, by a `calibrate` pass that writes `sensor_noise.yaml` per recording. Table 3.22 gives the result for the four recordings of Experiment 1.

Two consequences follow directly. Wheel *speed* is essentially unbiased on all four recordings, scale 0.9966–1.0012, so distance travelled is not the error source. And the rear-wheel differential is a far worse yaw-rate estimator than the steering-angle bicycle model, resid-

Table 3.22 Sensor noise and scale measured from each recording by `proprio_estimator.py calibrate` on 6 November 2026, against that recording's own ground truth. Scale is the least-squares gain of the sensor on ground truth through the origin; σ is the residual standard deviation. The static window is the stationary period at the head of the recording from which the inertial biases are taken.

Group	Quantity	`_000` U static	`_001` I static	`_002` U traffic	`_003` I traffic	Units
IMU	gyroscope z scale	1.0000	1.0001	0.9994	1.0001	–
	gyroscope noise density	3.5×10^{-4}	3.5×10^{-4}	3.1×10^{-4}	3.5×10^{-4}	$\mathrm{rad\,s^{-1}\,Hz^{-1/2}}$
	accelerometer x scale	1.0321	1.0954	1.0172	1.0339	–
	accelerometer x bias	0.038	0.013	0.013	0.079	$\mathrm{m\,s^{-2}}$
	accelerometer noise density	1.6×10^{-2}	1.5×10^{-2}	1.8×10^{-2}	2.4×10^{-2}	$\mathrm{m\,s^{-2}\,Hz^{-1/2}}$
	measured g	9.787	9.801	9.845	9.804	$\mathrm{m\,s^{-2}}$
Wheel	speed scale	1.0004	0.9966	1.0012	0.9993	–
	speed σ	0.0423	0.0432	0.0408	0.0292	$\mathrm{m\,s^{-1}}$
	yaw-rate scale, bicycle	1.0245	0.9321	1.0070	1.0171	–
	yaw-rate σ, bicycle	0.0680	0.0677	0.0766	0.0589	$\mathrm{rad\,s^{-1}}$
	yaw-rate scale, differential	1.0865	1.1873	1.2141	1.1627	–
	yaw-rate σ, differential	0.2442	0.2287	0.2516	0.1910	$\mathrm{rad\,s^{-1}}$
Window	static duration	1.90	2.16	1.40	1.14	s

uals of 0.19–0.25 $\mathrm{rad\,s^{-1}}$ against 0.059–0.077 $\mathrm{rad\,s^{-1}}$, because the track and the wheel base are both 0.42 m and a solid rear axle slips through every turn. The bicycle model is selected automatically and was chosen on all four.

The orientation field must not be used

`sensor_msgs/Imu.orientation` in these bags is not an estimate. Measured over `dataset_allsensor_realistic_000` it equals ground-truth yaw plus exactly 90°, with a residual standard deviation of 0.001° and no drift at all across 125.6 s: the plugin publishes the true orientation from the simulator's entity-component manager. Consuming it would give a dead-reckoning estimator a perfect attitude reference that no real IMU has, and its output would measure ground truth. All three estimators therefore integrate `angular_velocity.z` instead, which drifts honestly, by 3.66° over the same recording. Each run records `imu_orientation_used=NO` in its metadata so the exclusion is auditable.

Estimator definitions

Table 3.23 defines the three estimators. The EKF carries the planar state $\mathbf{x} = [p_x, p_y, \theta, v, b_\omega]^\top$, predicted at the 200 Hz IMU rate and corrected at 50 Hz by the wheels.

Which noise model the baseline runs on. By the design decision of 6 November 2026 the EKF reported as the baseline does *not* use the measured inertial noise of table 3.22. Its three inertial terms are the values both visual configurations declare, gyroscope density 0.002, accelerometer density 0.02 and gyroscope bias walk 1×10^{-4}, converted to a per-sample σ at 200 Hz by `script/make_vslam_noise_yaml.py`. The reason is that the proprioceptive and visual families must run on the same hand-set inertial model before their accuracies can be compared at all; the configured gyroscope density is five to seven

Table 3.23 Implemented non-visual estimators. "Anchoring" refers to the initial pose: dead reckoning has no global reference, so all three start from the ground-truth pose at their first sample, and this is recorded as an alignment decision rather than treated as neutral.

Estimator	Inputs	State	Correction
Wheel odometry	rear wheel speeds, steering angles (1 kHz)	$[p_x, p_y, \theta]$	none; $\mathbf{P}$ grows without bound
IMU dead reckoning	gyroscope z, accelerometer x (200 Hz)	$[p_x, p_y, \theta, v]$	none
EKF wheel + IMU	both of the above	$[p_x, p_y, \theta, v, b_\omega]$	wheel speed corrects v; $(\omega_{\text{gyro}} - \omega_{\text{wheel}})$ corrects b_ω

times the measured one, and a filter given the measured value would be competing against visual systems that were not. The measured-noise filter is run beside it as a diagnostic row. Biases and both wheel σ have no visual counterpart and stay measured in both.

EKF formulation

State and inputs. The filter carries five planar states,

$$\mathbf{x} = \begin{bmatrix} p_x & p_y & \theta & v & b_\omega \end{bmatrix}^\top, \tag{3.7}$$

where b_ω is the gyroscope z bias. Roll and pitch stay within 0.002° across every recording, so a full 15-state inertial formulation would spend nine states on quantities this platform does not have, and their covariances would be unobservable noise in the consistency test that motivates the experiment.

The IMU is not a measurement here: it drives the prediction. Bias-corrected yaw rate and longitudinal acceleration act as the input,

$$\omega = \omega_{\text{gyro}} - b_\omega, \qquad a = a_x - b_a, \tag{3.8}$$

with b_a a constant measured during calibration rather than a tracked state. Propagation uses the midpoint heading $\theta_m = \theta + \frac{1}{2}\omega\,\Delta t$, which matters over the 103–171 m these recordings cover:

$$\begin{aligned} p_x &\mathrel{+}= v\cos\theta_m\,\Delta t + \tfrac{1}{2}a\cos\theta_m\,\Delta t^2, \\ p_y &\mathrel{+}= v\sin\theta_m\,\Delta t + \tfrac{1}{2}a\sin\theta_m\,\Delta t^2, \\ v &\mathrel{+}= a\,\Delta t, \qquad \theta \mathrel{+}= \omega\,\Delta t. \end{aligned} \tag{3.9}$$

Covariance and process noise. With $\mathbf{F} = \partial f/\partial \mathbf{x}$ and $\mathbf{G}$ mapping the noise inputs into the state,

$$\mathbf{P} \leftarrow \mathbf{F}\mathbf{P}\mathbf{F}^\top + \mathbf{G}\,\mathrm{diag}(\sigma_a^2,\ \sigma_g^2,\ \sigma_b^2)\,\mathbf{G}^\top. \tag{3.10}$$

The non-trivial entries are $F_{13} = -v\sin\theta_m\Delta t$, $F_{14} = \cos\theta_m\Delta t$, $F_{23} = v\cos\theta_m\Delta t$, $F_{24} = \sin\theta_m\Delta t$ and $F_{35} = -\Delta t$; the last couples bias error directly into heading, which is what makes b_ω worth estimating. In $\mathbf{G}$, accelerometer noise enters position as $\frac{1}{2}\cos\theta_m\Delta t^2$ and speed as Δt, gyroscope noise enters heading as Δt, and the bias row carries $\sqrt{\Delta t}$ rather than

Δt because a random walk accumulates as $\sqrt{t}$.

Both IMU noise terms are therefore *process* noise. Nothing from the IMU appears in $\mathbf{R}$, because the IMU propagates the state rather than observing it.

Measurement. The wheels correct at 50 Hz with

$$\mathbf{z} = \begin{bmatrix} v_{\text{wheel}} \\ \omega_{\text{gyro}} - \omega_{\text{wheel}} \end{bmatrix}, \qquad \mathbf{H} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{R} = \operatorname{diag}\left(\sigma_{v,w}^2,\ \sigma_{\omega,w}^2 + \sigma_g^2\right). \tag{3.11}$$

The second variance is a sum because that observation is a *difference* of two noisy sensors and inherits both. The update uses the Joseph form,

$$\mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}(\mathbf{I} - \mathbf{K}\mathbf{H})^\top + \mathbf{K}\mathbf{R}\mathbf{K}^\top, \tag{3.12}$$

symmetrised afterwards. The shorter $(\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}$ does not reliably stay positive definite across the thousands of updates in a run, and here the covariance is itself the measurement under test, so it cannot be allowed to decay.

Where the numbers come from. $\sigma_{v,w}$ and $\sigma_{\omega,w}$ come from the wheel stream against ground truth (table 3.22); σ_a and σ_g come from the visual configuration, as explained above.

Why the bias, and not the heading, is observed

The EKF's second measurement is deliberately an observation of the *gyroscope bias* and not of heading. Integrating the wheel yaw rate into a heading and correcting θ with it would reproduce the wheel estimator inside the filter and inherit its unbounded drift. Treating the instantaneous rate difference as an observation of b_ω keeps each sensor supplying what it measures well: the wheels know rate, the gyroscope knows change, and neither knows absolute heading, so heading remains unobservable and its variance correctly continues to grow.

Measured scale errors are recorded but never divided out. Correcting them would mean calibrating the estimator with the ground truth it is about to be scored against; the scale error is a real property of encoder odometry and is retained.

The real robot: a different baseline

None of the three estimators above can be run on the real recordings. The rig publishes no wheel odometry in metric units and no joint states, so there is no wheel speed to integrate and no steering angle to put through the bicycle model; the wheelbase, 0.60 m, is known but has nothing to multiply. What the bags do carry is a firmware-unit speed feedback and a gyroscope (section 3.4.7), which support one estimator:

$$\theta(t) = \theta_0 + \int (\omega - b)\,\mathrm{d}t, \qquad \begin{bmatrix} p_x \\ p_y \end{bmatrix} \mathrel{+}= k\, v_{\text{fb}} \begin{bmatrix} \cos\theta \\ \sin\theta \end{bmatrix} \Delta t, \tag{3.13}$$

with b the gyroscope bias measured over the stationary head of each recording and k the unknown speed scale. `script/real_wheel_baseline.py` fits k, the gyroscope axis and its sign on one recording by minimising the planar ATE after rigid $SE(2)$ alignment, then holds all three on the others: the fit recording's score is a calibration and only

the held recordings are results. Because no yaw-rate observation exists, there is no bias state to correct and the EKF of equation (3.11) degenerates to this integration, so no filter row is reported in the real domain.

3.6.2 Experiment 2: Offline YOLO detector performance

Objective. Measure standalone detector performance independent of SLAM, so that later filtering results can be attributed: how reliably the detector finds and names warehouse objects, and how much of the frame a mask built from its boxes would remove.

Process. The detector is run on every left-camera frame of the four realistic recordings; ground-truth boxes are constructed by projecting the collision mesh of every model the detector has a class for into the camera from the simulator's pose stream; predictions are matched by class at an IoU threshold; localisation and class assignment are scored separately. The checkpoint is the four-class model `weight/runs_1/detect/train/weights/best.pt`, whose classes are bin, box, bucket and forklift; the fourth fires on the pallet jacks, which are the moving objects of the dynamic recordings, so moving-object recall is measurable. The walking people are simulator actors with neither a pose in the simulator's stream nor a collision mesh, and they are outside the class set, so no ground-truth box can be projected for them and they are scored on neither side.

Evaluation metrics. Precision, recall, F1, recall on the moving subset, image-area coverage, and conditional class precision, recall and F1. Recall is reported on two bases, because the scorer exempts truth boxes under twelve pixels or more than 70 % covered by nearer geometry: over the boxes the exemptions keep, which is the detector's task, and over every projected box, which is what is physically in view.

Validity rule. Overlays are inspected before any aggregate is accepted, because invalid pose stamps, mesh units or camera transforms produce plausible but wrong boxes.

Detector precision, recall, TP, FP, FN, moving/static recall, IoU, and image-area coverage are computed by `yolo_eval.py` at recorded confidence, matching-IoU, occlusion, minimum-size, and motion thresholds. Mask-use statistics are joined to trajectories by `timestamp_ns`. A semantic class is never treated as ground-truth motion.

Constructing the ground-truth boxes The detector is scored against boxes computed from the simulator's own state rather than against hand annotation. No image is labelled by hand anywhere in this project. The chain from a recorded object pose to a box in pixels is as follows; all of it is in `yolo_eval.py` and `yolo_gt.py`.

1. **Geometry, not just pose.** A pose is a point and a box needs extent, so `yolo_gt.vertices()` reads the prop's collision mesh (`.dae`) from the Gazebo model directory and returns its vertices V in the model frame. Meshes above 4000 vertices are decimated with a fixed seed: a few thousand points bound a silhouette as tightly as 100 000 and keep the per-frame cost flat.
2. **Object to world.** The pose (R_{wo}, p_{wo}) recorded for that object places the vertices: $v_w = R_{wo}V + p_{wo}$.
3. **Camera pose.** The robot pose gives T_{wb}, and the camera is placed on the body by the extrinsic T_{bc} read from `config/wil_sim/stereo_imu.yaml`, so $T_{wc} = T_{wb}T_{bc}$. Reading the extrinsic from the estimator's own configuration file is deliberate: the evaluation and VINS-Fusion then cannot disagree about where the camera is.
4. **World to camera**, by inverting that transform, using the transpose because the rotation

is orthonormal: $R_{cw} = R_{wc}^{\top}$, $t_{cw} = -R_{cw}t_{wc}$, $v_c = R_{cw}v_w + t_{cw}$.

5. **Projection.** Vertices with $z \leq 0.05\,\mathrm{m}$ are discarded *before* the division, then the remainder are projected through an ideal pinhole:

$$u = f_x \frac{x_c}{z_c} + c_x, \qquad v = f_y \frac{y_c}{z_c} + c_y. \tag{3.14}$$

6. **Box.** The axis-aligned extent of the projected points, $[\min u, \max u] \times [\min v, \max v]$, clipped to the image.

The z guard in step 5 is not a formality. A vertex behind the camera has negative z, and the division then mirrors it to the wrong side of the image, producing a well-formed box in entirely the wrong place — a failure that looks like a detector error rather than a projection error. Objects that straddle the image plane are projected from their visible vertices only.

Each object also stores $\tilde{z} = \mathrm{median}(z_c)$, which is what the occlusion test below uses to decide which of two overlapping objects is nearer.

Table 3.24 Parameters of the detection evaluation. The intrinsics are not calibrated: Gazebo renders an ideal pinhole, so $f_x = f_y = (W/2)/\tan(\phi/2)$ with every distortion term zero, and the values below are exact rather than estimated. The three exemption rules decide which ground-truth objects are scored at all, and are the reason the recall in section 4.2.1 is computed over a subset of the objects present.

Parameter	Value	Role
f_x, f_y	640, 640	ideal pinhole, 90° horizontal field of view
c_x, c_y	640, 360	principal point at image centre
Image size	1280×720	
Distortion	all zero	the renderer applies none
Mesh vertices per prop	≤ 4000	decimated with a fixed seed
Near-plane guard	0.05 m	vertices nearer than this are dropped before projection
Confidence, `-conf`	0.35	detections below this are never emitted
NMS IoU, `-iou-nms`	0.5	
Matching IoU, `-iou-match`	0.5	prediction counts as a hit above this
Motion threshold, `-move-thresh`	$0.05\,\mathrm{m\,s^{-1}}$	object is "moving" above this, from its pose history
Exemption: minimum size, `-min-box-px`	12 px	smaller ground-truth boxes are discarded and never appear
Exemption: occlusion, `-occlusion`	0.70	ground truth covered more than this by nearer objects is scored as neither hit nor miss (*yellow* in section 4.2.1)
Exemption: scenery cover, `-ignore-cover`	0.50	a prediction this covered by unlabelled scenery is neither credited nor penalised

What this convention costs Two consequences follow from scoring against projected geometry, and both bear on how the detector numbers in section 4.2.1 should be read.

The boxes are *tighter than a human would draw*. They are the exact silhouette of the collision mesh, whereas an annotator draws a looser rectangle and rounds generously. A detector trained on human boxes is therefore scored against a stricter convention than it

was fitted to, and loses IoU that represents no error on its part. Of the differences between how this detector is scored here and how it was scored during training, this is the one that cannot be removed without relabelling.

The exemptions are *not symmetric with the training convention*. A human-annotated set has no notion of an exempt object: every labelled box counts. The occlusion and minimum-size rules exist because a crate that is nine tenths behind a shelf is not a fair test of a detector, which is the right decision for reporting detector quality — but it means recall here is computed over the easier subset. The size of that effect is measurable from the stored counts: 34 476 of the 96 758 ground-truth objects across the five recordings are exempt, and recomputing recall with every one of them counted as a miss moves it from 0.593 to 0.382 (`script/yolo_generalization.py`). Any comparison against a training-set recall has to use the second figure, not the first.

3.6.3 Experiment 3: Visual-inertial odometry with dynamic-environment filtering

Objective. Determine whether masking dynamic-object features improves localization, measure the compute cost of doing so, and verify that tracking survives the filtering and that the added detector cost does not push either pipeline below the input frame rate.

Process, simulation. Accuracy and compute are measured on different recordings, because they need different designs.

Accuracy is measured on the four `dataset_allsensor_realistic` recordings, one run per cell. The factors are the estimator, the feature extractor (CPU or GPU) and the loop closer (active or disabled), each replayed at rate 1.0 from the bag with the node live, which is the deployment condition. The loop-closer state is read from each run's own log rather than from its name, because a run can be launched with the closer enabled and have it disabled by configuration.

Latency and resources are measured only on the two mapping recordings, each configuration run three times, so that a mean and a spread exist. One run per cell cannot support an uncertainty, and the accuracy recordings are too short to separate a steady state from the initialisation transient.

A filtered treatment, when one exists, is paired with the baseline of the same estimator, extractor and closer state, and the pair is admissible only when the filter demonstrably acted (matched detections, non-zero mask coverage, removed tracks), the two runs received the same input to within 5 % in image count and duration, and both ran on the same node binary. No filtered run exists in the current dataset set, so no pair has yet been formed. The collection protocol is given in section 3.7.3.

Process, real robot (planned). The online procedure has four steps. *Step 1, initialisation and time synchronisation:* power the NRU-51V, the stereo cameras and the IMU; start the optical tracker and calibrate the room. The tracker and the robot keep separate clocks and the recordings made so far are aligned after the fact by cross-correlating the tracker's speed with the wheel-speed feedback (section 3.7.1); synchronising the two clocks at record time, by PTP or Chrony, would remove that estimation step and the uncertainty it carries. *Step 2, execution and environment alteration:* drive from the origin in Zone A to the goal in Zone B at a linear speed not above $1.5\,\mathrm{m\,s^{-1}}$ through the Nav2 package; on arrival, trigger the dynamic variable by randomly re-placing the k stacks in Zone A; return, alter Zone B,

and alternate for five minutes per test cycle to represent a lifelong-mapping load. *Step 3, data logging:* record with rosbag2 the 6-DoF SLAM pose, the motion-capture reference, the detector boxes with confidences, and the whole-pipeline latency per frame. *Step 4, iteration over the experimental matrix:* repeat steps 1–3 for both pipelines under the three illumination levels. The `.db3` bags are then trajectory-extracted and timestamp-matched before evaluation.

Evaluation metrics. ATE and rotational RMSE after rigid alignment, RPE at $\Delta = 1\,\mathrm{s}$ (all pairs and jump-free), tracking-loss and reset events, retained input fraction, throughput, process CPU and peak memory, and per-module latency against the inter-frame budget.

Validity rule. A trajectory containing steps that imply more than $10\,\mathrm{m\,s^{-1}}$ is not read as ordinary drift; an indoor error above 50 m is divergence. A comparison across treatments is admissible only for input-matched pairs with an active filter. A timing comparison between two configurations is admissible only when their runs are interleaved from a cool start, because the laptop throttles at 90 °C and a block of consecutive runs in one configuration confounds the configuration with the thermal state.

3.6.4 Experiment 4: Dense map reconstruction and map-quality evaluation

Objective. Verify whether the reconstructed map accurately reflects the physical environment and is usable by a route planner.

Process. RTAB-Map builds a database from a mapping run; the occupancy grid is seated in the world by a fixed transform measured from the map alone; geometry is scored against the world's collision meshes; drivability is scored by stamping the true robot footprint along a ground-truth route that did not build the map.

Evaluation metrics. Occupied-cell precision, recall over observed surfaces and over the whole building, false-obstacle rate under the footprint, unmapped footprint poses, clearance statistics, and the graph's node and link counts.

Validity rule. Placement must be established independently before any metric is read, and a drivability figure computed against the run that built the map is a self-consistency check, not a result.

RTAB-Map grids are placed in the Gazebo world using one fixed rigid world-to-map transform measured by `script/fit_map_pose.py`. It is constant for a database; the live `map` $\rightarrow$ `odom` drift correction is not reused for map placement. On a warehouse of parallel aisles the placement search has a rival 180° away that can score higher than the true pose, so a fit is accepted only after its overlay is read, and the tool takes a `-yaw-near` hint for the basin the spawn implies.

`script/validate_map.py` rasterises world collision geometry and evaluates occupied cells at a stated tolerance. It reports occupied-cell precision, whole-world and observed-region recall, false obstacles under the true 0.83×0.50 m footprint, unmapped footprint poses, and clearance. Graph evaluation records neighbour, global-closure, local-space, and gravity links. Localisation additionally requires time to first fix, accepted and rejected matches, database checksums before and after read-only operation, pose continuity, and known-world bounds. Finite covariance or a localisation status message alone does not establish correctness.

A database is always probed with a recording that did not build it, because the build route

is drivable on its own map by construction: the camera cleared the cells the robot passed through, so a self-probe measures the route and not the map. The five databases evaluated are `map/map_tiny/map_tiny.db` against `small_warehouse/small_warehouse_static_tinymap.world`, and the four realistic-warehouse databases, one per layout under each of two build configurations, against the static twin of their layout and probed with `dataset_allsensor_realistic_000` and `_001` (section 4.4.1). The tiny map's own independent probe recording is lost, so its drivability figure cannot be regenerated.

3.7 Evaluation Metrics and Analysis

3.7.1 Trajectory evaluation

`script/vio_metrics.py` associates estimate and reference over their common time interval and interpolates the reference at estimator timestamps. A rigid Umeyama $SE(3)$ fit maps estimated positions to the reference with scale fixed at one. Fixed scale is appropriate for stereo-inertial estimators and preserves scale error. Outputs include translational ATE RMSE, maximum and final error, rotational geodesic error, path length, overlap duration, pose count, jump count, and longest jump-free segment. Optional startup trimming is relative to each estimator's first overlapping pose, preventing unequal initialization latency from being interpreted as steady-state behaviour.

The older `plot_vio_vs_gt.py` provides a first-pose-aligned diagnostic, but headline comparison uses common $SE(3)$ alignment. Real data without independent ground truth receives no ATE.

Relative pose error. RPE was added to `script/vio_metrics.py` after the reporting cutoff, on 18 September 2026, and is **implemented and exercised on the full campaign**. It follows the conventional definition: for a pair of poses separated by a fixed interval Δ, the error transform is

$$E_i = \left(Q_i^{-1} Q_{i+k}\right)^{-1} \left(P_i^{-1} P_{i+k}\right), \tag{3.15}$$

with P the estimate and Q the reference. Translational RPE is $\|\mathrm{trans}(E_i)\|$ and rotational RPE is the geodesic angle of $\mathrm{rot}(E_i)$, the same angle measure used for orientation error above.

RPE is reported because it needs *no* global alignment, and that is what makes it the appropriate metric for the dynamic runs. A single discontinuity re-seats the rigid $SE(3)$ fit, so every subsequent pose inherits the offset and ATE describes the jump rather than the tracking. RPE instead asks a local question — over the next Δ seconds, did the estimate move as the reference moved — and is insensitive to both the global frame and accumulated drift. It therefore supersedes the longest-jump-free-segment statistic, which answers a similar question through a segment-selection heuristic rather than a standard definition.

Four intervals are swept, $\Delta \in \{0.5, 1, 2, 5\}$ s, giving an error-versus-interval curve; $\Delta = 1$ s is the headline value and corresponds to roughly 1.4 m at the simulated robot's cruising speed. A pair is admitted only when its realised separation lies within 25 % of Δ, so a gap in the log cannot silently widen the interval and inflate the error; the surviving pair count is reported with every statistic. Pairs are overlapping, which is the conventional default.

Each statistic is computed twice: over all admissible pairs, and over only those pairs that

do not span a detected tracking jump. The difference isolates the teleport contribution from ordinary local error, mirroring the existing separation between global and clean-segment ATE. Both RMSE and median are reported, because the dynamic-run distributions are strongly right-skewed and a single teleport otherwise sets the RMSE.

The implementation is verified against synthetic trajectories with known answers: an estimate identical to the reference returns zero; a constant relative drift of $0.1\,\mathrm{m\,s^{-1}}$ returns $0.100\,\mathrm{m}$ at $\Delta = 1\,\mathrm{s}$ and $0.200\,\mathrm{m}$ at $\Delta = 2\,\mathrm{s}$; a $2\,^{\circ}\,\mathrm{s^{-1}}$ relative rotation returns 2.00° at $\Delta = 1\,\mathrm{s}$; and an injected teleport separates the all-pairs and jump-free statistics by the expected pair count. Results are written to `rpe_metrics.csv` for the interval sweep, with the $\Delta = 1\,\mathrm{s}$ values carried into `trajectory_metrics.csv`.

Planar reference (added after the cutoff). The real recordings of section 3.4.7 carry an optical-tracker reference that differs from the simulator's in three ways, and `script/eval_real_gt.py` handles each explicitly. It is planar and has no heading, so alignment is the rigid $SE(2)$ case of the fit above with scale fixed at one, ATE is evaluated in the plane at the tracker's own samples, and RPE is translational only. Its clock is not the robot's: the offset is estimated by cross-correlating the tracker's speed with the bag's wheel-speed feedback, which is independent of both estimators, and is accepted only when the correlation peak is sharp ($r \geq 0.9$ and at least 0.05 above any peak more than 1 s away); the ATE is re-evaluated at ± 0.1 and $\pm 0.2\,\mathrm{s}$ so that its sensitivity to the offset is visible. And the tracker follows a marker, not the camera origin: the body-fixed lever arm must be supplied, every result is labelled provisional until it is measured, and a least-squares fit of the arm is reported as a diagnostic but never applied, because it would absorb estimator error. A run is *completed* when it shuts down gracefully with no reset, no jump and at least 80 % of the tracker samples matched; *partial* when it recovers from a reset or falls short on coverage; and *failed* on divergence, a reset storm, coverage below 20 %, or a vertical range above 1 m on a floor run. The last rule was added on 20 October 2026 after `dataset_realv3_000`, on which VINS-Fusion satisfied every earlier rule while climbing 22 m; it is stated here as a post hoc addition.

3.7.2 Computational performance

Both estimator processes write per-frame `performance.csv` files. These record processing time, delivery and backlog counters, tracking or solver state, initialization and reset evidence, pose and keyframe accounting, cumulative process CPU time, monotonic wall time, and resident memory. VINS additionally records feature-front-end and estimator-back-end stages; ORB-SLAM3 records its tracking front end, local mapping, place recognition, loop closure, map merging, and global bundle-adjustment events. CPU utilization and throughput can therefore be derived over explicit wall-time intervals. The measurements are process-local and exclude Gazebo, rosbag, RViz, RTAB-Map, and the separate YOLO process. Synchronized whole-system monitoring and the repeated Jetson/laptop matrix remain **planned**. Module timings from asynchronous stages are reported as separate distributions and are not added as if they formed one serial critical path.

3.7.3 Collection protocol, validity and stored artifacts

The sensor source, estimator, optional detector, and viewer start as separate processes; launch files do not automatically play bags. Baseline and filtered treatments run sepa-

rately on the same recording, at a replay rate that permits comparable input consumption. Competing estimators are not run simultaneously in a scientific A/B comparison.

The common trajectory file has no header and uses:

```
t_ns, px, py, pz, qw, qx, qy, qz, vx, vy, vz
```

Timestamps are integer nanoseconds from the source header and quaternion order is w,x,y,z. ORB-SLAM3 writes this representation directly; VINS writes the state after nonlinear optimization. Simulation truth is extracted to the same shape by `script/extract_gt.py`. The tracker-use record `yolo_mask.csv` remains separate from the offline detector score.

Table 3.25 Principal stored artifacts.

Artifact	Purpose
`<run>/vio.csv`	Estimator trajectory and velocity
`<run>/ground_truth.csv`	Simulation reference trajectory, when available
`<run>/yolo_mask.csv`	Actual mask matching, coverage, and rejection
`<run>/run_metadata.csv`	Resolved configuration and supplied provenance
`<run>/performance.csv`	Per-frame timing, state, delivery, CPU, and memory
`<run>/frontend.csv` and `<run>/backend.csv`	VINS feature-tracking, estimator-stage, solver, and state-window diagnostics
`<run>/tracking_frontend.csv`	ORB-SLAM3 tracking-stage timing and feature counts
`<run>/local_mapping.csv`	ORB-SLAM3 keyframe insertion, map-point, and local-BA diagnostics
`<run>/loop_closing.csv`	Place-recognition, loop, merge, and global-correction events
`<run>/run_summary.csv`	Graceful-shutdown validity and robustness totals
`<run>/events.csv`	VINS initialization, reset-cause, and shutdown events
`output/yolo_eval/<experiment>/yolo_eval.csv`	Per-frame detector evaluation
`output/yolo_eval/<experiment>/yolo_detections.csv`	Per-detection matching
`output/yolo_eval/<experiment>/overlay/`	Projection and matching validation
`map/*.db` or `<run>/rtabmap.db`	Persistent RTAB-Map graph and signatures
`output/compare/<domain>/<experiment>/`	Derived figures and summaries

The current analysis code requires at least ten overlapping samples. Scientific acceptance additionally requires the same input interval, successful initialization, comparable pose and frame counts, and no unexplained sensor starvation. Steps implying over $10\,\mathrm{m\,s^{-1}}$ are tracking jumps; an indoor estimate with error beyond 50 m is divergence rather than ordinary inaccuracy. Without real-data ground truth, path length and mean speed are plausibility diagnostics, not accuracy measurements.

The estimator now records resolved local configuration and optional experiment, dataset, world, replay-rate, command, and note fields in `run_metadata.csv`. It does not

automatically determine bag health, repository revisions, complete dependency versions, external-process state, or the analyst's exclusion decision. Empty optional fields therefore remain incomplete provenance. Historical folders created before this instrumentation remain preliminary unless their provenance is recoverable from logs and engineering records.

3.7.4 Reproducibility controls

The following controls govern future experiments and acceptance of historical results:

1. Record absolute bag and exact `.world` file, treatment, configuration, commands, clock and replay rate, hardware, dependencies, and relevant repository commits.
2. Use a unique output directory and retain failed runs with failure labels.
3. Run paired treatments independently on the same sensor data and compare only their common valid interval.
4. Record inputs, outputs, initialization, dropped frames, resets, tracking loss, warnings, and exclusion reasons.
5. Validate camera order, transport, timestamps, metric scale, and every rigid transform before analysis.
6. Inspect representative detector and map overlays before aggregate scores.
7. Repeat nondeterministic RTAB-Map processing and report valid-run count; do not interpret differences within run-to-run variation as algorithmic.
8. Keep estimator, back end, filtering, environment dynamics, floor texture, domain, and hardware separate unless their interaction is the declared factor.
9. Record every switch that changes estimator behaviour in the run manifest, not only those the launch file exposes. Loop-closing and global-bundle-adjustment state are currently absent from `run_metadata.csv`, so two ORB-SLAM3 runs of the same dataset in opposite configurations produce manifests that differ only in identifier and timestamp.
10. Repeat every configuration of a nondeterministic estimator at least three times and report median and spread. ORB-SLAM3 is multi-threaded and its run-to-run variation on identical input is presently unmeasured, which places a floor on the differences that can be attributed to any factor.

3.8 Planned Versus Implemented

Material deviations from the originally proposed method are given in table 3.26. They are differences between the proposal and the system that exists, not failures: each is recorded so that no proposal capability is read into the implemented pipeline.

These deviations define the actual system at the reporting cutoff. Chapter 4 reports only measurements satisfying the validity rules above; failed, excluded and retracted runs are kept in a clearly labelled diagnostic record.

The principal evidence comprises `README.md`; the four `MEMORY_CLAUDE/README.md` engineering records; the ORB-SLAM3 integration record; `docs/report/project_limitations_and_map_benefits.md`; the source, launch, and configuration files cited in this chapter; and `script/vio_metrics.py`, `script/extract_gt.py`, `script/yolo_eval.py`, `script/fit_map_pose.py`, and `script/validate_map.py`. These project artifacts establish local implementation. They do not replace

Table 3.26 Planned-versus-implemented deviations at the reporting cutoff. "Added" marks work not present in the original proposal.

Component	Proposed	Implemented	Status
Semantic segmentation	pixel masks, as in the reference RY-SLAM approach	four-class bounding boxes (bin, box, bucket, forklift), no motion gate in the reported configuration	deviation
Dynamic rejection point	not specified	ORB-SLAM3 rejects before octree distribution; VINS-Fusion also removes persistent KLT tracks	implemented
Dynamic filtering result	accuracy with and without masking	the mechanism exists in both pipelines, but no filtered run exists in the current dataset set	untested
Estimator coupling	shared output representation	two independent estimators, not one fused estimator	deviation
RTAB-Map correction	integrated back end	separate process consuming external TF odometry; correction never injected into estimator state	deviation
Real-robot evaluation	metric accuracy against ground truth	planar accuracy against an optical tracker on four recordings, provisional until the marker-to-IMU lever arm is measured	partial
Metric infrastructure	resource, throughput, uncertainty, manifests	implemented; latency and resources measured over repeated runs on the two mapping recordings	implemented
Relocalisation and merging	robust arbitrary-start localisation, aliasing rejection, extend-mode validation, front-end global optimisation, multi-session merging	partially verified relocalisation only	planned
Navigation integration	not proposed	`vslam_navigation`: one entry point per task per domain for mapping, relocalisation and planner bring-up, with a localisation gate and, on the real rig, a bicycle-model command bridge (section 3.3.6); planning itself stays outside the evaluated scope	added
Non-visual baseline	not proposed	wheel, IMU and EKF estimators in simulation with a data-derived noise model, and wheel-plus-gyroscope dead reckoning on the real robot (section 3.6.1)	added
Covariance consistency	not proposed	NEES and σ-coverage testing for the non-visual	added

Table 3.27 Original objectives (section 1.2) against the work reported here.

Objective	Status at the cutoff
1. Visual-inertial odometry with lifelong mapping	**partial**: two VIO front ends implemented; RTAB-Map mapping and read-only localization implemented; multi-session merging unverified (table 3.26)
2. Low-light image enhancement with EnlightenGAN	**not pursued**: no implementation in the repository
3. Dynamic-object perception by deep-learning detection	**implemented**: four-class YOLO26 detector, evaluated offline (section 4.2)
4. Lane detection with PolyNET and PID lane tracking	**not pursued**: no implementation in the repository
5. Integrated system with a 2-D cost map and localization error below 10 cm under differing illumination	**planned**: the integrated live test of section 3.6.3 has not been run; illumination has not been varied in any recording. The bring-up the test needs now exists and serves the prior map to a planner (section 3.3.6), verified offline only

primary literature citations for the scientific origin of the algorithms.

CHAPTER 4 EXPERIMENTS AND RESULTS

Reporting cutoff: 7 November 2026. On 6 November the dataset set was fixed to the realistic-warehouse family in simulation (`dataset_allsensor_realistic_000–003`, the two mapping recordings and `dataset_map_tiny`) and to `dataset_realv6_000` and `dataset_realv7_000–002` on the real robot; every result drawn from any other recording was retired and is not reported. Table 4.1 gives the status of the eight result slots. All four simulation slots hold data; two real-robot slots hold provisional data and two are planned, for the reason stated in each. Each filled slot reports what was measured and its validity limits. Passages whose evidence is not yet trustworthy, and the status note of each slot, are collected in the separate Open Concerns document (`docs/report/concerns/concerns.pdf`), generated from this chapter's source.

Table 4.1 Status of the eight result slots at the reporting cutoff.

Experiment	Simulation	Real robot
1. Non-visual baseline and estimator comparison	**completed**: four realistic recordings, two layouts, with and without traffic; EKF at the visual systems' inertial noise	**partial**: wheel+gyro dead reckoning with a fitted speed scale on four tracker-truthed recordings; at least as accurate as the best visual run on each held recording; no EKF (no steering measurement)
2. Offline YOLO detector performance	**completed**: four realistic recordings, 12 698 frames, four-class checkpoint; precision 0.75, moving-object recall 0.29	**planned**: no annotated frames
3. VIO with dynamic-environment filtering	**partial**: twenty unfiltered runs, one per cell; the loop closer decides ORB-SLAM3's error; no filtered run; latency and resources from eighteen repeats on the two mapping recordings	**partial**: ORB-SLAM3 provisional on `realv6` and `realv7`; VINS-Fusion has no valid run; no filtered run
4. Dense map reconstruction and map quality	**partial**: tiny map passes; four realistic maps fail the criterion; large map **failed**	**planned**

4.1 Experiment 1: Non-Visual Baseline and Estimator Comparison

4.1.1 Simulation

Objective. Establish whether a non-visual estimator provides usable localisation on the same recordings the visual pipelines consumed, and whether each estimator's reported uncertainty describes its error.

Process. `script/proprio_estimator.py calibrate` measured each record-

ing's wheel and inertial biases and wheel noise against ground truth from its stationary head and its driven length; `run` produced the wheel, IMU and EKF trajectories offline; `script/analyse_proprio.py` scored them with the live VINS-Fusion and ORB-SLAM3 runs over the common interval. The EKF reported here does *not* use the measured inertial noise: by the design decision of 6 November 2026 its three inertial noise terms are the values both visual configurations declare (`config/wil_sim/stereo_imu.yaml`: gyroscope density 0.002, accelerometer density 0.02, gyroscope bias walk 1×10^{-4}), converted to per-sample σ at 200 Hz by `script/make_vslam_noise_yaml.py`, so that the proprioceptive and visual families run on the same hand-set inertial model; biases and both wheel σ have no visual counterpart and stay measured. The measured-noise filter is run beside it as a diagnostic row and is not the baseline.

Datasets. Table 4.2: the two static twins and the two traffic recordings of the realistic worlds, both layouts. Traffic is noted as a condition of a recording, not treated as a factor, since no two recordings share a route.

Table 4.2 Recordings of Experiment 1 (names without the `dataset_allsensor_` prefix), from the bags. Route is the ground-truth path length; samples are ground-truth odometry messages.

Recording	Layout	Traffic	Route [m]	Span [s]	Samples
`realistic_000`	U	none (static twin)	171.3	125.5	6 278
`realistic_001`	I	none (static twin)	133.0	96.9	4 847
`realistic_002`	U	pallet jacks and people	102.7	79.5	3 975
`realistic_003`	I	pallet jacks and people	158.0	117.0	5 852

Reporting rules. Aligned ATE is RMSE after a rigid $SE(3)$ fit with scale fixed, the convention of every slot; anchored error is the direct difference from ground truth with no alignment and is defined only for the estimators that start from the ground-truth pose; aligned rotation is the geodesic angle after the same fit, defined for every estimator; ANEES is the average normalised estimation error squared over decorrelated 1 Hz samples, 3.0 for a consistent planar filter. Ground truth is the bag's `/ground_truth/odometry` moved from the model origin to the IMU point every estimator reports (lever arm 0.338 m forward, 0.192 m up). The visual runs are pinned by name: VINS-Fusion `cpu`; ORB-SLAM3 `loopclosing_gpu` and `noloopclosing_gpu`, both with the GPU feature extractor, at replay rate 1.0. The loop-closer state is read from each run's `loop_closing.csv` (patch-7 signature: sub-millisecond rows mean a disabled closer), not from the run name. On `realistic_002` the directory named `loopclosing_gpu` first held a run whose manifest names the `realistic_003` bag (3 535 stereo pairs received against the 2 409 that recording holds); it was scored against the wrong truth for a day, is set aside under `trash/output/misfiled_20261106/`, and the run reported here was made on the right bag on the evening of 6 November (2 382 pairs received). Sources: `output/compare/simulation/exp1-realistic/` (`proprio_metrics.csv`, `proprio_rpe_metrics.csv`, the noise models under `noise/`), run directories under `output/output_{wheel,imu,ekf,ekf_measured}/simulation/`. One run per cell, $n = 1$.

Results. At the visual systems' inertial noise the EKF holds 0.68–2.80 m aligned ATE and is behind VINS-Fusion on all four recordings; the measured-noise diagnostic holds 0.21–0.58 m. Table 4.3 gives every estimator on every recording, table 4.4 the relative pose error at 1 s, and figure 4.1 the trajectories.

Table 4.3 Estimator comparison on the four realistic recordings. The closer state of each ORB-SLAM3 run is read from its artefact. ANEES is reported for the proprioceptive estimators, which carry a covariance; a consistent filter gives 3.0, and every value here is above it. **Bold** marks the lowest aligned ATE of a recording; the measured-noise filter is a diagnostic row, not the baseline, so it is shown bold where it is lowest but is not the slot's result.

Recording	Estimator	Closer	Anch. RMS [m]	Anch. final [m]	Aligned ATE [m]	Aligned rot. [°]	Yaw RMS [°]	ANEES
`realistic_000`	Wheel odometry	–	5.89	5.66	2.56	9.14	18.05	1810
	IMU dead reckoning	–	36.19	76.01	23.02	177.21	0.84	454
	EKF, VSLAM-config noise	–	2.49	3.42	1.34	5.83	10.62	19
	EKF, measured noise (diag.)	–	0.33	0.35	**0.23**	0.47	0.43	111
	VINS-Fusion	–	–	–	0.29	1.85	–	–
	ORB-SLAM3 GPU, LC run	active	–	–	0.29	12.80	–	–
	ORB-SLAM3 GPU, no-LC run	disabled	–	–	0.52	12.84	–	–
`realistic_001`	Wheel odometry	–	1.40	2.90	1.03	6.04	7.27	176
	IMU dead reckoning	–	15.40	39.82	13.05	0.25	0.55	160
	EKF, VSLAM-config noise	–	0.94	0.83	0.68	3.81	4.51	8
	EKF, measured noise (diag.)	–	0.26	0.26	**0.21**	0.33	0.36	31
	VINS-Fusion	–	–	–	0.22	1.18	–	–
	ORB-SLAM3 GPU, LC run	active	–	–	4.78	18.98	–	–
	ORB-SLAM3 GPU, no-LC run	disabled	–	–	0.40	15.13	–	–
`realistic_002`	Wheel odometry	–	6.54	6.08	4.81	17.03	25.00	1778
	IMU dead reckoning	–	25.88	39.17	19.01	118.74	1.91	695
	EKF, VSLAM-config noise	–	2.06	3.29	1.58	9.92	13.65	37
	EKF, measured noise (diag.)	–	0.59	0.30	0.37	0.70	1.03	191
	VINS-Fusion	–	–	–	0.66	3.57	–	–
	ORB-SLAM3 GPU, LC run	active	–	–	0.40	16.04	–	–
	ORB-SLAM3 GPU, no-LC run	disabled	–	–	**0.17**	16.08	–	–
`realistic_003`	Wheel odometry	–	6.71	7.19	4.30	19.24	35.82	3399
	IMU dead reckoning	–	13.72	35.47	11.60	180.00	1.68	140
	EKF, VSLAM-config noise	–	4.44	6.23	2.80	15.92	28.01	158
	EKF, measured noise (diag.)	–	0.79	1.49	0.58	3.90	5.72	106
	VINS-Fusion	–	–	–	**0.27**	1.51	–	–
	ORB-SLAM3 GPU, LC run	active	–	–	0.50	13.16	–	–
	ORB-SLAM3 GPU, no-LC run	disabled	–	–	1.91	13.53	–	–

Table 4.4 Relative pose error at $\Delta t = 1\,\mathrm{s}$, RMSE over all pairs and over pairs that span no detected jump (a step above $10\,\mathrm{m\,s^{-1}}$ between consecutive poses); the jump count is per run. **Bold** marks the lowest jump-free translational RPE of a recording.

Recording	Estimator	Jumps	Translation [m]		Rotation [°]	
			all pairs	clean	all pairs	clean
`realistic_000`	Wheel odometry	0	0.028	0.028	1.720	1.720
	IMU dead reckoning	0	1.941	1.941	0.023	0.023
	EKF	44	0.240	0.218	0.979	0.893
	EKF	0	0.039	0.039	0.098	0.098
	VINS-Fusion	0	0.021	0.021	0.443	0.443
	ORB-SLAM3 GPU, LC run	3	0.246	0.017	5.902	0.094
	ORB-SLAM3 GPU, no-LC run	3	0.254	**0.013**	5.909	0.090
`realistic_001`	Wheel odometry	0	0.033	0.033	1.733	1.733
	IMU dead reckoning	0	0.980	0.980	0.023	0.023
	EKF	42	0.254	0.202	0.964	0.811
	EKF	0	0.045	0.045	0.090	0.090
	VINS-Fusion	0	0.018	**0.018**	0.366	0.366
	ORB-SLAM3 GPU, LC run	7	1.191	0.034	7.031	0.120
	ORB-SLAM3 GPU, no-LC run	2	0.302	0.020	6.845	0.155
`realistic_002`	Wheel odometry	0	0.027	0.027	1.997	1.997
	IMU dead reckoning	0	1.505	1.505	0.033	0.033
	EKF	67	0.243	0.158	0.985	0.780
	EKF	0	0.035	0.035	0.075	0.075
	VINS-Fusion	0	0.038	0.038	0.790	0.790
	ORB-SLAM3 GPU, LC run	4	0.463	**0.017**	10.403	0.084
	ORB-SLAM3 GPU, no-LC run	1	0.147	0.028	7.422	0.070
`realistic_003`	Wheel odometry	0	0.023	0.023	1.447	1.447
	IMU dead reckoning	0	0.936	0.936	0.023	0.023
	EKF	22	0.202	0.185	1.026	0.969
	EKF	0	0.036	0.036	0.154	0.154
	VINS-Fusion	0	0.014	**0.014**	0.327	0.327
	ORB-SLAM3 GPU, LC run	3	0.165	0.028	6.068	0.174
	ORB-SLAM3 GPU, no-LC run	3	0.151	0.036	6.088	0.333

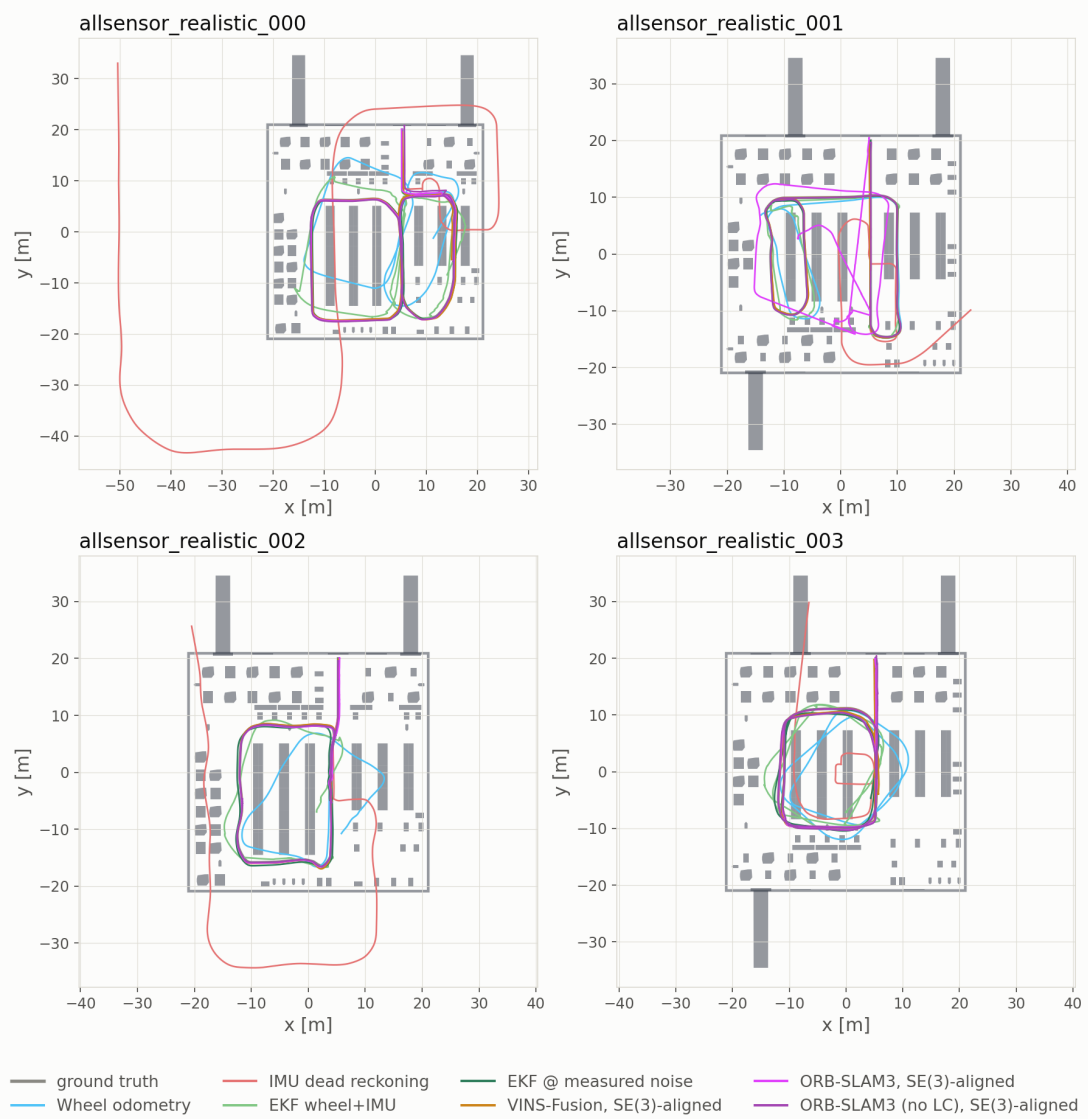


Figure 4.1 Estimator comparison on every realistic recording, world frame: the dead-reckoning estimators as estimated from the ground-truth start pose, the visual estimators after the rigid $SE(3)$ alignment used for their metrics, one legend for all four panels. The IMU leaves the building on every recording; the EKF at the visual systems' noise (light green) wanders in heading where the measured-noise filter (dark green) stays on the route; ORB-SLAM3's divergent runs are the excursions on `001` (loop closer active) and `003` (closer disabled).

Analysis. With both families on the same hand-set inertial model the ordering is settled: VINS-Fusion is the most accurate estimator on every recording (0.22–0.66 m) and the EKF, at 0.68–2.80 m, is behind it on all four and behind the better ORB-SLAM3 run on three. The measured-noise diagnostic puts the same filter at 0.21–0.58 m, ahead of VINS-Fusion on three of four, so the whole of the EKF's deficit is the noise model and none of it the architecture — the September control, reproduced on new recordings and larger.

The mechanism is the one isolated in September. The configuration's gyroscope density is five to seven times the measured value, which inflates the heading process noise forty-fold; with the heading unobserved except through the wheel yaw-rate residual, that residual steers the heading instead of the bias, and yaw RMS rises from 0.4–5.7° to 4.5–28.0°. On routes of 100 to 170 m with repeated turns that heading error is what the aligned ATE measures.

ORB-SLAM3's divergences are not a loop-closure effect. With the GPU front end six of eight runs hold 0.17–0.52 m and two diverge, to 4.78 m with the closer active on `realistic_001` and to 1.91 m with it disabled on `realistic_003`, while the other closer state on each of those recordings holds 0.40 and 0.50 m. Each divergent run carries three or seven jumps (table 4.4) and its clean relative error is as low as the stable runs', so the error is a few discrete events, not steady-state tracking. Which front-end condition produces them is not isolated here; the CPU runs of the same recordings belong to Experiment 3, where the same pattern recurs.

Traffic appears as a condition, not a tested factor. VINS-Fusion is 0.66 m on the U traffic recording against 0.29 m on its static twin and 0.27 m against 0.22 m on the I layout; the proprioceptive estimators are unaffected by construction. The U recordings do not share a route, so the difference cannot be credited to traffic alone.

The complementary failure of the two proprioceptive sensors holds. The IMU keeps heading to 0.5–1.9° RMS and loses position by 12–23 m; the wheels keep position to 1.0–4.8 m and lose 7–36° of heading. Every proprioceptive covariance is overconfident: ANEES 8–3399 against 3.0, the EKF at the configuration noise least so because its inflated process noise is closer to the error it actually makes.

Limitations, each stated once. *n:* one run per cell, no uncertainty; ORB-SLAM3's two divergences out of eight are the only run-to-run figure in the slot. *Equal handicap, not equal effort:* the shared inertial model was fitted to neither family. *Idealised wheels:* simulated encoders with slip under 1 %, the proprioceptive estimators' best case. *Route and traffic confounded:* no two recordings share a route. *Run bookkeeping:* one directory held a run on another recording for a day; the manifest caught it, and the launch should refuse an output directory whose name disagrees with its dataset.

Summary. On the realistic recordings IMU-only dead reckoning drifts by 12–23 m and wheel odometry by 1.0–4.8 m of aligned ATE; the EKF, run on the visual systems' own inertial noise as the design requires, holds 0.68–2.80 m and loses to VINS-Fusion on every recording, while the same filter on a measured noise model would hold 0.21–0.58 m. VINS-Fusion holds 0.22–0.66 m; ORB-SLAM3 with the GPU front end holds 0.17–0.52 m on six runs of eight and diverges on two, one in each closer state. Against the project's target of 0.05 m ATE no estimator qualifies; against the 1.5 m RPE bound at 1 s, all but the IMU do.

Future plan. Fit the visual systems' inertial noise to this platform rather than copying a file, since the slot shows the noise model is worth more than the estimator; feed the VIO pose into the EKF as a third measurement; and bridge the real robot's wheel speed in metric units so that this slot's real-robot half can be formed.

4.1.2 Real robot

Objective. The real-robot half of the question of section 4.1.1: does a non-visual estimator provide usable localisation on the recordings the visual pipelines consumed, and do the visual pipelines earn their place over it.

Process. The bags carry `/feedback/velocity` and `/feedback/steering` (50 Hz, `std_msgs/Float64`) and the WitMotion yaw sensor on `/hwt101ct_yaw_publisher` (200 Hz). The velocity is in a firmware unit, not metres per second; the steering is a command echo that takes two values (0 and −30) on every recording and so carries no yaw rate; the yaw sensor's orientation field is identity and its rate on the y axis is the live gyro (the sensor is mounted with y vertical: that axis integrates to 6.29 rad over `realv7_000`, whose tracker heading sweeps 6.3 rad; x and z integrate to under 0.05 rad).

`script/real_wheel_baseline.py` integrates heading from that gyro with its rest bias removed (3–7 s at rest at the head of each bag) and position from the scaled speed. The scale k, the gyro axis and its sign are fitted on `realv7_000` alone by minimising the planar ATE after rigid $SE(2)$ alignment with scale fixed (a wrong sign mirrors the path, which a proper rotation cannot align), then held: the fit recording's own score is a calibration, the three held scores are the result. The fitted $k = 0.325$ agrees with the plain path-length ratio of each `realv7` bag (0.323 to 0.329). The time offset between tracker and bag is the wheel-speed cross-correlation of `script/eval_real_gt.py` ($r = 0.93$ to 0.99 on `realv7`, 0.61 on the sparse `realv6` export), scoring is that script's planar procedure at the tracker's own samples, and the one off-path `realv6` sample is removed by the rule the realv6 analysis fixed.

The rig's wheelbase, 0.60 m, is recorded and unused: with no steering measurement there is nothing to put through a bicycle model, so the yaw-rate observation that gives the simulation EKF its bias state does not exist, and that filter would reduce to this integration.

Datasets. The four tracker-truthed real recordings of Experiment 3's real-robot section: routes of 22–37 m over 97–219 s. Sources: `output/compare/real/exp1-wheel/` (`wheel_metrics.csv`, `manifest.json`), estimates under `output/output_wheel/real/`.

Results. Table 4.5 and figure 4.2. On the fit recording the baseline reads 0.06 m ATE with a path ratio of 0.99, which is the calibration. Held, it reads 0.26 and 0.22 m on `realv7_001` and `_002` with path ratios of 0.99 and 1.01, and 0.46 m on `realv6_000` against its sparse export, with no jump and a relative error at 1 s of 0.01–0.02 m.

Analysis. The comparison this slot exists for now has a number on both sides, and it does not favour vision. On the same recordings the best ORB-SLAM3 configuration reads 0.44 m on `realv7_001`, 0.47 m on `realv7_002` and 0.46 m on `realv6_000` (Experiment 3, real-robot section, after the scorer's frame repair); the scaled dead reckoning reads 0.26, 0.22 and 0.46 m, with path ratios within 4 % of one where the visual

Table 4.5 Wheel+gyro dead reckoning on the real recordings against the optical tracker: the feedback speed scaled by k (fitted on `realv7_000` by minimising planar ATE, held on the others), heading from the integrated gyro with its rest bias removed, rigid $SE(2)$ alignment with scale fixed, lever arm zero and unmeasured. ATE at the tracker's samples; path ratio is estimated over true path length.

Recording	Role	k	Offset [s] (r)	Samples	ATE RMSE [m]	ATE max [m]	Path ratio	$\mathrm{RPE}_{1\,\mathrm{s}}$ [m]
`realv7_000`	fit	0.325	-2.57 (0.99)	491	0.057	0.11	0.990	0.008
`realv7_001`	held	0.325	-3.77 (0.98)	632	0.259	0.47	0.988	0.021
`realv7_002`	held	0.325	-2.60 (0.93)	499	0.219	0.27	1.006	0.008
`realv6_000`	held	0.325	-4.14 (0.61)	233	0.463	0.70	1.041	0.015

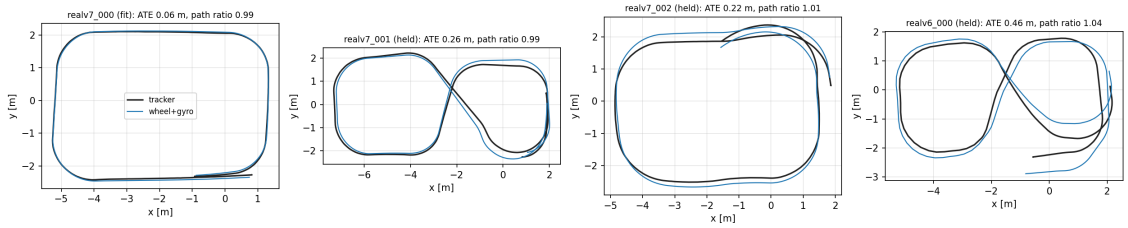


Figure 4.2 Wheel+gyro dead reckoning (blue) against the tracker (black) after rigid planar alignment, one panel per recording; the first is the recording the speed scale was fitted on.

runs read 5–15 % long.

Three things bound that reading. The scale is a fitted constant, not a measured one, so the baseline has had one parameter tuned against the truth it is scored on, on a different recording; a second calibration route, the wheel circumference and the gear ratio, would remove that. The routes are 22–37 m with the robot at $0.2\,\mathrm{m\,s^{-1}}$: a gyro drifting at the measured rest bias of 0.003–$0.006\,\mathrm{rad\,s^{-1}}$ accumulates 0.3–1.3 rad over a run if the bias is not removed and under 0.1 rad after the rest-window removal, and on a longer route that term grows while the visual error need not. And the lever arm is zero for both sides. Against the project's targets no run approaches 0.05 m except the calibration itself; every run is inside the 1.5 m RPE bound.

Summary. With one scale fitted on one recording, wheel+gyro dead reckoning localises the real robot to 0.22–0.26 m on the two held `realv7` recordings and 0.46 m on `realv6`, at least as well as the best visual run on each. On routes of this length vision has not earned its place over the proprioceptive sensors; the simulation slot said the same of the measured-noise filter.

Next action. Replace the fitted scale with a measured one (wheel circumference and gear ratio, or a tape-measured straight run); record the steering angle as a measurement so that the bicycle model and the EKF's bias state can be formed with the 0.60 m wheelbase; measure the marker-to-IMU lever arm; and record a route long enough for gyro drift to show.

4.2 Experiment 2: Offline YOLO Detector Performance

4.2.1 Simulation

Objective. Measure how well the project detector localises and names warehouse objects, independently of either SLAM integration, so that the dynamic filters of Experiment 3 have a known input quality; the moving-object recall is the headline, because it is the ceiling of any filtering benefit.

Process. `script/yolo_eval.py` projected the collision mesh of every model it has a class for into `cam0` from the simulator's pose stream to form truth boxes (the pallet jacks included, moving or parked), ran the checkpoint at confidence 0.35 with NMS at IoU 0.5 on the GPU, matched predictions greedily by class at IoU 0.5, and wrote per-frame and per-detection records; twelve overlays per recording were inspected before the aggregates were accepted. Truth boxes under 12 pixels or more than 70 % covered by nearer geometry are exempt (scored as neither hit nor miss), and predictions on unlabelled box-like scenery (loaded shelves, desks) are ignored. The three-class checkpoint of the retired evaluation is not used: it has no class for the moving objects. The earlier evaluation on the scripted worlds was retired on 6 November with its recordings and is not cited.

Datasets. The four `dataset_allsensor_realistic` recordings: `000` and `001` static (U and I layout), `002` and `003` with four pallet jacks on logistics jobs and four walking people. Ten pallet jacks stand in each world, so the forklift class has truth in every recording and *moving* truth only in `002` and `003` (3 333 and 4 562 moving truth boxes). Sources: `output/yolo_eval/dataset_allsensor_realistic_*_4class/` (`yolo_eval.csv`, `yolo_detections.csv`, `yolo_confusion.csv`, `overlay/`), `output/yolo_eval/summary_metrics.csv`, `output/compare/simulation/exp2-insights/`.

Reporting rules. Precision is over all retained predictions that were not ignored; recall is quoted on two bases, over the truth boxes the exemptions keep (the detector's task) and over every projected truth box (what is physically in view); moving-object recall counts a truth box as moving when its model moved more than 0.05 m between frames. Class metrics are conditional on a correct localisation.

Results. Table 4.6. The detector names what it finds and finds about half of what the exemptions leave it: pooled precision 0.747, recall 0.502 on the scored basis and 0.288 on the all-box basis, class F1 0.994 conditional on a match. Moving pallet jacks are found in 0.283 and 0.291 of their frames on `002` and `003`. The forklift class is the weakest on the all-box basis (recall 0.12 to 0.18 against 0.18 to 0.45 for the others, table 4.11), and the box class carries 34–70 % of the false positives.

Table 4.6 Detector scores per recording and pooled, four-class checkpoint, confidence 0.35, matching IoU 0.5. "Truth" counts every projected box and "exempt" those under 12 pixels or more than 70 % covered; R_{scored} is recall over the rest, R_{all} over every box, R_{mov} over the moving truth boxes.

Recording	Frames	Truth	Exempt	Pred.	TP	FP	P	R_{scored}	R_{all} / R_{mov}
`realistic_000`	3 805	38 401	16 157	16 673	10 940	2 693	0.802	0.492	0.285 / —
`realistic_001`	2 937	33 260	14 286	14 359	8 834	3 907	0.693	0.466	0.266 / —
`realistic_002`	2 409	28 049	10 907	13 002	8 821	2 332	0.791	0.515	0.314 / 0.283
`realistic_003`	3 547	40 498	18 235	19 148	11 852	4 757	0.714	0.532	0.293 / 0.291
Pooled	12 698	140 208	59 585	63 182	40 447	13 689	0.747	0.502	0.288 / 0.288

Analysis. The misses are a size effect and the false positives are near misses. Frame-level recall rises monotonically with the truth-box side, from 0.02 at 24–32 pixels to 0.64 above 192 pixels (table 4.7, figure 4.3); at object level every object is found in most of the frames where its box is large, and the moving pallet jacks follow the same curve as the parked ones, so the moving-object recall of 0.29 is what the pallet jacks' typical distance from the camera gives, not a motion effect. Of the unmatched predictions, 74–79 % overlap the right object below the IoU threshold and only 8–9 % land on nothing (table 4.8, figure 4.4): a looser matching threshold would move most false positives into the hits, and a filter that masks by box does not care which. The misses come as flicker rather than blindness: 39–47 % of the gaps between consecutive detections of one object are a single frame, and gaps longer than 2 s hold 20–32 % of the misses (table 4.9). Raising the confidence threshold buys precision at a steep price in recall: at 0.75 precision is 0.84 to 0.91 and moving-object recall 0.16 to 0.20 (table 4.10).

Table 4.7 Recall against truth-box size, pooled over the four recordings. Frame level: the frame's truth boxes are binned by the frame's mean truth-box side (square root of the mean truth area; the per-box truth is not logged), recall is TP/(TP+FN) of those frames. Object level: each object's median matched-box side; a miss is a frame inside a gap shorter than 2 s between two matches of that object, so objects that leave the view are not charged. Moving: object moving in at least half of its matched frames. Source: `recall_by_size.csv`.

Side [px]	frame level			object level			
	boxes	recall	R_{mov} / R_{stat}	objects mov/stat	frames	R_{mov}	R_{stat}
0–24	0	—	— / —	0/1	21	—	0.24
24–32	745	0.02	— / 0.02	1/1	932	0.96	0.93
32–48	30027	0.10	0.16 / 0.10	0/18	7120	—	0.79
48–64	29454	0.18	0.11 / 0.18	3/17	7453	0.89	0.72
64–96	37685	0.30	0.35 / 0.30	0/42	14312	—	0.80
96–128	21342	0.43	0.56 / 0.42	2/39	9856	0.53	0.85
128–192	17840	0.54	0.70 / 0.53	3/45	9524	0.87	0.85
192–∞	3115	0.64	0.60 / 0.64	0/14	2794	—	0.83

Table 4.8 What the unmatched predictions are, per recording. An unmatched prediction is a *duplicate* when it overlaps a matched prediction of the same class by IoU ≥ 0.5; a *near miss* when its best IoU with an unmatched truth box of its class is in $[0.2, 0.5)$; *slight overlap* when that IoU is in $(0, 0.2)$; *no overlap* when it touches no truth box. The scorer excludes the predictions that land on unlabelled scenery (*ignored*); they are counted here from the per-frame totals because the per-box file does not flag them, so the four categories sum to unmatched, not to scored false positives. "At border": the box touches an image edge. Source: `output/compare/simulation/exp2-insights/fp_anatomy.csv`.

Recording	unmatched	scored FP	ignored (scenery)	duplicate	near miss [0.2,0.5)	slight (0,0.2)	none 0	at border
`dataset_allsensor_realistic_000_4class`	5210	2693	3040	2 (0 %)	3994 (77 %)	745 (14 %)	469 (9 %)	12 %
`dataset_allsensor_realistic_001_4class`	5229	3907	1618	0 (0 %)	4330 (83 %)	626 (12 %)	273 (5 %)	13 %
`dataset_allsensor_realistic_002_4class`	3860	2332	1849	1 (0 %)	2860 (74 %)	684 (18 %)	315 (8 %)	13 %
`dataset_allsensor_realistic_003_4class`	6849	4757	2539	2 (0 %)	5101 (74 %)	1213 (18 %)	533 (8 %)	12 %

Limitations, each stated once. *Projected truth:* boxes are the projection of collision meshes, tighter than a human would draw and exempted by rules a human would not apply;

Table 4.9 Structure of the misses, per recording, from the gaps between consecutive matches of the same object (gaps under 2 s only). "Single-frame" is the share of gaps one frame long; "in long gaps" the share of the gap frames that lie in gaps of 1 s or more. "Short-gap misses / FN" compares the misses this reading finds with the scorer's total false negatives: the remainder are misses at the ends of tracks or in gaps longer than 2 s, which this reading cannot separate from objects leaving the view. "Objects under 0.5": objects with at least 30 frames whose in-track recall is below one half. Source: `miss_structure.csv`.

Recording	objects	gaps	median [frames]	p90 [frames]	single-frame	in long gaps	short-gap misses / FN
`dataset_allsensor_realistic_000_4class`	45 (4 under 0.5)	513	2	9	47 %	20 %	2038 / 27461 (7 %)
`dataset_allsensor_realistic_001_4class`	51 (12 under 0.5)	513	2	17	40 %	24 %	2952 / 24426 (12 %)
`dataset_allsensor_realistic_002_4class`	45 (7 under 0.5)	288	2	8	44 %	30 %	1330 / 19228 (7 %)
`dataset_allsensor_realistic_003_4class`	45 (4 under 0.5)	646	2	15	39 %	32 %	3658 / 28646 (13 %)

Table 4.10 Score-threshold sweep over the retained predictions. Precision, recall and moving-object recall if every prediction below the threshold were discarded; the logged operating point is 0.35, and nothing below it was retained, so the sweep can only go up. Matching is not re-run, so a discarded higher-scored box cannot free its truth for a lower-scored one; the values are therefore lower bounds on recall by a small margin. Source: `threshold_sweep.csv`.

Threshold	`realistic_000`			`realistic_001`			`realistic_002`			`realistic_003`		
	P	R	R_{mov}	P	R	R_{mov}	P	R	R_{mov}	P	R	R_{mov}
0.35	0.80	0.28	0.00	0.69	0.27	0.00	0.79	0.31	0.28	0.71	0.29	0.29
0.45	0.82	0.28	0.00	0.72	0.26	0.00	0.82	0.31	0.27	0.74	0.28	0.28
0.55	0.85	0.27	0.00	0.74	0.25	0.00	0.85	0.30	0.26	0.76	0.27	0.26
0.65	0.87	0.25	0.00	0.77	0.23	0.00	0.88	0.28	0.23	0.79	0.24	0.23
0.75	0.89	0.21	0.00	0.81	0.19	0.00	0.91	0.23	0.20	0.84	0.20	0.16
0.85	0.94	0.12	0.00	0.86	0.08	0.00	0.94	0.11	0.09	0.93	0.09	0.06

Table 4.11 Misses and false positives by class, per recording, from the scorer's class-agnostic confusion counts. Recall is the share of truth boxes of the class associated with any prediction; FP share is the class's share of the recording's background-row predictions. The pallet jack is not a class of the loaded 3-class weights and is an ignore region, so it does not appear. Source: `output/yolo_eval/dataset_allsensor_realistic_*_4class/yolo_confusion.csv`.

Class	`realistic_000`			`realistic_001`			`realistic_002`			`realistic_003`		
	truth	recall	FP share	truth	recall	FP share	truth	recall	FP share	truth	recall	FP share
bin	4219	0.30	0.09	1072	0.18	0.00	3038	0.40	0.05	1624	0.00	0.04
box	15541	0.22	0.34	19658	0.32	0.70	11161	0.45	0.46	20068	0.35	0.51
bucket	11428	0.44	0.29	6529	0.21	0.07	6515	0.26	0.19	10816	0.35	0.16
forklift	7213	0.18	0.27	6001	0.17	0.22	7335	0.12	0.30	7990	0.15	0.29

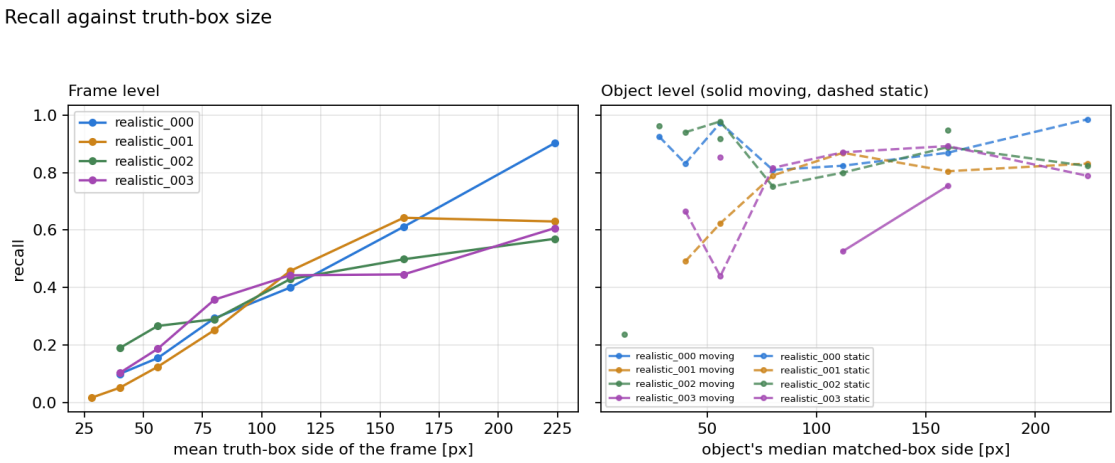


Figure 4.3 Recall against truth-box size: per frame (left, the frame's mean truth-box side) and per object (right, the object's median matched-box side; solid moving, dashed parked). The curves of the four recordings coincide, and the moving pallet jacks lie on the parked objects' curve.



Figure 4.4 What the unmatched predictions are, per recording: near misses (right object, IoU 0.2–0.5), slight overlaps, boxes on nothing, and predictions ignored on unlabelled scenery.

the scored basis is the easier one and both are given. *People:* the walking people are actors without pose or mesh and outside the checkpoint's classes; no truth exists for them, their detections (if any) are charged as false positives, and a filter built on this detector cannot mask them at all. *One operating point:* a single confidence threshold was retained, so no average precision. *Checkpoint provenance:* the four-class checkpoint was trained on 8 October 2026 and its training set is not registered in this write-up.

Summary. The detector is reliable at naming and unreliable at finding: class F1 0.994 given a match, recall 0.50 over the boxes it is asked to find and 0.29 over everything in view, moving-object recall 0.29. The miss is a size effect and the false positives are mostly boxes a little off the right object. For the filters of Experiment 3 the input is a mask that covers under a third of the moving pallet jacks' frames and none of the people.

Future plan. Add a person class and give the actors projectable geometry, or annotate a frame subset by hand, so that the moving agents the filter exists for are all scorable; raise the input resolution or the training set's share of small boxes, since the miss is a size effect; and report a precision–recall curve from a run with the threshold at zero.

4.2.2 Real robot

Reason. Projected ground truth needs object poses and meshes, which the real site does not provide. The four real recordings in use (`realv6_000`, `realv7_000–002`) hold the imagery, so the detector can be run on them, but nothing can score it until frames are annotated by hand.

Next action. Annotate a stratified frame subset of `dataset_realv6_000` and `dataset_realv7_000` to `_002` across the illumination levels of the test site, run the detector at confidence 0.35 and matching IoU 0.5, and report precision, recall, F1 and moving-object recall; the same annotated subset is the only route to a scored simulation slot as well, since the realistic worlds give the detector nothing projectable to find.

4.3 Experiment 3: Visual-Inertial Odometry With Dynamic-Environment Filtering

4.3.1 Simulation

Objective. Determine whether masking dynamic-object features improves localisation, at what compute cost, and whether tracking survives it. In the present state of the slot the question answered is the precondition: how each unfiltered pipeline behaves on the realistic warehouse, with and without traffic, and which of its own switches (front end, loop closer) decides its error.

Process. Every run replayed at rate 1.0 from the bag with the node live on the laptop, the deployment condition; no detector ran. VINS-Fusion ran once per recording (run `cpu`). ORB-SLAM3 ran four times per recording: the CPU or the GPU feature extractor (fork patches 10/10b, runs `*_cpu` and `*_gpu`), each with the loop closer active (`loopclosing_*`) or disabled (`noloopclosing_*`), all on 6 November 2026 with the same node binary and configuration (`config/wil_sim/stereo_imu.yaml`). One run per cell, so no within-condition spread exists.

Datasets. `realistic_000` (U layout) and `_001` (I layout) are the static twins; `_002` (U) and `_003` (I) carry ten and twelve moving props, pallet jacks on pick–carry–drop jobs and walking people. Routes are 103 to 171 m over 80 to 126 s (table 4.2); no two recordings share a route.

Reporting rules. Every run is reported on its own. Aligned ATE is RMSE after a rigid $SE(3)$ fit with scale fixed, in three dimensions; RPE at 1 s is translational over pairs that span no jump (a step above $10\,\mathrm{m\,s^{-1}}$ between consecutive poses). The loop-closer state is read from `loop_closing.csv` (sub-millisecond rows mean a disabled closer), not from the run name, and every closure the closer accepted is listed with its time and correction. Ground truth is the bag's `/ground_truth/odometry`, moved from the model origin to the IMU point both estimators report (lever arm 0.338 m forward, 0.192 m up) before alignment; the figures draw that moved path. "Kept" is the fraction of the bag's stereo pairs the node processed, so frames lost before the node's own counter count as lost. The onset analysis uses the start-anchored planar error against a threshold of three times the estimator's median error over its two static runs (VINS-Fusion 1.07 m, ORB-SLAM3 1.31 m). A filtered run, when one exists, is paired with the baseline of the same estimator, front end and loop-closer state, and the pair is admissible only under the three gates of section 3.6.3: an active filter (`detection_match_rate` ≥ 0.98), processed frames and duration within 5 % of the baseline, and both runs on the same node binary. Sources: `output/compare/simulation/exp-realistic-ab/` (`runs_metrics.csv`, `baseline_table.tex`), `output/compare/simulation/exp3-timelines/` (`timelines.csv`, `thresholds.csv`), run directories under `output/output_orb/simulation/` and `output/output_vins/simulation/`. The run that first occupied `realistic_002/loopclosing_gpu` had been launched on the `realistic_003` bag with the wrong output path (its manifest names that bag and world, and it received 3 535 stereo pairs where `realistic_002` holds 2 409); it is set aside under `trash/output/misfiled_20261106/` and the cell was re-run on the right bag on the evening of 6 November.

Results. VINS-Fusion holds 0.22–0.66 m aligned ATE on the four recordings with no jump and 1.2–3.6° of rotation error; its worst recording is the U layout with traffic, 0.66 m against 0.29 m on that layout's static twin, while on the I layout traffic changes nothing (0.27 against 0.22 m). ORB-SLAM3 spans 0.14–6.02 m: eleven of its sixteen runs lie between 0.14 and 0.52 m and five diverge, and which five is decided by the loop closer and the recording, not by the front end (table 4.12, figures 4.5 and 4.6). Neither front end loses tracking on any recording, with or without traffic: zero tracking-loss events and zero resets in twenty runs.

Analysis: the loop closer decides, in both directions. On the static I layout both loop-closing runs accept one closure at 73.2 s, in the heading change at 76 % of the route, with a correction of 14.7 m (CPU) and 13.3 m (GPU) and 1.9–2.8°; a global bundle adjustment follows and the trajectory is 6.02 and 4.78 m off for the rest of the run, where the two closure-free runs on the same recording hold 0.50 and 0.40 m (figure 4.8). A route with half a metre of accumulated error cannot need a fourteen-metre correction, so the match is false: the I layout's long parallel aisles present the same rack faces twice, and the matched keyframes (97 against 255, 104 against 272) are one aisle apart. The same candidate was then re-proposed four times and rejected by the inertial geometry check, after the damage. That both front ends accept it at the same frame with the same correction says the

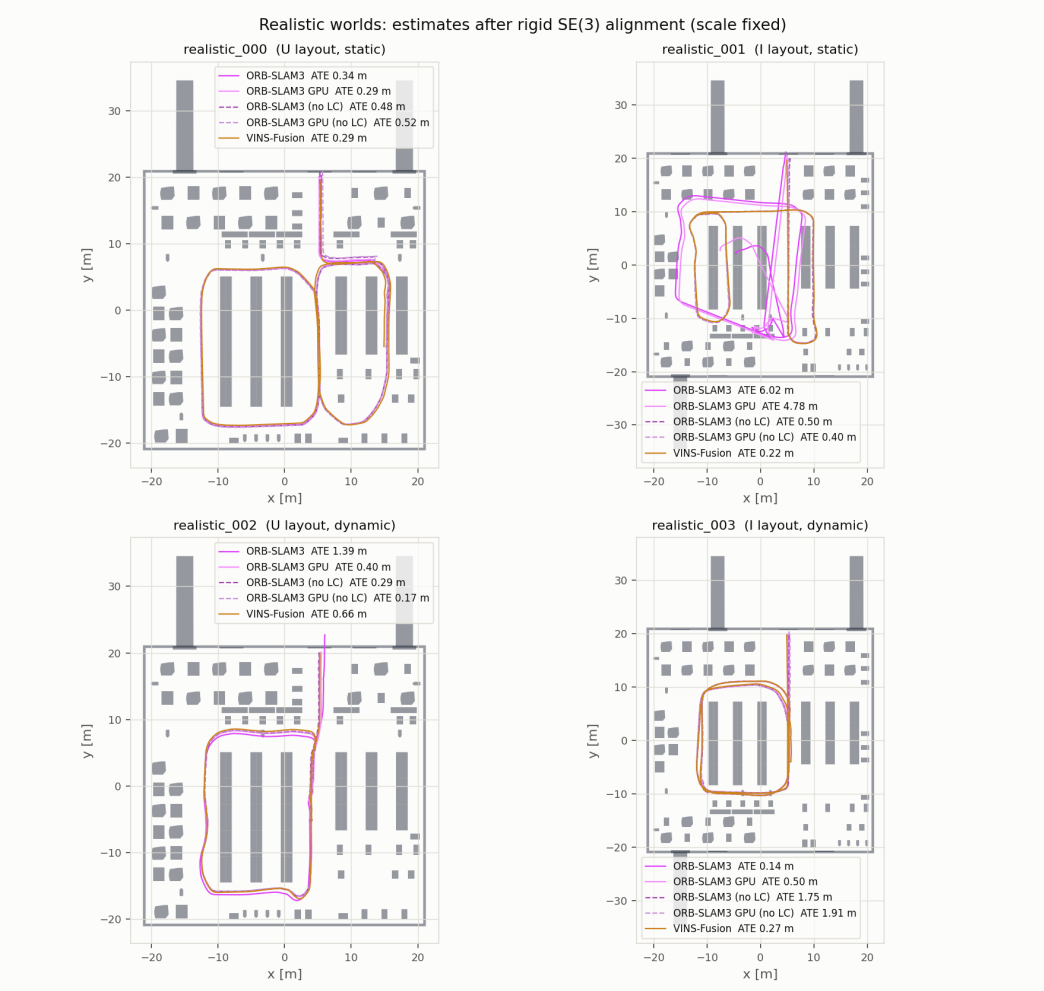


Figure 4.5 Unfiltered estimates on the four realistic-warehouse recordings after rigid $SE(3)$ alignment with scale fixed, on the warehouse footprint. Top: the static twins; bottom: the same layouts with traffic. Solid magenta is ORB-SLAM3 with its loop closer active, dashed with it disabled, lighter for the GPU front end; orange is VINS-Fusion. The two excursions on `realistic_001` are the loop-closing runs after their false closure; the 1.4 m run on `realistic_002` is the CPU loop-closing run; the closure-free runs on `realistic_003` lie on the route in plan and diverge vertically.

Table 4.12 Unfiltered arms on the four realistic-warehouse recordings, 6 November 2026, replay rate 1.0. Aligned ATE is RMSE after a rigid $SE(3)$ fit with scale fixed; RPE is translational, at $\Delta = 1\,\mathrm{s}$ over pairs that do not span a jump; "Jumps" counts steps above $10\,\mathrm{m\,s^{-1}}$; "Kept" is the fraction of the bag's stereo pairs the estimator processed (frames lost before the node's own queue count as lost). Loop-closer state is read from `loop_closing.csv`, not from the run name. Static and dynamic rows of one layout share a world geometry and differ in traffic only. **Bold** marks the lowest ATE and the lowest jump-free RPE among the five arms of a recording; with one run per arm it records which run came lowest, not that the arm is better.

Recording	Layout	Scene	Estimator	ATE [m]	Rot. [°]	$\mathrm{RPE}_{1\,\mathrm{s}}$ [m]	Jumps	Kept	Resets
`realistic_000`	U	static	ORB-SLAM3	0.338	13.74	0.083	2	0.72	0
			ORB-SLAM3 GPU	0.293	12.80	0.079	3	0.99	0
			ORB-SLAM3 (no LC)	0.483	14.05	0.090	2	0.74	0
			ORB-SLAM3 GPU (no LC)	0.523	12.84	**0.078**	3	0.99	0
			VINS-Fusion	**0.290**	1.85	0.084	0	0.77	0
`realistic_001`	I	static	ORB-SLAM3	6.021	16.92	0.094	6	0.73	0
			ORB-SLAM3 GPU	4.780	18.98	0.092	7	0.99	0
			ORB-SLAM3 (no LC)	0.496	16.05	0.093	1	0.74	0
			ORB-SLAM3 GPU (no LC)	0.400	15.13	**0.088**	2	0.97	0
			VINS-Fusion	**0.221**	1.18	0.089	0	0.78	0
`realistic_002`	U	dynamic	ORB-SLAM3	1.393	16.99	**0.078**	4	0.77	0
			ORB-SLAM3 GPU	0.400	16.04	**0.078**	4	0.98	0
			ORB-SLAM3 (no LC)	0.293	17.12	0.080	2	0.83	0
			ORB-SLAM3 GPU (no LC)	**0.170**	16.08	0.083	1	0.99	0
			VINS-Fusion	0.663	3.57	0.116	0	0.88	0
`realistic_003`	I	dynamic	ORB-SLAM3	**0.139**	13.52	0.075	2	0.94	0
			ORB-SLAM3 GPU	0.504	13.16	0.076	3	0.99	0
			ORB-SLAM3 (no LC)	1.755	14.22	0.080	2	0.88	0
			ORB-SLAM3 GPU (no LC)	1.911	13.53	0.083	3	0.99	0
			VINS-Fusion	0.268	1.51	**0.071**	0	0.87	0

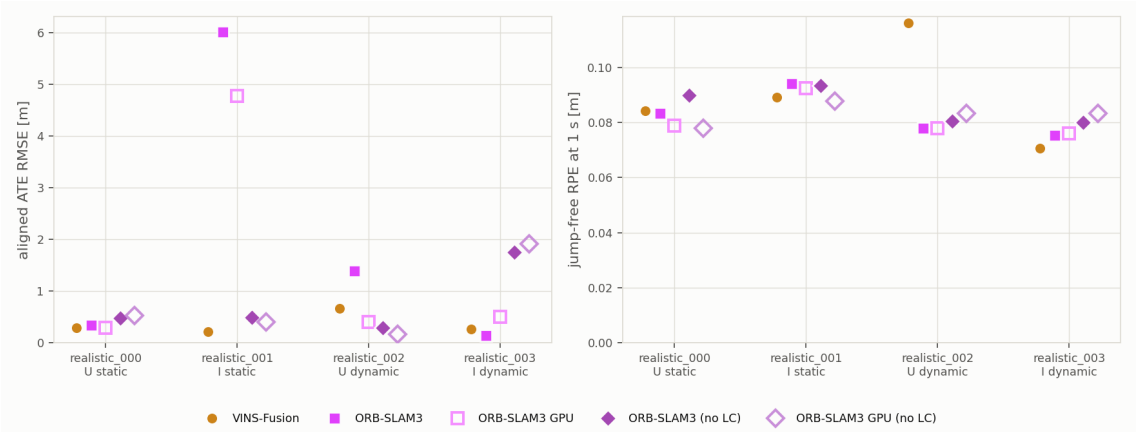


Figure 4.6 Aligned ATE (left) and jump-free translational RPE at 1 s (right) of every unfiltered run, one marker per run; filled markers are the CPU front end, open markers the GPU front end of the same configuration. The relative error is 0.07–0.12 m for every run, so the ATE differences are discrete events, not tracking quality.

place-recognition stage, not the extractor, is where the fault lies.

On the I layout with traffic the sign reverses. Both loop-closing runs accept one closure at 56.5 s (48 % of the route) with a 0.33 m correction and hold 0.14 and 0.50 m; the closure-free runs read 1.76 and 1.91 m, yet their planar error never exceeds 0.7 m (figure 4.10). The difference is vertical: the closure-free estimates climb to 4.7 and 5.1 m above the floor by the end of the run while the loop-closing runs stay within 0.6 and 1.4 m, and the three-dimensional ATE reports that climb. The GPU loop-closing run on this recording also rejected 148 further candidates by inertial geometry, every one of them after its single accepted closure. On the U layout the closer is nearly inert: two accepted closures per run on `realistic_000` with corrections of 0.07–0.12 m, one of 0.26–0.33 m per run on `realistic_002`, and the eight U-layout ORB-SLAM3 runs within 0.17–0.52 m except the CPU loop-closing run on `realistic_002`, whose 1.39 m is an offset present from 6.5 s on, before any closure (figure 4.9); with four jumps and the lowest relative error on the recording it is an initialisation event, not a tracking one, and the GPU run of the same configuration, which accepts the same closure at 65 s, does not reproduce it (0.40 m).

Analysis: the front end changes throughput, not accuracy. The GPU extractor processes 97–99 % of each bag's frames where the CPU extractor processes 72–94 %, and its tracking median is 30–37 ms against 49–55 ms (from the per-run `performance.csv`). Across the eight CPU/GPU pairs with the same closer state the ATE differs by at most 0.16 m in five; the three larger differences (-1.2, -1.0 and 0.4 m) are each one run's discrete event, in both directions, and the two divergence patterns above occur in both front ends alike. At this frame budget the extractor is not the limiting stage of accuracy on these recordings.

Analysis: traffic as a condition. VINS-Fusion is 0.37 m worse on the U layout with traffic than on its static twin and 0.05 m worse on the I layout; its relative error on `realistic_002` is the highest of any run (0.116 m) and its error trace carries two waves of 0.7 m at 25 and 45 s on the western and southern aisles, where the pallet jacks work, that no static run shows (figure 4.9). ORB-SLAM3's closure-free runs are *better* on the U traffic recording (0.29, 0.17 m) than on its static twin (0.48, 0.52 m). With one run per cell and routes that differ between twin and traffic recording, neither number is a traffic effect; what the slot establishes is that physically simulated traffic degrades the tracker that follows features longest, and breaks neither.

Analysis: the rotation column. ORB-SLAM3's 13–19° rotational RMSE is one event on every run: the first jump lies at 2.5–4.8 s (table 4.13), where inertial initialisation re-aligns the world frame to gravity, and after it the clean rotational RPE is 0.07–0.33°. The jump detector is translational, so the column includes the event; the median is the number to read.

Filtered arms. untested. The current dataset set holds no filtered run: the October 2026 filtered runs on these recordings were retired on 6 November 2026 with the rest of the earlier campaign, and the filter has not been re-run under the present pipeline (GPU front end, patch-15 initialisation switches). When it is, each filtered run pairs with the baseline of the same estimator, front end and closer state under the three gates above, and the loop-closer finding here fixes the pairing that matters: on the I layout the comparison must be made with the closer disabled, or the false closure will dominate both arms.

Table 4.13 Realistic world, unfiltered runs: onset of sustained error and its place on the route, per run. Onset is the first time the start-anchored position error stays above 3× the estimator's median error over its static runs (VINS-Fusion 1.07 m, ORB-SLAM3 1.31 m) for the rest of the run; "where" is the route segment and the share of the route completed at that time; the first jump is a step above 10 m s^{-1} between consecutive poses. "never" means the error stayed below the threshold for the whole run.

Recording	Estimator	LC	Onset [s] (where)	First jump [s]
`realistic_000`	ORB-SLAM3	active	never	4
	ORB-SLAM3	disabled	never	4
	ORB-SLAM3 GPU	active	never	4
	ORB-SLAM3 GPU	disabled	60 (straight 48 %)	4
	VINS-Fusion	—	never	—
`realistic_001`	ORB-SLAM3	active	73 (turn 78 %)	5
	ORB-SLAM3	disabled	never	5
	ORB-SLAM3 GPU	active	73 (turn 78 %)	5
	ORB-SLAM3 GPU	disabled	never	5
	VINS-Fusion	—	never	—
`realistic_002`	ORB-SLAM3	active	6 (straight 6 %)	3
	ORB-SLAM3	disabled	never	3
	ORB-SLAM3 GPU	active	never	4
	ORB-SLAM3 GPU	disabled	never	3
	VINS-Fusion	—	never	—
`realistic_003`	ORB-SLAM3	active	never	3
	ORB-SLAM3	disabled	never	3
	ORB-SLAM3 GPU	active	never	3
	ORB-SLAM3 GPU	disabled	never	3
	VINS-Fusion	—	never	—

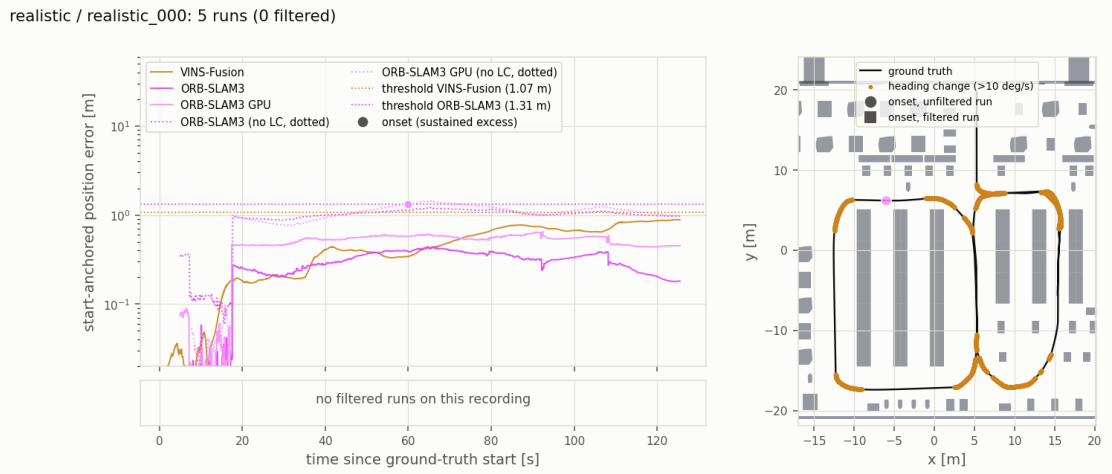


Figure 4.7 `realistic_000` (U, static), every run. Left: start-anchored planar error on a log axis, solid for loop closing active, dotted for disabled, lighter for the GPU front end, with each estimator's threshold; the dot marks the onset of sustained excess. Right: the route with heading changes in orange and every onset placed on it. The one onset is the GPU closure-free run at 60 s, a sustained 1.3 m offset on the straight, not a jump.

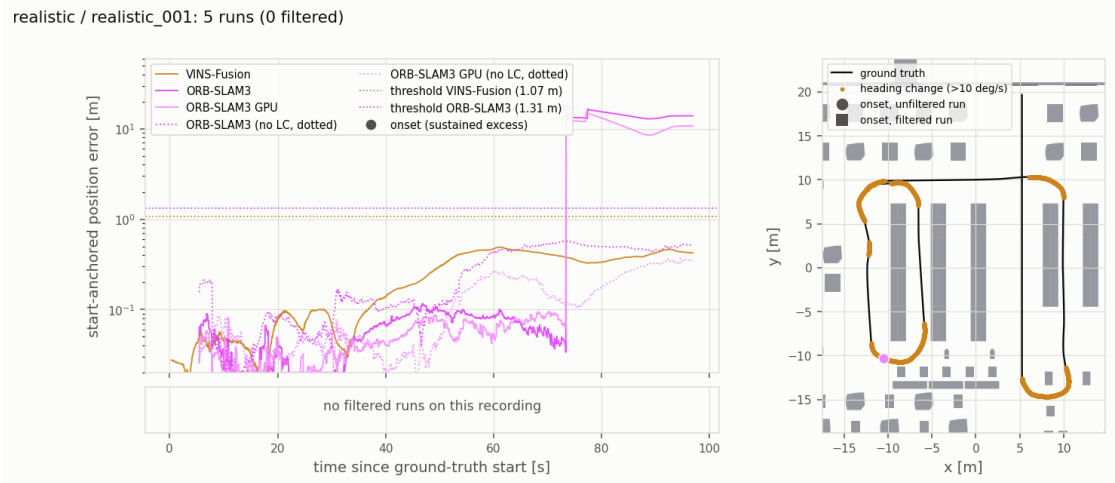


Figure 4.8 `realistic_001` (I, static), every run; panels as in figure 4.7. Both loop-closing runs step to 14 m of error at 73 s, in the heading change at 76 % of the route, at the single accepted closure; the closure-free runs and VINS-Fusion never cross.



Figure 4.9 `realistic_002` (U, traffic), every run; panels as in figure 4.7. The CPU loop-closing run carries a 1.5 m offset from 6.5 s, which the GPU loop-closing run does not share; VINS-Fusion's two 0.7 m waves at 25 and 45 s lie on the aisles where the pallet jacks work.

Latency and resources. By the design decision of 6 November 2026 the real-time and resource measurement of this experiment is taken only on the two mapping recordings, each configuration run three times: `dataset_allsensor_realistic_map` (U layout, 286 s, 8 725 stereo pairs) and `dataset_allsensor_realistic_01_map` (I layout, 315 s, 9 552 pairs). Eighteen runs exist, six configurations of three repeats each at replay rate 1.0 with no detector: ORB-SLAM3 with the CPU and with the GPU front end, loop closer disabled, and VINS-Fusion (table 4.14). The repeats are tight, front-end medians within 1–7 ms of their mean and peak memory within 30 MiB.

The two pipelines divide the cost differently, and neither is cheaper than the other outright. VINS-Fusion optimises in 44 ms where ORB-SLAM3's local mapping takes 211–406 ms, and holds its state in a quarter of the memory, but spends more than twice the processor time: about eight cores of one against three and a half. Their front ends are within 6 ms of each other, and every configuration keeps 75–96 % of the bag's frames, so the choice between them is a memory-against-cores trade, not a speed one.

Between ORB-SLAM3's two front ends the recordings disagree. On the U recording the GPU extractor behaves as the single runs showed, tracking in 37 against 42 ms and keeping 96 against 93 % of frames; on the I recording the sign reverses, 52 against 44 ms and 75 against 91 % kept, with the local-mapping median at 406 ms. VINS-Fusion reads the same on both recordings (front end 38.1 and 38.6, back end 44.3 and 44.5 ms), so the recordings themselves do not impose a difference of that size.

The per-run resources of the twenty accuracy runs above are generated beside this table (`realistic_resources.tex`) and are not reported, except the frame retention already in table 4.12.

Table 4.14 Real-time and resource measurements on the two mapping recordings, replay rate 1.0, no detector: mean ± SD over n repeats of each configuration. **Bold** marks the best mean in each column within a recording. Front-end and back-end are each run's median module latency (ORB-SLAM3: tracking and local mapping; VINS-Fusion: feature tracking and back-end optimisation); throughput is processed frames per second of sensor time; "Kept" is the processed fraction of the bag's stereo pairs; CPU is process time over wall time, in percent of one core; RSS the peak resident set. The loop-closer state is read from `loop_closing.csv`.

Recording	Estimator	LC	n	Front-end [ms]	Back-end [ms]	Throughput [Hz]	Kept [%]	CPU [%]	Peak RSS [MiB]
`realistic_map`	ORB-SLAM3	disabled	3	42.0 ± 3.3	211.3 ± 14.8	28.3 ± 0.9	93 ± 3	379 ± 11	1732 ± 15
	ORB-SLAM3 GPU	disabled	3	**36.7 ± 1.2**	284.2 ± 14.7	**29.1 ± 0.4**	**96 ± 1**	**332 ± 7**	1794 ± 19
	VINS-Fusion	—	3	38.1 ± 3.3	**44.3 ± 5.0**	27.5 ± 2.7	91 ± 9	819 ± 10	**479 ± 15**
`realistic_01_map`	ORB-SLAM3	disabled	3	43.9 ± 1.0	220.4 ± 12.6	**27.5 ± 0.3**	**91 ± 1**	381 ± 6	1816 ± 29
	ORB-SLAM3 GPU	disabled	3	51.9 ± 3.8	405.7 ± 22.6	22.9 ± 1.4	75 ± 4	**361 ± 9**	1719 ± 21
	VINS-Fusion	—	3	**38.6 ± 0.8**	**44.5 ± 1.5**	27.4 ± 0.5	90 ± 2	814 ± 5	**497 ± 11**

Limitations, each stated once. *n:* one run per cell; the only run-to-run figure is the agreement of the two front ends on where and how the closer fails. *Route and traffic confounded:* no two recordings share a route. *Three-dimensional ATE:* the metric charges vertical drift to a floor robot; the planar onset table is the complement, not a replacement. *Deployment rate on one laptop:* frame retention and every latency figure are properties of this machine at rate 1.0, and the mapping-bag repeats were not interleaved between front ends.

Summary. On the realistic warehouse neither pipeline loses tracking, with or without traffic. VINS-Fusion holds 0.22–0.66 m with no jump and is worst on the U layout with traffic; ORB-SLAM3 holds 0.14–0.52 m in eleven runs of sixteen, and its five divergences

are the loop closer's: a false closure on the I layout's repeated aisles that puts both front ends 4.8–6.0 m off, a vertical drift on the same layout with traffic that only the closer corrects, and one initialisation offset. The front end changes throughput (97–99 % of frames kept against 72–94 %) and not accuracy. The filtering hypothesis is untested in the current dataset set. On the mapping recordings VINS-Fusion optimises five to nine times faster than ORB-SLAM3's local mapping and in a quarter of the memory, for more than twice the processor time; within ORB-SLAM3 the GPU front end tracks in 37 ms and keeps 96 % of frames against the CPU front end's 42 ms and 93 % on the U layout, in three repeats each, and the I-layout repeats contradict that and were run in a thermally confounded order.

Future plan. Run both pipelines filtered on `realistic_002` and `_003` with the detector live, paired under the three gates and with the closer disabled on the I layout; add a correction bound to the loop closer and test it on `realistic_001`; repeat the mapping-recording runs interleaved between all three configurations from a cool start, so that the thermal state is shared rather than confounded with the configuration; and record a second route in each layout so that traffic and route are separable.

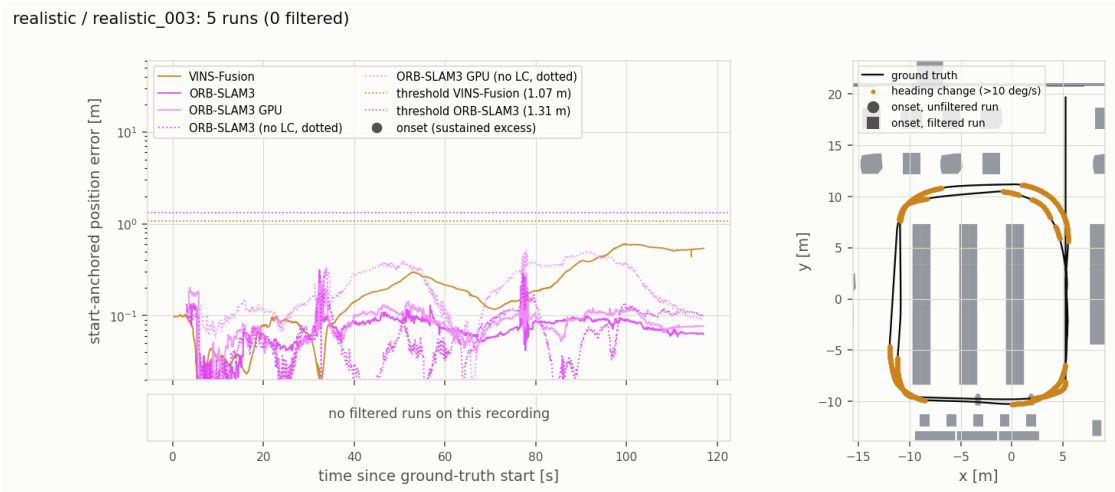


Figure 4.10 `realistic_003` (I, traffic), every run; panels as in figure 4.7. No run crosses its threshold in the plane; the 1.8 and 1.9 m ATE of the closure-free runs is vertical drift, which the single accepted closure of the loop-closing runs removes.

4.3.2 Real robot

Objective. Establish whether the two unfiltered pipelines track the real robot against an independent reference, as the precondition for a filtered-versus-unfiltered test on real imagery.

Process. The bag was replayed into each node at real time on the afternoon of 3 November 2026; no detector ran. ORB-SLAM3 used `config/wil/stereo_imu.yaml` with the repaired library (fork patches 11–14b, node and library built alike), the 3 November stereo extrinsic and `IMU.NoiseGyro` 0.1 with `IMU.NoiseAcc` 1.0, ten times the values that had been copied from the VINS file, in four configurations: GPU or CPU feature extractor, loop closing on or off. VINS-Fusion used the same file with the gravity norm of an August recording. Earlier runs on this bag were overwritten by these and are not reported. The optical tracker export was scored with the real-robot procedure of Experiment 1.

Dataset. 220 s, 6 601 stereo pairs at 30 Hz, RealSense IMU at 200 Hz on the inherited topic name, wheel-speed feedback, a figure-of-eight of about 27 m at a median 0.15 m s^{-1} on the Gensurv floor, no scripted moving objects. Tracker export: 277 samples, median spacing 0.6 s, 66 gaps above 1 s, one sample 5.2 m off the path at 184.9 s.

Reporting rules. `script/eval_real_gt.py`: rigid SE(2) alignment with scale fixed, lever arm set to zero and unmeasured, every value provisional. The export fails the tool's gap and step rules, so the tool labels each run *failed (ground truth invalid)*; the values below are diagnostics. All are scored with the one off-path sample removed, an edit recorded here and applied nowhere else; table 4.16 shows what that sample does to one run. The bag was not on the machine, so the time offset is pinned to the value the wheel-speed cross-correlation found on 2 November and the manifests record the substitution. Sources: `output/compare/real/realv6_1103pm_*/`, `output/compare/real/realv6-insights/`, run directories under `output/output_orb/real/dataset_realv6_000/` and `output/output_vins/real/dataset_realv6_000/`.

Results. ORB-SLAM3 follows the whole 218 s route in every configuration: 0.52 and 0.46 m with the GPU extractor (loop closing on and off), 0.62 and 1.03 m with the CPU extractor, which drops two thirds of the frames at real time (table 4.15, figure 4.11). VINS-Fusion tracked 18 s before 217 IMU-gap resets took over (table 4.18) and has no entry in the accuracy table.

Analysis. The front end is what separates the runs. The GPU extractor drops a third to a half of the frames and holds 0.5 m; the CPU extractor drops two thirds and loses a factor of two, one of its runs with a 2 m vertical excursion the tool flags as implausible and the other with three resets and a half-second of lost tracking. Loop closing changes nothing that the front end does not change more. The remaining resets are an initialisation effect, two or three `local_mapping_bad_imu` map resets inside the first 13 s and none after (zero in the GPU run without loop closing); after them the error is a slow wave between 0.05 and 1 m, not an accumulating drift (figure 4.12), and it does not rise while turning in any run (table 4.17). The estimates are a uniform 12–15 % smaller than the tracker's path, which the unmeasured lever arm, a stereo-baseline error or a tracker scale error could each explain. One tracker sample decides the headline: scored against the export as delivered, the GPU run reads 1.17 m with a path ratio of 0.65, because that sample adds 10 m of false path and dominates the time alignment (table 4.16).

Table 4.15 ORB-SLAM3 on `dataset_realv6_000` against the cleaned tracker truth, 3 November 2026 afternoon series: repaired library, replay rate 1.0, 3 November stereo extrinsic, `IMU.NoiseGyro` 0.1 and `IMU.NoiseAcc` 1.0, GPU or CPU feature extractor, loop closing (LC) on or off. Aligned planar ATE after a rigid SE(2) fit with scale fixed; RPE at 1 s translational; path ratio = estimate / truth; resets = `local_mapping_bad_imu` map resets; dropped = camera frames the node's queue discarded of about 6 480 received. Lever arm tracker-marker to IMU unmeasured (zero applied); time offset pinned to −4.14 s; all values provisional under the scorer's validity rules.
textbfBold marks the lowest ATE of the five runs; with one run per configuration that records which run came lowest, not that the configuration is better.

Run	ATE [m]	median [m]	max [m]	RPE_{1s} [m]	path ratio	resets	dropped
ORB-SLAM3, GPU, LC on	0.52	0.44	1.11	0.057	0.85	2	2420
ORB-SLAM3, CPU, LC on	0.62	0.48	1.38	0.110	0.88	2	4363
ORB-SLAM3, GPU, LC off	**0.46**	0.36	0.99	0.056	0.87	0	3201
ORB-SLAM3, CPU, LC off	1.03	0.89	1.83	0.223	0.96	3	4204

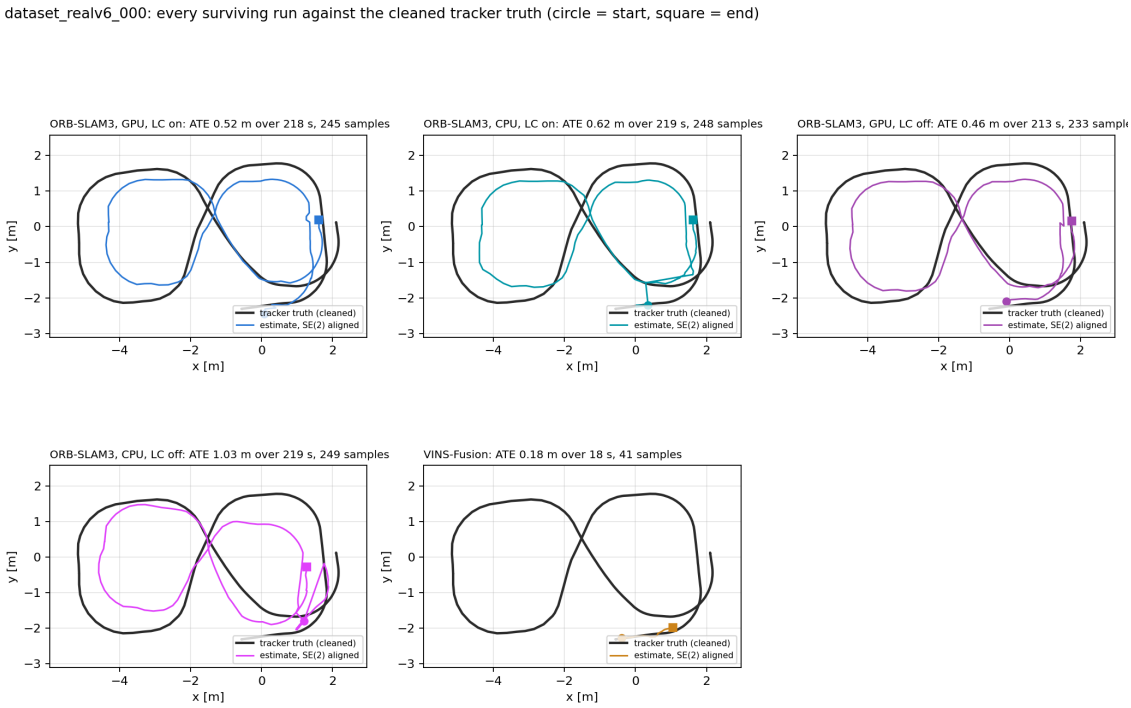


Figure 4.11 Every run against the cleaned tracker truth after rigid SE(2) alignment, one panel per run: ORB-SLAM3 with GPU or CPU extractor and loop closing on or off, and VINS-Fusion's 18 s.

Table 4.16 The same ORB-SLAM3 run (GPU, LC on) scored against the tracker export as delivered and with one sample removed (the 5.2 m excursion at 184.9 s). The time offsets are the values the wheel-speed cross-correlation found on each export on 2 November (peak correlation 0.10 raw, 0.61 cleaned), applied here as pinned values. The cleaned truth still fails the gap rule (66 gaps above 1 s, tracker at 1.7 Hz).

Truth	GT path [m]	offset [s]	ATE [m]	median [m]	path ratio
as delivered	36.8	4.38	1.174	0.851	0.65
one sample removed	27.1	-4.14	0.519	0.437	0.85

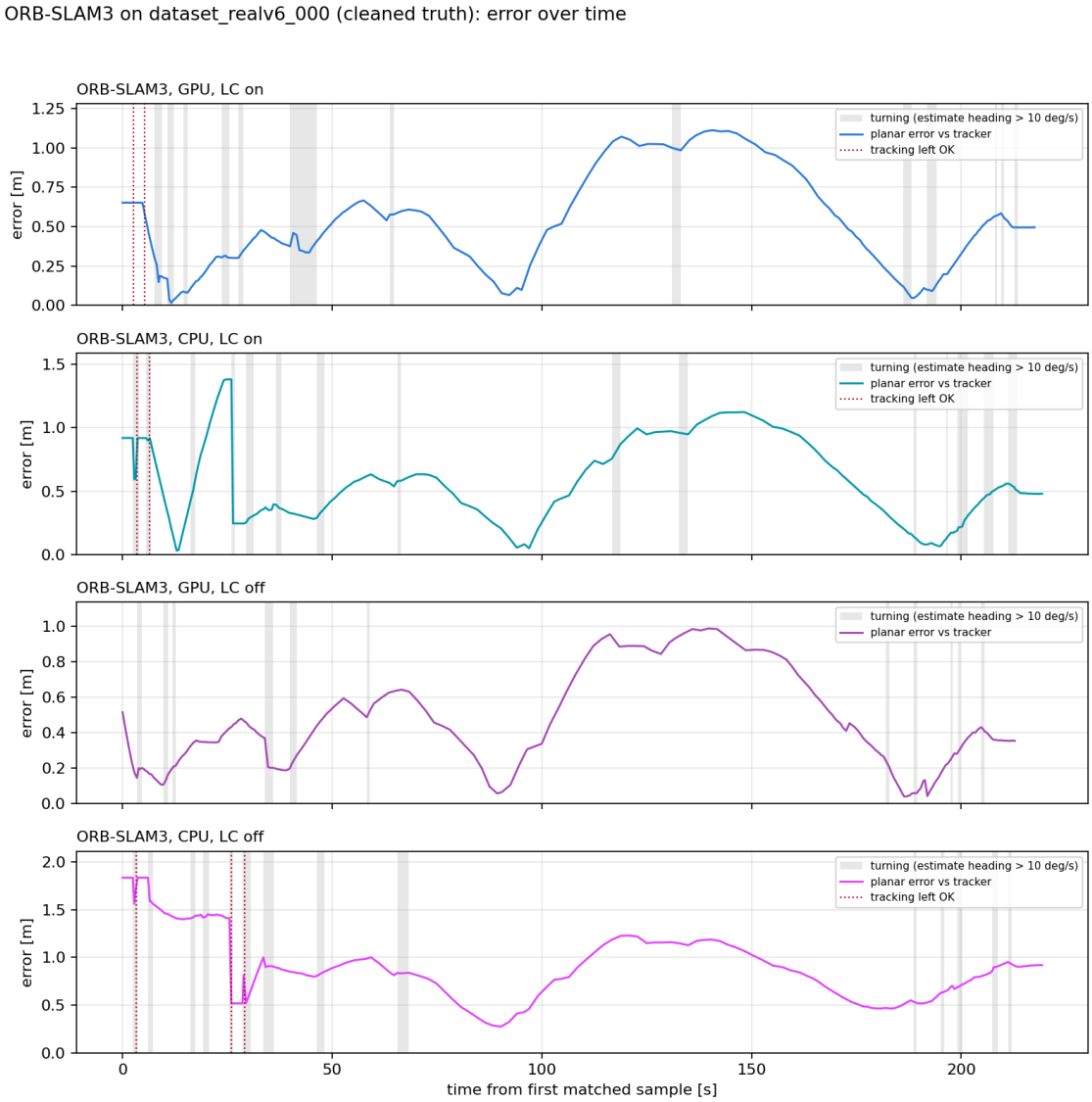


Figure 4.12 ORB-SLAM3 planar error over the run, one panel per configuration; grey bands are turns, dotted red the initialisation resets.

Table 4.17 ORB-SLAM3 on `dataset_realv6_000`, cleaned truth: growth rate of the planar error while the aligned estimate's heading changes faster than $10\,^{\circ}\,\mathrm{s}^{-1}$ and on straights, with the time in each state. Positive = error growing. One run per configuration, so no test is possible.

Run	turning [mm/s]	straight [mm/s]	time turning [s]	time straight [s]
ORB-SLAM3, GPU, LC on	-11.2	-0.3	23	171
ORB-SLAM3, CPU, LC on	-29.8	-1.4	23	162
ORB-SLAM3, GPU, LC off	4.9	-2.1	11	185
ORB-SLAM3, CPU, LC off	-49.9	-0.9	20	182

The setting was reached by elimination. With the IMU noise densities copied from the VINS file the estimator was unstable on this recording, running away in its velocity state at a different place in each run; six out-of-bounds reads in the library were fixed without removing it, every other candidate was varied without removing it, and raising the IMU noise densities tenfold removed it in every run since. Those earlier runs were overwritten and carry no number here; the search is in the ORB-SLAM3 engineering record.

VINS-Fusion's failure is a delivery defect. The node received 3 187 of 6 601 frames but used 6 498 of the 43 494 IMU messages it received, reset 217 times on `imu_gap`, and stopped after 19 s; behind that defect the configuration still fixes a gravity norm of 9.2642 against the 9.878 m s^{-2} this IMU reads at rest.

Table 4.18 VINS-Fusion on `dataset_realv6_000`, 3 November 2026 (`config/wil/stereo_imu.yaml`, IMU noise ×10, gravity norm still 9.2642 against the 9.878 m s^{-2} this IMU reads at rest). The scored span is the 18 s it tracked before its last reset and is not a result.

Quantity	Value
camera frames received (bag: 6 601)	3187
IMU messages received / samples used	43494 / 6498
`imu_gap` resets	217
poses written / span of poses	286 / 19 s
tracker samples matched	41 of 43 in span
ATE over the tracked span	0.18 m (not a result)

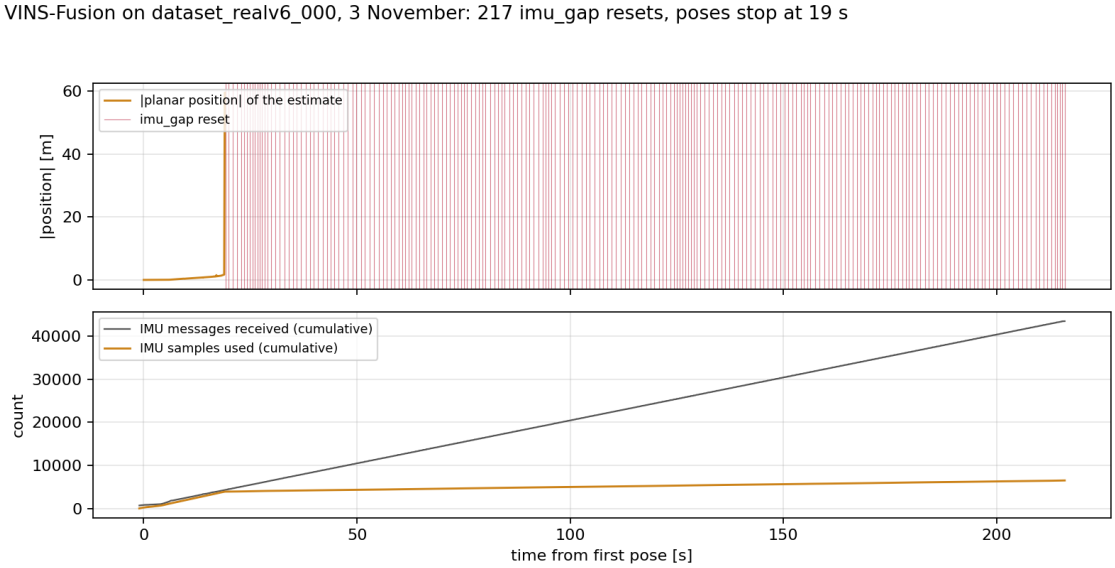


Figure 4.13 VINS-Fusion: distance from origin with every `imu_gap` reset marked, and IMU messages received against samples used.

Second recording set, realv7. Three further recordings of 3 November 2026 (`dataset_realv7_000`, `_001`, `_002`: 21.6, 31.7 and 23.4 m over 97, 134 and 104 s) have tracker exports that pass every validity rule (5–6 Hz, no gap, no step). Each carries the same four ORB-SLAM3 configurations and one VINS-Fusion run. As written in the live log (table 4.19, figure 4.14), a plane fitted to the ORB-SLAM3 estimate lies within 10° of horizontal in three runs of twelve and at 88–90° in five, every `000` run among them, with

the plane normal exactly a body axis in every run; scored as written, those nine runs are not scorable.

The per-frame diagnostics give the mechanism. The IMU initialised 3–4 s after the first pose. The upstream rule that resets a map with less than 2 cm between consecutive keyframes in the first ten seconds after the initialisation then discarded it, the robot being still at rest. The new map starts in the camera frame of the reset instant, and in it the re-initialisation was starved: upstream attempts it only with an empty keyframe queue, which a keyframe every 0.25 s and 250 ms of map-point creation per keyframe never allowed. The IMU re-initialised after 37–92 s in seven runs, never in two, and early (6.7 s) in the one run that was level. Until then the estimate is stereo-only in a map whose vertical is the camera's y axis, and the late re-initialisation rotates the map under the already-written poses (the 1.9–6.8 m steps).

The scorer repairs both without the truth: the pre-initialisation poses are rotated by the rotation measured at the initialisation frame, and each map's fitted plane is levelled. With the bags on the dataset drive the time offsets come from the wheel speed on all three recordings (table 4.20, figure 4.15). `000` scores 0.25–0.27 m in all four configurations over 89 s (RPE at 1 s 0.05 m), with the IMU initialised for at most a quarter of the frames. `001` scores 0.44–0.82 m over 126 s, one run failing the scorer's one-metre vertical-range rule by two centimetres. `002` scores 0.47 m for the CPU front end with loop closing; two runs lose tracking inside the route and one drifts (vertical range 4.3 m, failed). The init-time resets leave every other run *partial* under the scorer's rules.

The fork's patch 15 adds two switches that attempt the initialisation with a queued backlog and skip the low-motion reset for stereo; they are on in the real-rig configuration and not yet exercised on a recording. VINS-Fusion diverges on all three with the IMU stream intact (zero resets) and 40–45 % of the frames received (table 4.21); its gravity norm has since been set to the measured 9.886 m s^{-2}.

Table 4.19 ORB-SLAM3 on the three `dataset_realv7` recordings, 3 November 2026 afternoon, same setting as the realv6 runs: the plane of each estimate as written in the live log, before any repair. "Tilt" is the angle between the plane fitted to the estimate's 3-D positions and the horizontal; "off-plane" the rms distance of the positions from that plane. A floor robot's estimate should be horizontal: a tilt near 90° means the poses were written while the map was not gravity-aligned (IMU not initialised; the vertical is then the camera's y axis). The scores are in table 4.20, after the repair.

Recording	Run	tilt [°]	off-plane [m]	z range [m]	resets
realv7_000	GPU, LC on	89.1	0.35	6.87	3 s
realv7_000	GPU, LC off	89.5	0.02	6.98	3 s
realv7_000	CPU, LC on	88.8	0.41	6.88	5 s
realv7_000	CPU, LC off	87.9	0.58	6.79	4 s
realv7_001	GPU, LC on	88.8	0.05	9.64	4 s
realv7_001	GPU, LC off	6.1	1.12	5.77	3 s, 5 s
realv7_001	CPU, LC on	1.5	0.07	0.38	4 s
realv7_001	CPU, LC off	13.0	1.05	5.23	5 s
realv7_002	GPU, LC on	37.7	0.87	4.08	3 s, 49 s, 59 s
realv7_002	GPU, LC off	59.7	0.83	4.80	49 s
realv7_002	CPU, LC on	5.2	0.73	2.94	none
realv7_002	CPU, LC off	21.7	0.84	7.03	none

Summary. The real robot has an unfiltered ORB-SLAM3 reference at a provisional 0.46–0.52 m on `realv6` (GPU front end) and 0.25–0.27 m on `realv7_000` with valid truth

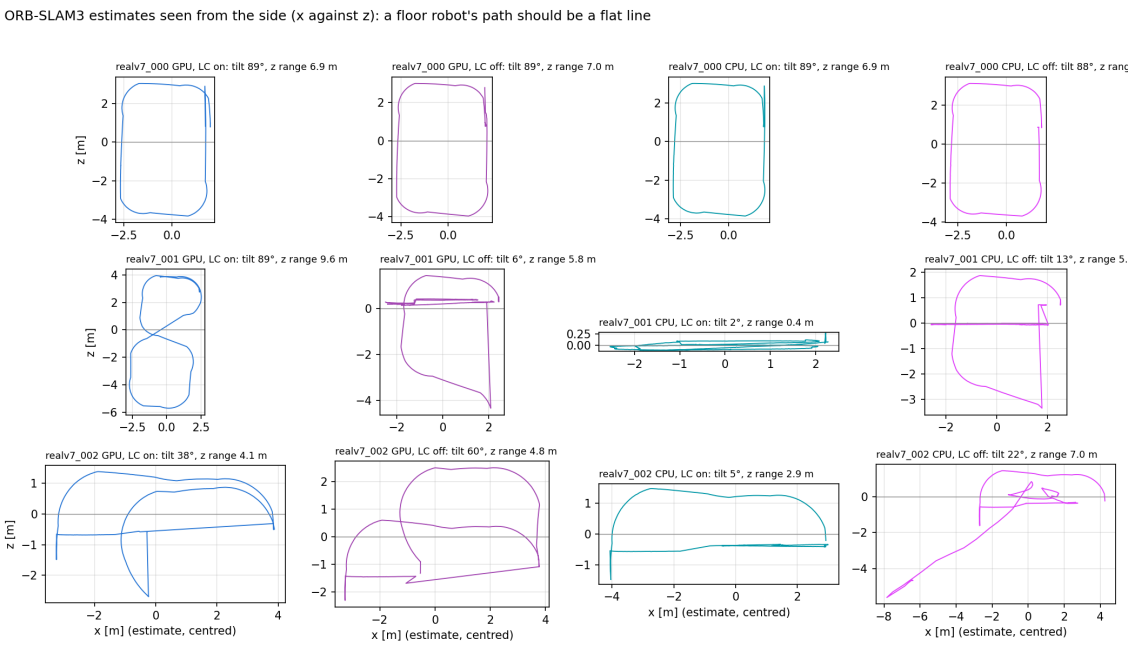


Figure 4.14 The twelve `realv7` ORB-SLAM3 estimates as written in the live log, seen from the side (x against z, centred); a floor robot's path should be a flat line. The repaired estimates are in figure 4.15.

Table 4.20 ORB-SLAM3 on the three `dataset_realv7` recordings (3 November 2026, same setting as realv6), scored on 4 November with the bags on the dataset drive (wheel-speed time offsets) and the two frame repairs of `script/eval_real_gt.py`: the poses written before each IMU initialisation rotated into the frame used after it (`-join-imu-init`), and each map's fitted plane levelled (`-level-plane`). "IMU" is the share of tracked frames with the IMU initialised; "resets" the active-map resets after the first pose, in seconds (a reset restarts the position at the origin and is not stitched; those inside the route are tracking losses); "tilt" the plane tilt of the raw log, zero after the repair in every run. ATE is planar after SE(2) alignment, RPE over 1 s. Status from the scorer's rules ("partial" = a reset or loss the run recovered from). Lever arm unmeasured (zero); VINS-Fusion diverged on all three (table 4.21) and is omitted. **Bold** marks the lowest ATE among the four configurations of a recording; with one run per configuration that records which run came lowest, not that the configuration is better.

Recording	Run	IMU [%]	resets [s]	tilt [°]	ATE [m]	median [m]	RPE_{1s} [m]	path ratio	status
`realv7_000`	GPU, LC on	6	3	89	**0.25**	0.24	0.05	1.05	partial
`realv7_000`	GPU, LC off	0	3	89	0.26	0.21	0.05	1.09	partial
`realv7_000`	CPU, LC on	11	5	89	0.27	0.21	0.05	1.06	partial
`realv7_000`	CPU, LC off	23	4	88	0.26	0.23	0.05	1.08	partial
`realv7_001`	GPU, LC on	3	4	89	0.48	0.47	0.07	1.07	partial
`realv7_001`	GPU, LC off	67	3, 5	6	0.82	0.63	0.15	1.15	partial
`realv7_001`	CPU, LC on	94	4	2	**0.44**	0.40	0.07	1.07	partial
`realv7_001`	CPU, LC off	65	5	13	0.56	0.50	0.10	1.11	failed
`realv7_002`	GPU, LC on	22	3, 49, 59	38	2.11	1.28	0.26	1.18	partial
`realv7_002`	GPU, LC off	22	49	60	2.08	1.83	0.23	1.29	partial
`realv7_002`	CPU, LC on	61	none	5	**0.47**	0.41	0.12	1.13	partial
`realv7_002`	CPU, LC off	62	none	22	1.67	0.98	0.70	1.68	failed

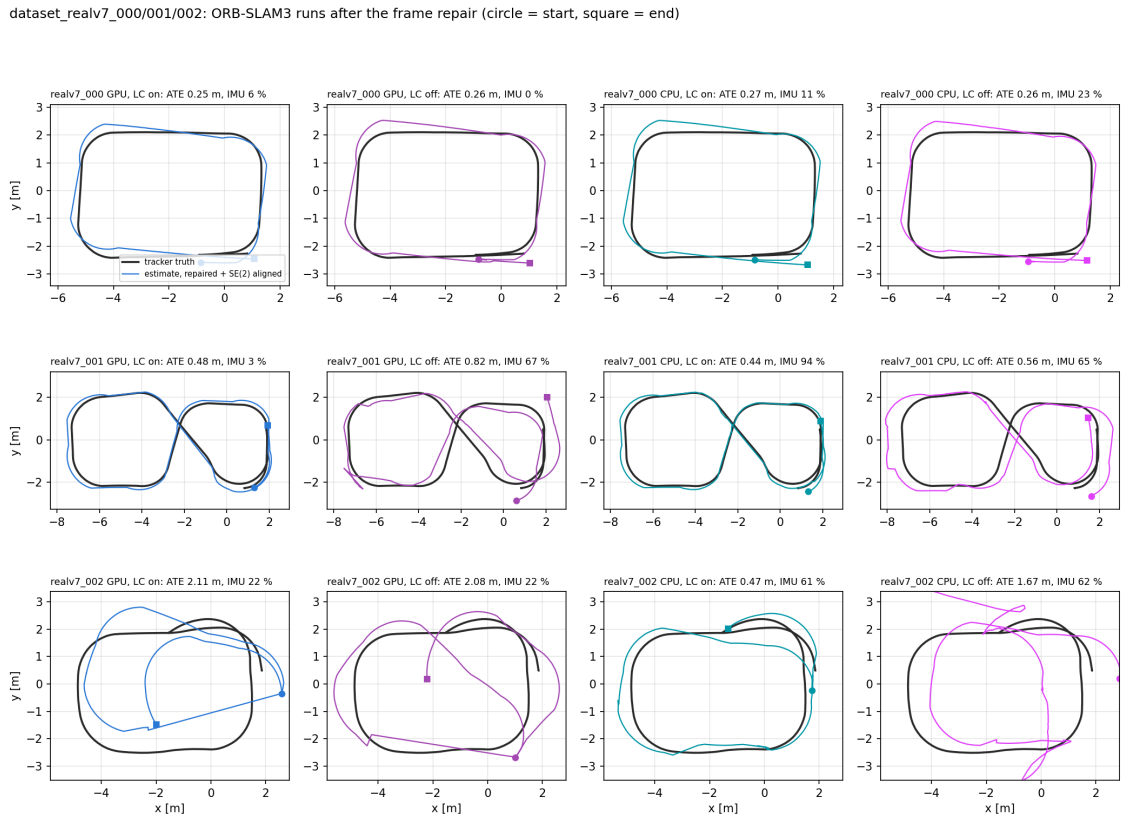


Figure 4.15 The twelve `realv7` ORB-SLAM3 runs after the scorer's frame repair, against the tracker truth after rigid SE(2) alignment; each panel's title gives the planar ATE and the share of tracked frames with the IMU initialised.

Table 4.21 VINS-Fusion on the three `dataset_realv7` recordings (`config/wil/stereo_imu.yaml`, IMU noise $\times 10$, gravity norm 9.2642). Frames received against the bag's count (30 Hz over the recording), IMU use, resets, and where the estimate ended.

Recording	frames received	IMU received / used	resets	z range [m]	$\lvert p \rvert$ at end [m]
`realv7_000`	1155 of about 2663	17906 / 19053	0	12834403	37554
`realv7_001`	1541 of about 3686	24809 / 26340	0	3701	15974
`realv7_002`	1271 of about 2910	19368 / 20627	0	25937	5252

after the scorer's frame repair, a measured cost of the CPU front end at real time, the cause of the IMU staying uninitialised for most of nine realv7 runs (an upstream map reset at the initialisation and a starved re-initialisation) patched in the fork but not yet exercised, and two diagnosed VINS-Fusion defects, one of them (the gravity norm) now corrected in the configuration. The experiment's own question, whether filtering helps, is still untested on real imagery.

Next action. Run one realv7 recording with the fork's two initialisation switches on and confirm from the per-frame log that the IMU stays initialised from the first seconds; re-run VINS-Fusion with the corrected gravity norm; repeat each ORB-SLAM3 configuration under a new run name; measure the marker-to-IMU lever arm; then run both pipelines filtered and unfiltered on the same recording, with the detector live, and score them with the admissibility gates of the simulation slots.

4.4 Experiment 4: Dense Map Reconstruction and Map-Quality Evaluation

4.4.1 Simulation

Objective. Verify that the reconstructed map reflects the physical environment and is usable by a planner (section 3.6.4).

Process. RTAB-Map 0.23.7 built map/map_tiny.db; its placement was measured by script/fit_map_pose.py; geometry and drivability were scored by script/validate_map.py against the world file and a ground-truth route. Four further databases were built in the realistic worlds with loop closure disabled on both sides of the pipeline, two per layout under the October and the current configuration, and scored by the same two scripts, each against a recording that did not build it (section 4.4.1).

Dataset. dataset_map_tiny (the build run) and the independent route dataset_static_tiny_repeat, which no longer exists; and, for the large warehouse, dataset_map_01 (table 3.18). The realistic-warehouse builds postdate that registry: dataset_allsensor_realistic_map (U layout, 285.7 s; held until 26 October as _mapv2) built map/map_realistic and map/dataset_allsensor_realistic_map, and dataset_allsensor_realistic_01_map (I layout, 314.9 s) built map/map_realistic_01 and map/dataset_allsensor_realistic_01_map; their independent probes are the static twins dataset_allsensor_realistic_000 and _001 of section 3.4.6.

Question, scope, and conditions

Trajectory error answers whether the robot knew where it was. It does not answer whether the map it produced describes the building, nor whether a planner could have used that map. This section evaluates the one dense map reconstructed so far against two references of very different strength: the simulator's own collision geometry, which is independent of the map, and the ground-truth route of the run that built the map, which is not. The two questions are kept separate because they fail in different ways — a map can be geometrically faithful and still unusable, and it can be navigable over an easy route while misrepresenting the parts of the building that route never entered — and because only the first reference is independent evidence.

The evaluation is confined to the **tiny** warehouse. A large-warehouse equivalent is **planned**; no database for it exists at the reporting cutoff. Tiny-map and large-map results must not be pooled or read as repeated trials of one condition, because the two worlds differ in extent, aisle width, and route length. Table 4.22 records the conditions.

Table 4.22 Conditions for the dense-map evaluation. The database and the ground-truth route come from the *same* run, which bounds what the drivability figures can establish.

Factor	Level
Map under test	`map/map_tiny.db`, RTAB-Map 0.23.7, created 2026-09-08
World	`aws-robomaker-small-warehouse-world/worlds/small_warehouse_static/small_warehouse_static_tinymap.world` (tiny geometry, despite residing in the `small_warehouse_static` family directory)
Scene dynamics	static
Grid resolution	0.05 m
Geometry tolerance	0.25 m
Robot footprint	0.83×0.50 m rectangle from `nav2_ackermann.yaml`, not a circumscribed radius
World-to-map placement	$x = 1.850\,\text{m}$, $y = 8.600\,\text{m}$, $\theta = -89.90^\circ$, measured once by `script/fit_map_pose.py`
Drivability reference, as plotted	`dataset_static_tiny_repeat`, 2973 poses; an independent run, no longer present in the workspace or on the removable drive
Substitute available	`dataset/dataset_map_tiny`, `/ground_truth/odometry`: 3032 messages over 60.6 s of simulation time. This is the run that *built* the map, so it is not an equivalent replacement
Evaluator	`script/validate_map.py`
Experimental unit, n	one database; $n = 1$

Map placement as a validity precondition

Both metrics depend on seating the occupancy grid in world coordinates, so placement is a precondition rather than a display choice: a mis-seated grid scores real structure against the wrong cells and makes a good map look bad. The transform is a constant for a given database and is measured once, by fitting occupied cells to the world file's wall and obstacle edges without using odometry or ground truth. It is deliberately not taken from the live `map` $\rightarrow$ `odom` correction, which moves with front-end drift.

The measured fit is $(1.850, 8.600)$ m at -89.90°, with a margin of 60.9 % over the best rival orientation more than 5° away. An independent prediction — the spawn pose of the IMU body frame, which sits 0.338 m ahead of `base_footprint` — gives $(1.800, 8.662)$ m. The two agree to 0.080 m, inside the grid's own 0.05 m resolution. Two independent routes to the same number is what makes this a measurement rather than a tuned parameter.

Graph and database summary

Table 4.23 summarizes the stored graph, read directly from the database in read-only mode. The counts are small: 30 nodes over a 59.3 s span, reflecting RTAB-Map's retention policy rather than the rate at which images were offered to it.

Of the three distinct closure pairs, $(9, 22)$ and $(10, 23)$ join nodes roughly half a lap apart

Table 4.23 Stored contents of `map/map_tiny.db`. Link rows are stored per direction; the undirected count is the number of distinct node pairs.

Element	Rows	Undirected	Note
Nodes	30	–	ids 1–30, stamp span 59.3 s
Neighbour links	56	28	sequential odometry constraints
Global loop closures	5	3	pairs $(9,22)$, $(10,23)$, $(29,30)$
Gravity links	30	30	one per node
Local-space links	0	0	none recorded
Visual words	10 357	–	vocabulary size
Features	28 374	–	stored keypoints

and are genuine revisits of an earlier place. The third, $(29,30)$, joins consecutive nodes and is therefore not a revisit; it should not be counted as evidence of place recognition.

Geometry against the world file

Geometry is scored against the rasterized collision geometry of the world file at a 0.25 m tolerance. Recall is measured against *surfaces* rather than filled interiors, because a stereo camera can only observe the outside of a shelf; counting its interior as missed geometry would cap even a perfect map well below 100 %.

Table 4.24 Occupied-cell geometry of `map/map_tiny.db` against `small_warehouse_static_tinymap.world` at 0.25 m tolerance.

Metric	Value	Population
Occupied-cell precision	96.6 %	11 634 occupied cells
Recall, region the map observed	89.5 %	observed surfaces
Recall, whole building	67.2 %	all surfaces in the world file

The gap between 89.5 % and 67.2 % is route coverage, not mapping failure: the run never entered the side aisles, so roughly a third of the building was never observed by any sensor. The two numbers answer different questions and neither replaces the other — the first is a statement about mapping quality, the second about where the robot went.

Drivability against the true route

The robot demonstrably drove the reference route, so every pose on it was collision-free in reality. Stamping the real footprint onto the map at each pose therefore converts any collision into a *proven* false obstacle, and any unknown cell into a hole a planner would have to cross blind. This metric needs no assumption about what a good map looks like.

It does, however, depend on where the route came from, and that is the weak point of this result. The route used to produce table 4.25 came from a separate run, `dataset_static_tiny_repeat`, which is no longer present in the workspace or on the removable drive. The figures below therefore stand as a dated independent measurement that cannot currently be re-derived.

The only surviving recording of this warehouse, `dataset/dataset_map_tiny`, is not an equivalent substitute, because it is the run that built the map. Three measurements establish this. Applying the placement of section 4.4.1 to the stored node poses puts

them a median of 0.069 m from that recording's ground-truth route; comparing each node against the ground-truth pose carrying the *same* timestamp gives a median of 0.340 m and a maximum of 0.52 m; and all 30 nodes fall inside the recording's sim-time interval. The second of these is the discriminating one, since a different run over the same loop would agree in space but not in time. The two routes are also visibly different: the lost run is a single oval circuit, while this one is a figure-eight through the centre of the building.

The evaluation was nevertheless repeated against this recording, and the outcome confirms the objection rather than resolving it. Both results are reported in table 4.25.

Table 4.25 Drivability of `map/map_tiny.db` under the true 0.83×0.50 m nav2 footprint, evaluated against two different routes. "Independent" is `dataset_static_tiny_repeat`, the run that did not build the map and is no longer available. "Build run" is `dataset/dataset_map_tiny`, the run the map was constructed from. Geometry is identical in both cases because it does not depend on the route.

Metric	Independent	Build run	Meaning
Poses scored	2973	3032	
Clear and mapped	99.3 %	100.0 %	footprint free and observed
False obstacle	0.7 %	0.0 %	proven error
Unmapped under footprint	0.0 %	0.0 %	blind cells on the route
Clearance, median	1.07 m	1.01 m	to nearest obstacle
Clearance, 5th percentile	0.45 m	0.46 m	inflation radius is 0.80 m

The difference between the two columns is the independence effect, measured rather than assumed. Against the run that built it, the map records *no* false obstacle at all: it cannot contradict the trajectory it was constructed from. The 0.7 % that the independent run exposes is precisely the part of the map that only a route the mapper had not seen could reveal. The right-hand column is therefore not a better result than the left; it is a weaker question, and the 0.0 % should not be quoted as a map-quality figure.

Clearance is nearly unchanged between the two, which is expected: it is a property of where each route runs relative to mapped geometry, not of which run built the map. Both routes spend part of their length inside nav2's inflation radius.

Interpretation and limitations

Within its stated scope the tiny map is geometrically faithful and navigable: 96.6 % of what it marks as occupied is real, it recovers 89.5 % of the surfaces it actually observed, and it contains no blind cells and only 0.7 % proven false obstacles along a route the robot really drove. The geometry half of that is independent evidence and is the strongest map result the project currently holds; the drivability half is a self-consistency check and is weaker than it looks. Both are a direct improvement on earlier assessments that judged the same database unnavigable — those were measured with the grid mis-seated, so the comparison was against the wrong cells. That correction is a methodological finding in its own right and is why section 4.4.1 reports placement before any metric.

Four limitations bound what these numbers support.

- **The drivability reference cannot be re-read.** The run that provided it is lost, and the only surviving recording of this warehouse is the one that built the map, which would answer a weaker question. Re-establishing this result needs a fresh

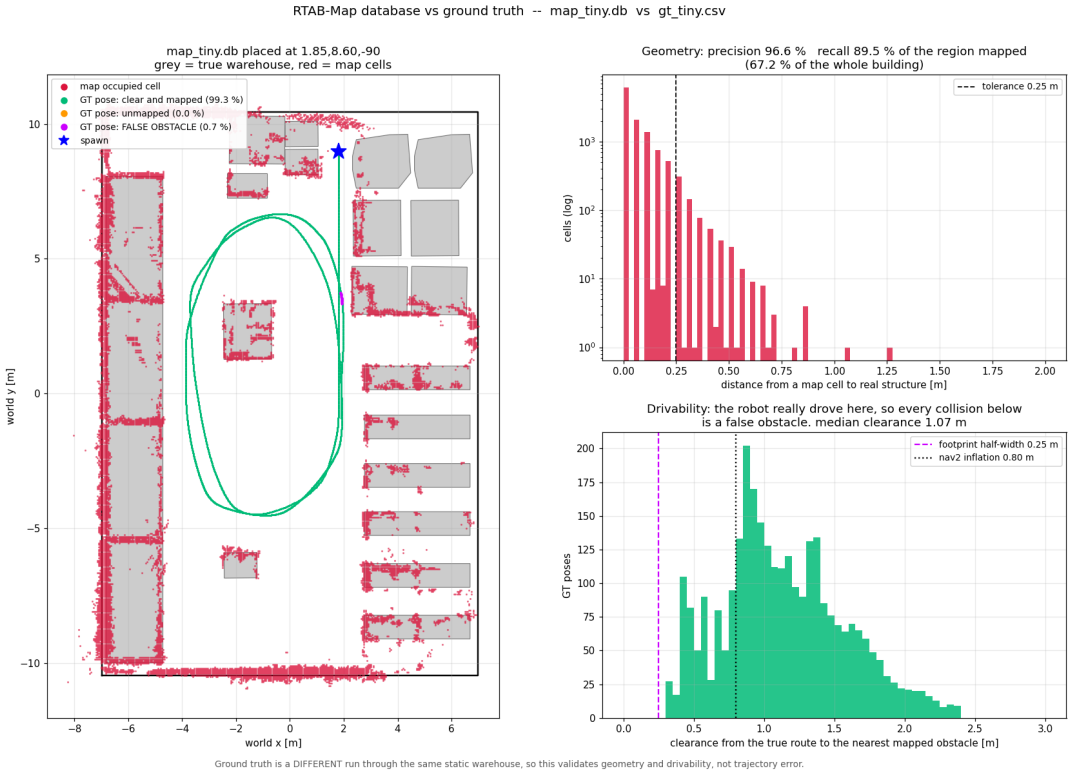


Figure 4.16 Dense-map evaluation of `map/map_tiny.db`. Left: occupied cells (red) over the world file's true collision geometry (grey), with the ground-truth route coloured by drivability and the spawn marked. Top right: distance from each occupied cell to real structure, with the 0.25 m tolerance marked; the bulk falls well inside tolerance with a thin tail to about 1.25 m. Bottom right: clearance from the true route to the nearest mapped obstacle, against the footprint half-width and the nav2 inflation radius. The ground truth was a different run through the same static warehouse, so this validates geometry and drivability, not trajectory error; that run is no longer available (section 4.4.1).

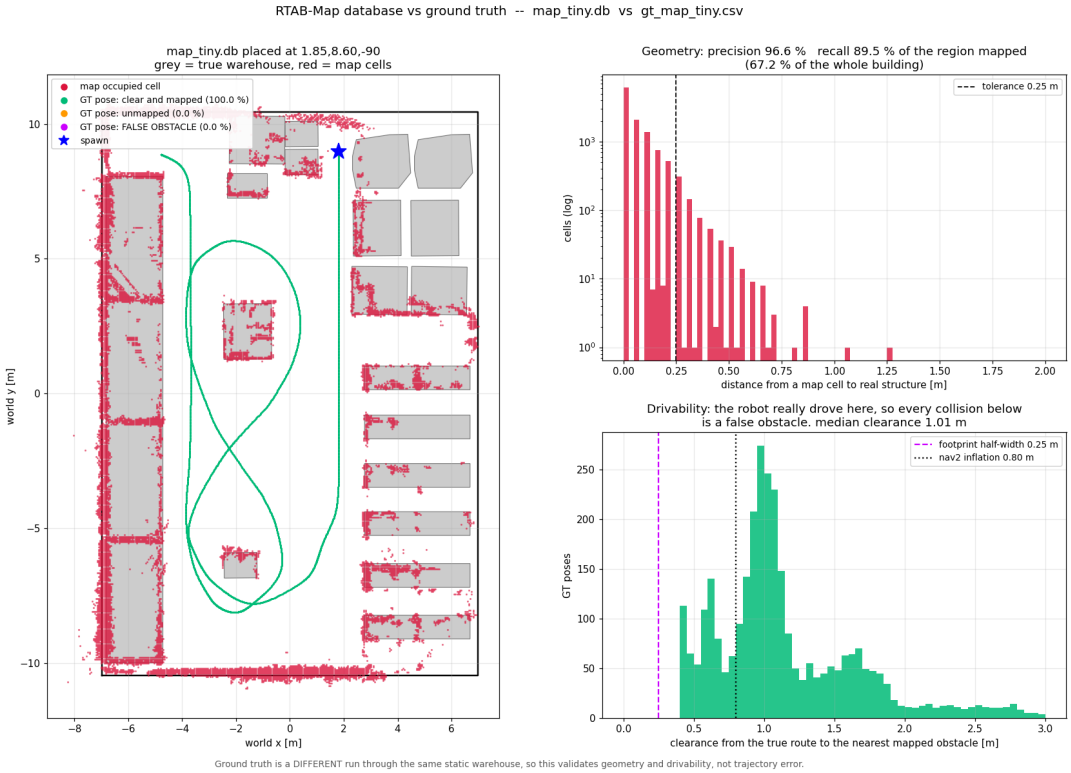


Figure 4.17 The same database scored against the run that built it, `dataset/dataset_map_tiny`. The route is a figure-eight rather than the oval of figure 4.16, and no pose on it collides with the map: the false-obstacle rate falls from 0.7 % to 0.0 %. The geometry panels are identical to figure 4.16 because geometry does not depend on the route. The footer claiming a different run is emitted unconditionally by the evaluator and is incorrect for this figure.

independent recording — a second run, not a second map.

- **The route is easy.** It is one large loop down the wide central aisle and never squeezes into a side aisle. The 0.7 % false-obstacle figure is therefore a floor, not a worst case. A route through the narrow aisles is the test that would actually stress this map, and it has not been run.
- **Clearance couples to planner cost.** The 5th-percentile clearance of 0.45 m sits below nav2's 0.80 m inflation radius, and a visible share of the true route lies inside it. Nothing is blocked, but a planner would charge a high cost for driving where the robot demonstrably drove. That coupling should be examined before any planned path is trusted.
- $n = 1$**, and the scope is one world.** A single database on the tiny warehouse supports no uncertainty estimate and no claim about the large warehouse, where longer routes and narrower bays change both the drift exposure and the clearance margin.
- **Residual distortion is not a placement problem.** Walls appear doubled and thickened and shelf rows bowed in the overlay. No rigid transform removes that; it is internal map distortion and is the property that would limit navigation, independently of how well the grid is seated.

Planned extension to the large warehouse

The same evaluation is **planned** for the large warehouse and is not started: no large-map database exists at the cutoff. Three things must be settled before those numbers would mean anything. The world-to-map placement must be measured for the new database rather than assumed, since the tiny-map constant $(1.850, 8.600, -89.90^\circ)$ is specific to that database and world. A ground-truth route must be recorded from a run *other* than the one that builds the map, which the tiny-map case did not do and which is what would turn drivability into independent evidence. Finally, the route should deliberately include narrow bays, so that the false-obstacle rate is exercised rather than merely reported. Until then, no map-quality claim extends beyond the tiny warehouse.

Large-warehouse attempt

The progress update of 23 September 2026 reports that the large-warehouse map attempt **failed**: the front end lost tracking in the repetitive bay structure of the large world, the estimated pose diverged from the ground-truth route within a few time steps of the loss, and the resulting map was not scorable. The failure was attributed there to the front end under repeated structure, not to the mapping back end. No database, run directory or figure for this attempt is registered in the repository at the cutoff, so the result is reported on the strength of the progress record and cannot be audited. A database identified from its contents as this attempt (18 nodes, 37 m along one aisle, poses leaving the floor, placeable in no large world) was found among the unregistered databases on 22 October 2026 and deleted with them on 6 November, unscored. The attempt must be repeated with its artifacts retained before it is carried into any conclusion.

Realistic-warehouse maps

Four databases built in the realistic worlds of section 3.4.6 replace the unauditable large-warehouse attempt: two mapping recordings, one per layout, each built twice, on 26 October 2026 and on 6 November 2026 under the pipeline's current configuration. Every one has a registered build recording, a fitted placement, a score and a figure, and all four

fail the quality criterion fixed before they were scored.

Reporting rules. The criterion, fixed in `script/eval_realistic_maps.py` before any map was placed, is occupied-cell precision $\geq 90\,\%$, observed-region recall $\geq 85\,\%$ and false obstacles $\leq 1\,\%$ of probe poses, each taken from the tiny map's result; whole-building recall is reported, not tested. The metrics are those of section 3.6.4. Placement was measured by `script/fit_map_pose.py` with the realistic spawn $(5.250, 19.995, -90^\circ)$ and confirmed on the overlays before scoring. Scoring is the unchanged `script/validate_map.py` at 0.25 m tolerance with the 0.83×0.50 m footprint, each map probed by the static twin of its own layout, a recording that built none of them. All four databases are closure-free (`Rtabmap/LoopThr` 1.0, `RGBD/ProximityBySpace` false; `loop_closure:=false`, RTAB-Map record §7c) and optimised with GTSAM. The two builds of each recording differ in exactly three back-end parameters, `Rtabmap/DetectionRate` 4 against 1 Hz, `Grid/RangeMax` 8 against 10 m and `Grid/MaxGroundHeight` 0.29 against 0.10 m (October against November), and in front-end session, which neither database records; they are two configurations, not a repeat pair, so $n = 1$ per cell and no difference between builds is attributable to one parameter. The recording now named `dataset_allsensor_realistic_map` is the one held until 26 October as `_mapv2` (identical message counts); the original 103–316 s mapping recording and the 114-node database built from it were deleted that day, and `map/map_realistic` is the former `map_realistic_v2`. Node stamps fix the pairing: the U databases span 13–300 s and the I databases 20–335 s, which the 125 s and 97 s probes cannot have produced. The November U placement was fitted in the -180° basin with `-yaw-near`, because the global winner, 4.2 % better by agreement, was the rotational rival the overlay rejects; both candidates are in `fit_dataset_allsensor_realistic_map.log`. Every number is under the experiment directory `exp4-realistic-maps` of `output/compare/simulation`: the scores in `realistic_map_metrics.csv`, one `validate_*.json` per map, the inventory with the three parameters in `map_inventory.json`, and the placement logs.

Results. Table 4.26 gives geometry, table 4.27 drivability against the independent probes, and table 4.28 the stored graphs and the three parameters that separate the builds. Figures 4.18 and 4.19 are the placement overlays of the two U builds and figures 4.20 to 4.23 the evaluation overlays.

Table 4.26 Occupied-cell geometry of the four realistic-warehouse databases against their world files at 0.25 m tolerance, beside the tiny map. Bold marks a value below the fixed criterion.

Database	Built	Nodes	Occupied cells	Precision [%]	Recall observed [%]	Recall building [%]
`map_tiny`	Sep	30	11 634	96.6	89.5	67.2
`map_realistic`	Oct	165	26728	**81.2**	**82.5**	55.7
`dataset_allsensor_realistic_map`	Nov	148	22715	**77.8**	**74.9**	57.1
`map_realistic_01`	Oct	185	25440	**84.8**	**81.4**	54.5
`dataset_allsensor_realistic_01_map`	Nov	127	23255	**82.2**	**77.1**	55.3

Analysis. All four maps fail, and the November build of each recording scores below its October sibling on every criterion: precision by 3.4 and 2.6 points, observed recall by 7.6 and 4.3, false obstacles from 3.8 to 5.0 and from 4.4 to 10.2 %. The October pair fail by similar margins on both layouts, which made their failure a property of the build rather than of one run; the November pair move it further in the same direction.

Table 4.27 Drivability of the four realistic-warehouse databases under the true 0.83×0.50 m nav2 footprint, against the independent probe of each layout; U-Oct is `map_realistic`, U-Nov `dataset_allsensor_realistic_map`, I-Oct `map_realistic_01`, I-Nov `dataset_allsensor_realistic_01_map`. The tiny map's independent figures were 2 973 poses, 0.7 % false obstacles, clearance 1.07 and 0.45 m; its probe was one loop of a wide aisle, so the comparison is not route-matched.

Metric	U-Oct	U-Nov	I-Oct	I-Nov
Probe poses	6278	6278	4847	4847
Clear and mapped [%]	96.2	95.0	94.9	89.5
False obstacle [%]	**3.8**	**5.0**	**4.4**	**10.2**
Unmapped under footprint [%]	2.4	1.9	0.7	0.5
Clearance, median [m]	1.96	2.00	2.16	1.98
Clearance, 5th percentile [m]	0.62	0.38	0.36	0.18

Table 4.28 Stored contents and build parameters of the four realistic-warehouse databases, read in read-only mode. The three parameter rows are the only ones that differ between the builds of a recording. Neighbour links are stored per direction; a "spanning" closure joins nodes more than three apart and none exists, by configuration. The margin is the agreement over the best rival more than 5° away; "basin" marks the placement chosen with `-yaw-near`.

	U-Oct	U-Nov	I-Oct	I-Nov
Built	2026-10-26	2026-11-06	2026-10-26	2026-11-06
`Rtabmap/DetectionRate` [Hz]	4.0	1.0	4.0	1.0
`Grid/RangeMax` [m]	8.0	10.0	8.0	10.0
`Grid/MaxGroundHeight` [m]	0.29	0.1	0.29	0.1
Nodes	165	148	185	127
Node stamp span [s]	285.7	286.2	314.9	313.9
Optimised path [m]	369.5	364.2	384.7	372.3
Neighbour links, rows	322	294	356	248
Closure rows / spanning > 3	3 / 0	0 / 0	4 / 0	0 / 0
Visual words	55802	51539	63276	44342
Database size [MB]	417	462	438	377
Placement (x, y, θ)	$(5.300, 19.500, -179.9°)$	$(5.200, 19.800, -180.1°)$	$(5.150, 20.450, 179.4°)$	$(4.950, 20.400, -179.9°)$
Fit agreement, margin	0.183, 10.1 %	0.161, −4.2 % (basin)	0.260, 66.1 %	0.252, 60.2 %

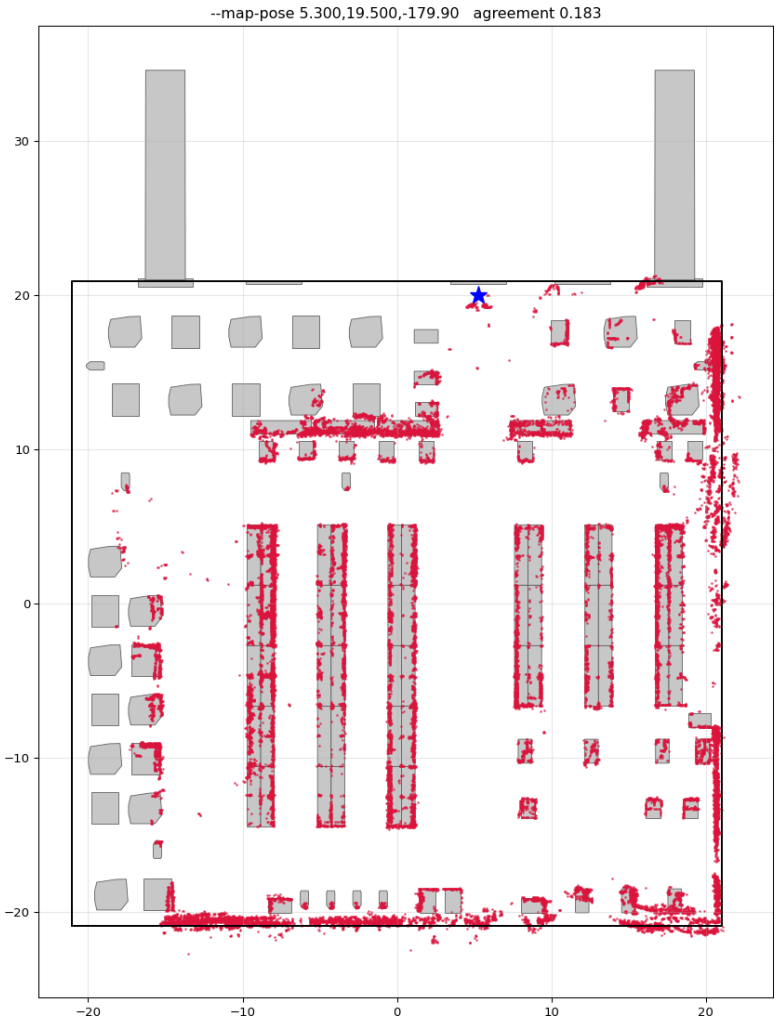


Figure 4.18 Placement overlay of `map/map_realistic` (U-Oct), agreement 0.183, 10.1 % margin over the rival orientation. Red occupied cells over the world file's obstacle footprints and wall; shelf rows, the bay strip at $y \approx 11$ and all four walls coincide.

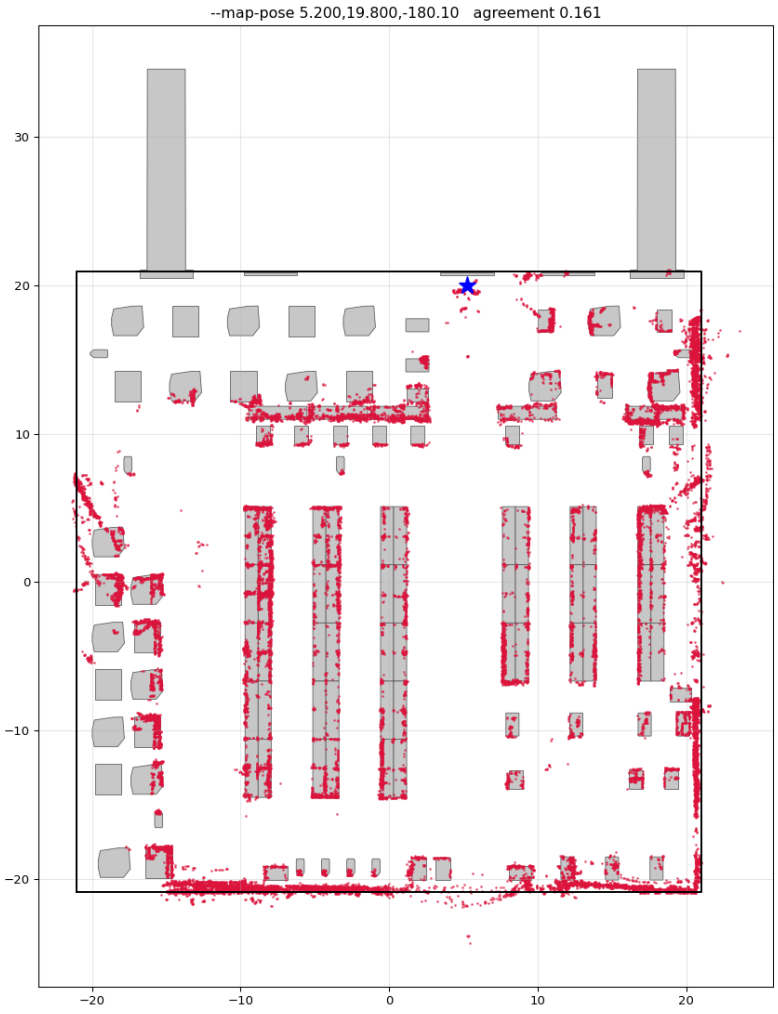


Figure 4.19 Placement overlay of `map/dataset_allsensor_realistic_map` (U-Nov). The unconstrained sweep preferred the 180°-rotated rival at $(-5.6, -19.6)$, agreement 0.168 against 0.161, which puts shelf rows between shelves; the basin placement shown sits 0.15 m from the spawn of the IMU body, as its October sibling does.

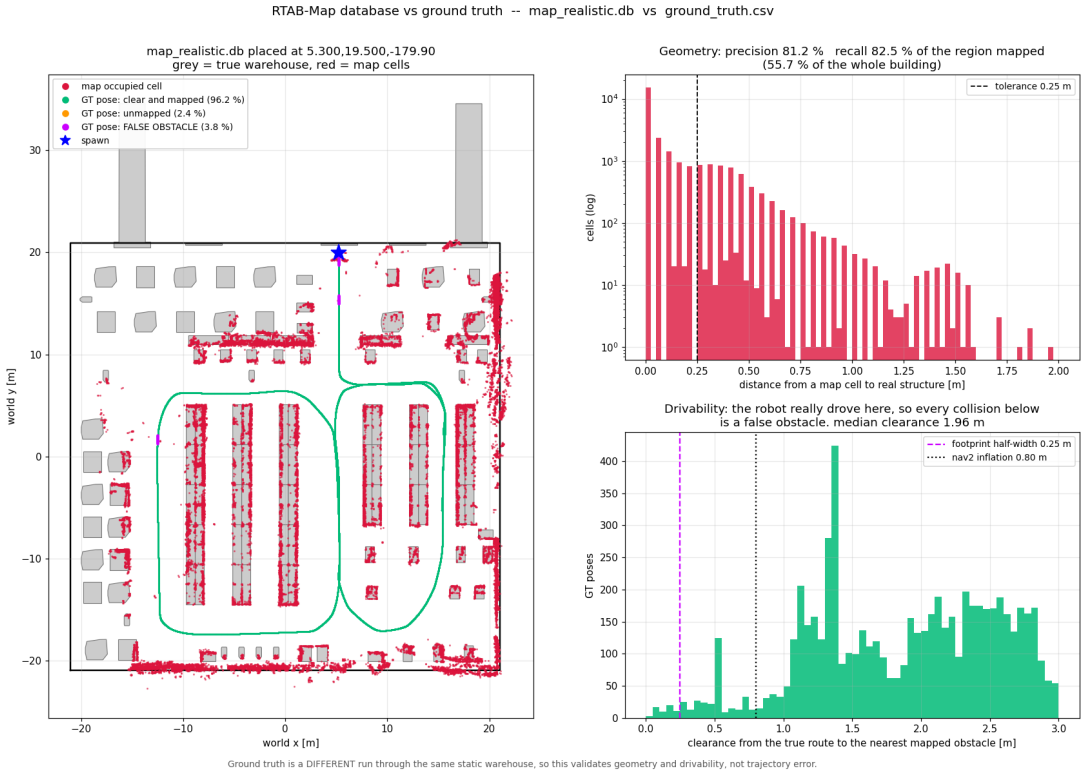


Figure 4.20 Dense-map evaluation of `map/map_realistic` (U-Oct) against `dataset_all sensor_realistic_000`. Panels as in figure 4.16. The shelf rows are thickened toward the aisles; the false-obstacle poses sit on the first metres below the spawn and where the probe hugs the left shelf column at $x \approx -12.5$.

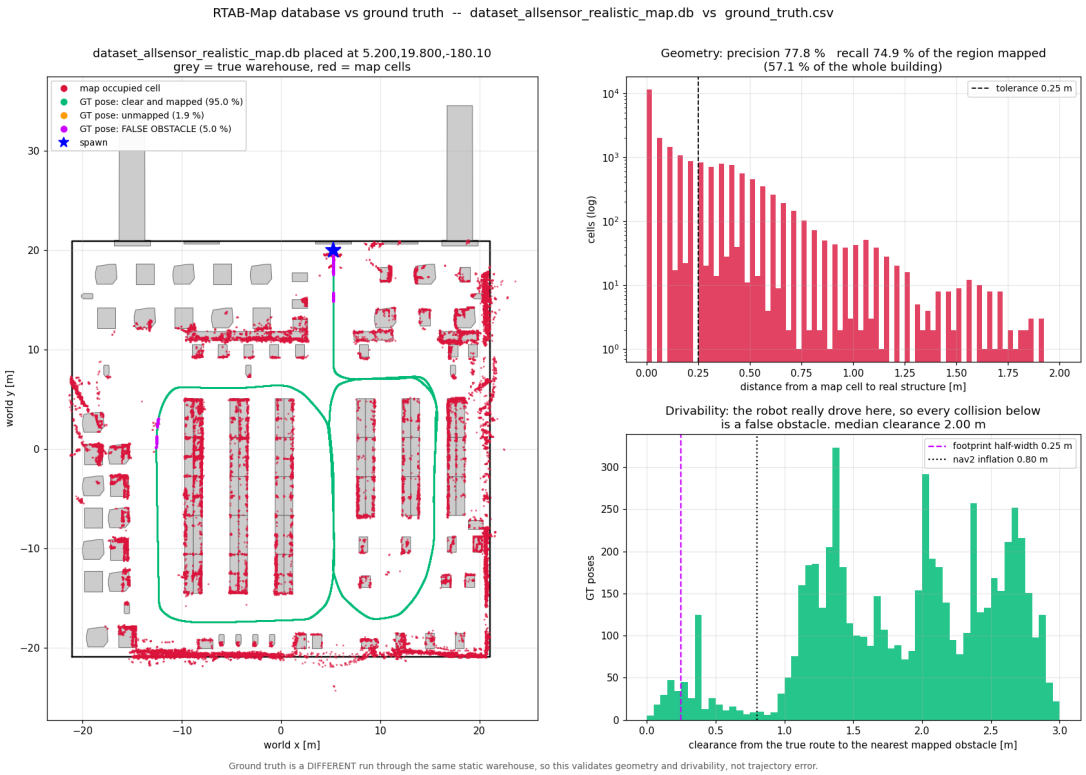


Figure 4.21 Dense-map evaluation of `map/dataset_allsensor_realistic_map` (U-Nov) against the same probe. The same two blocked places as U-Oct, the spawn stretch now longer, and more scatter in the outer bays and along the west wall, which this build's 10 m grid range reaches and the October build's did not.

Three parameters moved together between the builds, and each has its own signature in the data, so the reading is consistent-with, not attribution. The detection rate, 4 to 1 Hz, shows as fewer nodes (148 against 165, 127 against 185) and fewer observations per cell, which the view-count effect of the tiny-map insight analysis turns into lower precision and recall. The grid range, 8 to 10 m, shows as structure the October builds never reached: the west wall and the far outer bays now carry occupied cells, and the largest new phantom clusters sit on them. The ground-height threshold, 0.29 to 0.10 m, shows where it would: on the I layout the November build blocks 518 probe poses against 213, and the new stretches are mid-route, in the aisles, on cells first observed at median node 27 rather than at the start. A one-parameter rebuild is the test that would separate the three, and is the next action.

In the October builds the loss is mostly thickened structure, with a phantom share the tiny map did not have. Classifying the wrong cells (`script/exp4_insights.py`): 13.4 and 9.7 % of occupied cells are surfaces drawn too thick, within 0.50 m of true structure, and 5.2 and 5.3 % are phantoms further out, against 3.0 and 0.4 % on the tiny map; in the November builds the shares are 15.4 and 11.5 % thickened and 7.3 and 6.3 % phantom. The thickening sits on the shelf rows the build route passed; the phantoms are the east and south walls, drawn about 0.5 m off their true line in all four maps and far from any route. In the October builds the false obstacles are not the thickened shelving: every blocked probe pose lies within 0.6 m of a phantom cell, the largest blocked stretch of each map is the first metres from the spawn, and 96 and 63 % of the phantom cells under those stretches were first observed by one of the first five graph nodes, during the inertial initialisation transient. The November U build repeats that pattern (224 spawn poses, 88 % of the blocking cells early); the November I build does not, which is the ground-height signature above.

All four are an odometry picture by design, and should be read as one. Loop closure was disabled on both sides of the pipeline after the A/B of 26 October showed the front end's own closures corrupting an otherwise 0.1 m map (RTAB-Map record §7c), so no node was ever corrected against a revisited place; the stored graphs confirm it, with no closure spanning more than three nodes and none at all in the November builds. The thickening and the displaced walls are therefore uncorrected drift along a 365–385-metre build route re-observing the same structure from slightly different estimated places, which is what this configuration was built to expose. It is not evidence about RTAB-Map's loop closing, which was not exercised.

Whole-building recall of 54.5 to 57.1 % is route coverage, as on the tiny map: the build routes loop the central block and the outer bays were seen only in passing. Observed-region recall, the mapping figure, is 74.9 to 82.5 %.

Placement needed the overlay on three of the four fits, and the spawn check did not apply to any. The fits land about 180° from the −90° spawn prediction because the map frame was seated during the inertial initialisation transient rather than at the body's spawn pose: node 1 of the October databases carries a yaw of 57 and −89° and a height still settling over the first nodes, where the tiny map's node 1 is the identity. On the November U build the unconstrained sweep chose the rotated rival outright; the overlays leave no doubt about any of the four placements, and the two builds of each layout agree to 0.3 m and 0.5°.

Limitations, each stated once. *Not route-matched:* the tiny probe was one loop of a wide aisle, these probes thread shelf aisles (5th-percentile clearance 0.18–0.62 against 0.45 m), so part of the false-obstacle gap is probe difficulty and the 1 % threshold was set against an easier route. *No attribution:* world and route vary together between layouts, and three

parameters and the front-end session vary together between builds, so no factor may be credited for any difference in this slot. *n:* one database per cell supports no uncertainty estimate. *Front end unrecorded:* the databases store the back end's parameters but not which front end, in which session, supplied the odometry chain.

Summary. Where tracking holds, the mapping pipeline is geometrically sound on the tiny map: occupied-cell precision 96.6 % and observed-surface recall 89.5 %, with whole-building recall of 67.2 % explained by route coverage; the drivability half is a dated measurement whose independent reference is lost. On the realistic warehouse four maps built without loop closure, two per layout under the October and the current configuration, are auditable end to end and all fail the same criterion: precision 77.8 to 84.8 % and false obstacles 3.8 to 10.2 % against independent probes. Their loss is thickened shelving, displaced walls and phantoms at the spawn from uncorrected drift, which the closure-free build was configured to expose; the current configuration scores lower on every metric, for reasons the three parameters it changed at once do not let this slot separate. The earlier large-warehouse attempt that lost tracking remains unauditable.

Future plan. Attribute the November drop by rebuilding one recording with a single parameter restored at a time, first the ground-height threshold (`Grid/MaxGroundHeight`), then the detection rate, and rescoring with the unchanged evaluator. Name the front end, its session and the three parameters in every build manifest. Use the four maps as the odometry baseline against which a corrected map is compared: re-enable closure with a translation bound on accepted corrections, or merge the closure-free sub-maps through Atlas, and rescore. Discard the cells of the first few nodes, or start the map after the inertial transient has settled, which the data predict would remove the spawn stretch. Record the tiny-warehouse independent route again and retain it.

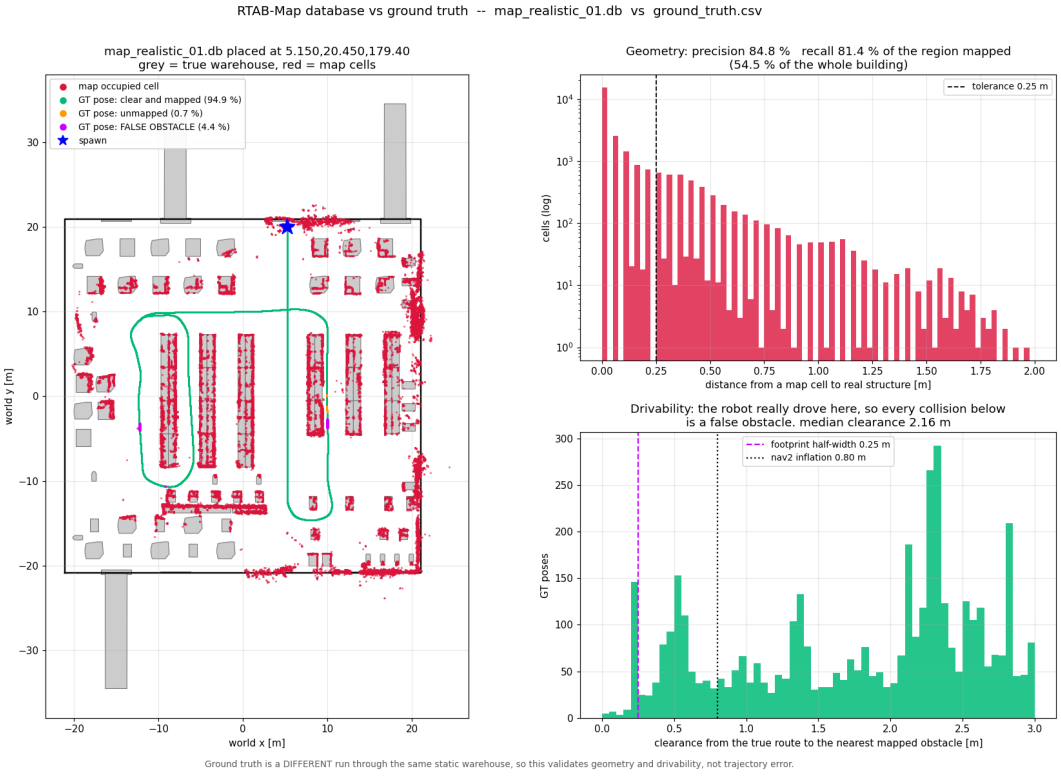


Figure 4.22 Dense-map evaluation of `map/map_realistic_01` (I-Oct) against `dataset_allsensor_realistic_001`. The clearance mode at exactly 0.25 m is the probe threading the one-aisle gap at $x \approx 10$ between two mapped shelf rows.

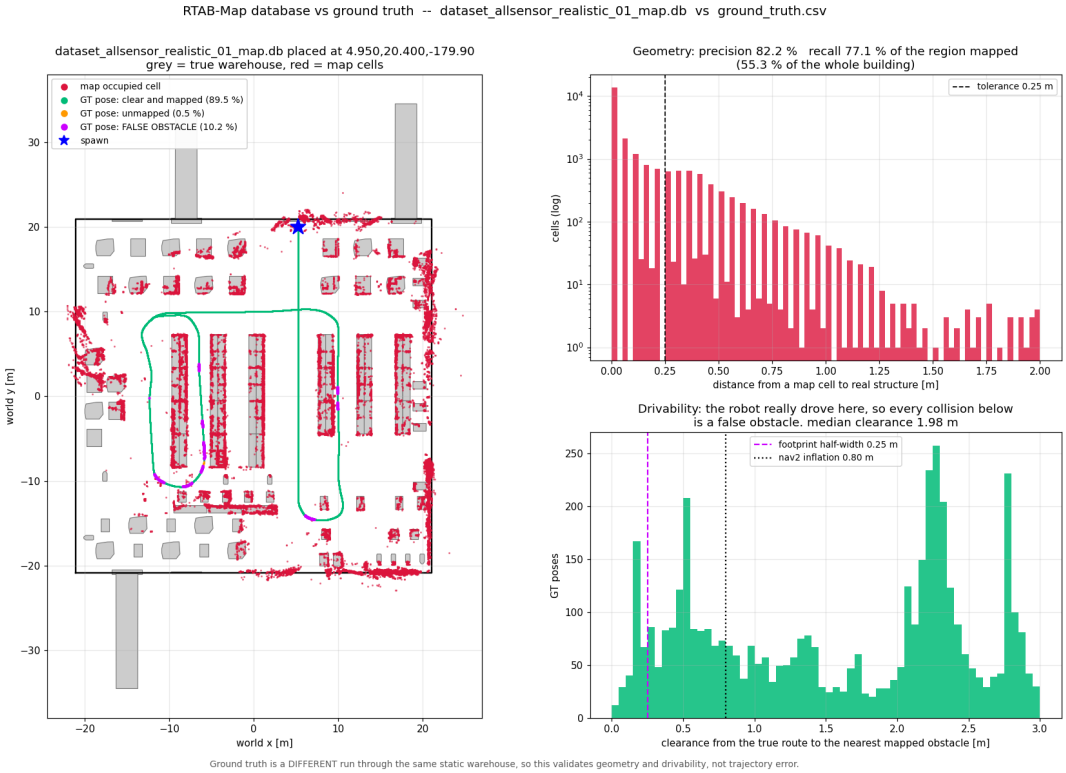


Figure 4.23 Dense-map evaluation of `map/dataset_allsensor_realistic_01_map` (I-Nov) against the same probe. The blocked poses are no longer confined to the spawn: stretches of 68, 55 and 50 poses sit mid-route in the aisles of the probe's left loop and along its bottom straight, on cells the map places on the floor of the aisle.

4.4.2 Real robot

Reason. The geometry metric of section 3.6.4 scores occupied cells against known collision geometry; the real site has no such model yet, and the placement step needs it. The drivability metric needs the motion-capture route. Neither input exists at the cutoff.

Next action. Survey the mock warehouse (walls, stacks, walkway) into a reference occupancy grid at the same 0.05 m resolution, build a database from a real mapping run, and then score it with the unchanged evaluator, using a motion-capture route from a second run for drivability.

4.5 Cross-Experiment Discussion, Simulation-to-Real Comparison and Limitations

4.5.1 Simulation against the real robot

Two slots now hold a number in both domains, and the comparison they allow is narrow and consistent. ORB-SLAM3 with the GPU front end and no divergence event holds 0.14–0.52 m aligned ATE on the realistic simulated warehouse (section 4.3.1) and, against the optical tracker, 0.25–0.27 m on `realv7_000` and 0.46–0.52 m on `realv6_000` (section 4.3.2); the real figures are planar, provisional until the marker lever arm is measured, and on routes a fifth to a tenth as long, so the agreement is of order, not of value. Its failure modes differ by domain. In simulation every divergence is the loop closer's: a false closure between the repeated aisles of the I layout in both front ends, and a vertical drift that only the closer corrects. On the real robot the closer changed nothing on either recording set, and the failures were the inertial initialisation (an upstream map reset at the initialisation and a starved re-initialisation on nine `realv7` runs) and the CPU front end's frame loss at real time. VINS-Fusion has the opposite record: the most accurate estimator on every simulated recording (0.22–0.66 m, no jump) and no valid real run on any of the four, through IMU delivery and configuration defects that are diagnosed and in part corrected but not yet re-run. The non-visual baseline now exists in both domains, and in both it is at least as accurate as vision: in simulation the measured-noise filter, on the real robot wheel+gyro dead reckoning with one fitted speed scale, 0.22–0.46 m against the best visual run's 0.44–0.52 m on the held recordings (section 4.1.2); and the two hypotheses the project is named for, filtering and dense mapping, have no real-robot number at all. What the simulation slots establish for the real campaign is the order in which its risks should be retired: the inertial noise model and the initialisation before any pipeline is tuned (sections 4.1.1 and 4.3.2); the loop closer's correction bound before it is trusted in repeated structure (section 4.3.1); and the detector's moving-object recall of 0.29, which bounds any filtering benefit before the filter is judged (section 4.2.1).

4.5.2 Limitations

Table 4.29 lists the limitations of the results reported in this chapter with the consequence each one carries. "Scope" identifies the experiment affected.

No root cause is assigned in this chapter beyond what a run's own artefacts show: the loop-closure corrections, the manifests, the initialisation logs. The decisions the chapter leaves open are the ones the Open Concerns document collects.

Table 4.29 Limitations of the results reported in this chapter, and what each one prevents. Scope: 1–4 the experiment, S simulation, R real robot.

Scope	Limitation	Consequence
all S	one run per cell; no condition repeated	no mean, deviation or interval; a between-cell difference is a candidate, not an effect; the only run-to-run figure is where two ORB-SLAM3 front ends agree
all S	route and traffic confounded: no two recordings share a route	traffic is a condition of a recording, not a tested factor
all	one laptop, one replay rate, per-process resource figures	no evidence of real-time capability on the robot's computer; frame retention is a property of this machine
1 S	the shared inertial noise model was fitted to neither family	the EKF–VIO ordering is a tuning result; the measured-noise filter shows the size of the effect
1 S	idealised simulated encoders, slip under 1 %	the proprioceptive estimators' best case
1 R	the wheel speed scale is fitted against the tracker on one recording, not measured; the steering feedback is a command echo	the baseline has one tuned constant and no yaw-rate observation, so no EKF
2 S	projected truth with exemptions; the walking people are actors without pose or mesh and outside the detector's classes	the scored basis is the easier one; people are unscorable and unmaskable
3 S	no filtered run in the current dataset set	the filtering hypothesis is untested
3 S	the ATE is three-dimensional	vertical drift is charged to a floor robot; the planar onset table is the complement
3 S	the false closure is diagnosed from its correction, not its images	a diagnosis, not a fix
3 S	the mapping-recording repeats were run one configuration at a time, not interleaved, on a laptop that throttles	the I-recording front-end comparison is confounded with thermal state
3 R	marker-to-IMU lever arm unmeasured; `realv6` tracker export at 1.7 Hz fails the validity rules	every real ATE provisional; `realv6` values are diagnostics
3 R	VINS-Fusion has no valid real run	no real-robot estimator comparison
3 R	the fork's initialisation switches (patch 15) are untested on a bag	the `realv7` repair is in the scorer, not yet in the estimator
4 S	the tiny map's independent drivability route is lost	its drivability result cannot be regenerated
4 S	the two configurations of each realistic map are one build each	the parameter attributions are signatures, not ablations
4 R	no real-imagery database	the slot is planned

CHAPTER 5 CONCLUSION AND RECOMMENDATIONS

This project set out to replace infrastructure-based navigation with a vision-based pipeline that stays accurate when the warehouse changes, and to decide between two ways of building it. At the reporting cutoff the pipeline exists in both forms, the four experiments that judge it are defined for both domains, and the evidence is drawn from one simulated warehouse in two layouts, recorded with and without traffic, and from four real recordings with an optical-tracker reference. All four simulation slots and two real-robot slots hold data; the two hypotheses the project is named for, dynamic-object filtering and dense mapping on the real robot, hold none. The conclusions below are bounded by the validity limits recorded beside each slot in chapter 4 and collected in table 4.29.

5.1 Conclusions by Experiment

5.1.1 Experiment 1: is vision earning its place?

On the four realistic recordings a five-state wheel-plus-gyroscope EKF, run on the inertial noise model the visual systems themselves declare, localises to 0.68–2.80 m aligned ATE and is behind VINS-Fusion (0.22–0.66 m) on every recording (table 4.3). The same filter on a noise model measured from each recording holds 0.21–0.58 m and would be ahead of VINS-Fusion on three of four, so the whole of the gap is the noise model and none of it the architecture: over this range the inertial noise model is worth more than the choice of estimator, and the visual systems have never been tuned to this platform. Two results survive that control. The proprioceptive sensors fail in complementary ways, the IMU holding heading to under 2° and losing 12–23 m of position, the wheels the reverse, and the filter is better than either in both quantities because it observes the gyroscope bias rather than the heading. And every dead-reckoning covariance is overconfident, by one to three orders of magnitude. Against the project's target of 0.05 m ATE, no estimator qualifies; against the 1.5 m RPE target at 1 s, all but the IMU do. The comparison is from one run per cell, on simulated encoders that barely slip, and with traffic present as a condition rather than a factor.

On the real robot the slot now holds the comparison it exists for (section 4.1.2). Wheel+gyro dead reckoning, with the feedback speed scaled by one constant fitted against the tracker on `realv7_000` and held, localises the robot to 0.26 and 0.22 m on the other two `realv7` recordings and 0.46 m on `realv6`, where the best ORB-SLAM3 run reads 0.44, 0.47 and 0.46 m. On routes of 22–37 m vision has not earned its place over the proprioceptive sensors in either domain. The baseline carries one tuned constant and no yaw-rate observation, so no EKF could be formed, and both sides are scored with the marker lever arm unmeasured.

5.1.2 Experiment 2: how good is the detector?

The four-class detector is reliable at naming and unreliable at finding (section 4.2.1): over 12 698 frames of the four realistic recordings, pooled precision 0.747, class F1 0.994 given a match, recall 0.502 over the truth boxes the exemptions keep and 0.288 over everything in view, and moving-object recall 0.288 on the pallet jacks. The miss is a size effect,

recall rising from 0.02 to 0.64 with the truth box's side, and three quarters of the false positives are boxes a little off the right object. That moving-object recall is the ceiling of any filtering benefit downstream: a mask built on this detector covers under a third of the moving pallet jacks' frames and none of the walking people, which are simulator actors outside its classes and outside the score. The real recordings have no annotated frames, so the real-robot slot stays planned.

5.1.3 Experiment 3: does filtering help?

The question the project is named for has no answer yet: no filtered run exists in the current dataset set, in either domain. What the twenty unfiltered simulation runs establish is the precondition and one of its answers (section 4.3.1). With physically simulated traffic neither pipeline loses tracking. VINS-Fusion holds 0.22–0.66 m with no jump and is worst on the U layout with traffic, where its error rises on the aisles the pallet jacks work; ORB-SLAM3 holds 0.14–0.52 m in eleven runs of sixteen, and its five divergences are the loop closer's, in both directions: a false closure between the repeated aisles of the I layout that both front ends accept at the same frame with a 14 m correction, putting them 4.8–6.0 m off where the closure-free runs hold half a metre, and a vertical drift on the same layout with traffic that only an accepted closure removes (table 4.12). The front end changes throughput, 97–99 % of frames kept with the GPU extractor against 72–94 % with the CPU one, and not accuracy. ORB-SLAM3's 13–19° rotational RMSE is one initialisation event per run, not its tracking; the median rotational error of every run is under 0.4°. Eighteen repeated runs on the two mapping recordings measure what each pipeline costs: VINS-Fusion optimises in 44 ms against ORB-SLAM3's 211–406 ms of local mapping and holds its state in a quarter of the memory, for more than twice the processor time, about eight cores of one against three and a half (table 4.14). Neither pipeline is cheaper outright, and the choice between them is a memory-against-cores trade.

On the real robot ORB-SLAM3 tracks every recording: a provisional 0.46–0.52 m planar ATE on `realv6_000` with the GPU front end and 0.25–0.27 m on `realv7_000` in all four configurations after the scorer's frame repair, with the marker lever arm unmeasured (section 4.3.2). The cause of the IMU staying uninitialised for most of nine `realv7` runs is identified in the upstream code and patched in the fork, and the patch is untested on a recording. VINS-Fusion has no valid real run: its two defects, IMU delivery and the gravity norm, are diagnosed and in part corrected. Loop closing changed nothing on the real robot that the front end did not change more.

5.1.4 Experiment 4: is the map usable?

Where tracking is sound, the mapping pipeline is: the tiny-warehouse map is 96.6 % precise in what it marks occupied, recovers 89.5 % of the surfaces it observed, and contains no blind cells and 0.7 % proven false obstacles along an independently driven route (tables 4.24 and 4.25); the drivability half of that result is dated, because its independent route is lost. The four closure-free maps of the realistic warehouse all fail the quality criterion fixed before scoring: precision 77.8–84.8 %, observed recall 74.9–82.5 %, false obstacles 3.8–10.2 % against the static-twin probes (section 4.4.1). The false obstacles are not thickened shelving: they are phantoms laid during the inertial initialisation transient at the spawn and wall bands displaced from their true lines, and the current configuration makes the maps worse than the October one through three parameters with separable

signatures, the lower detection rate costing views, the longer range laying wall phantoms, and the higher ground-height cut laying aisle obstacles on the I layout. On the large warehouse the attempt failed through tracking loss in repeated structure and its artefacts were not retained (section 4.4.1).

5.2 Limitations and Threats to Validity

The limitations of table 4.29 carry over without softening. Six of them bound the conclusions above most tightly.

- Every simulated cell holds one run, so no between-cell difference is an effect; the one run-to-run figure in the report is the agreement of ORB-SLAM3's two front ends on where and how its loop closer fails.
- No two recordings share a route, so traffic is a condition of a recording, not a tested factor.
- Every result is from one laptop at one replay rate, and the mapping-recording repeats were not interleaved between configurations, so nothing here is evidence of real-time capability on the robot's own computer.
- The non-visual baseline rests on idealised encoders in simulation and on a speed scale fitted against the truth on one recording on the real robot.
- The detector is scored against projected geometry with exemptions, and its moving-object truth excludes the walking people.
- The real-robot numbers are planar and provisional until the marker-to-IMU lever arm is measured, and rest on one tracker export that fails the validity rules and three that pass; VINS-Fusion has no valid real run.

The two hypotheses the project is named for, dynamic-object filtering and dense mapping, have no real-robot evidence at all.

5.3 Recommendations and Next Steps

Give Experiment 3 its test. Run both pipelines filtered and unfiltered on `realistic_002` and `_003` with the detector live, paired under the admissibility gates of section 3.6.3, with the loop closer disabled on the I layout so that the false closure does not dominate both arms; then the same on one `realv7` recording. Make the repeated runs on the two mapping recordings for the latency and resource table.

Bound the loop closer. Add a maximum correction against the inertial drift bound, or a covisibility-consistency window, and test it on `realistic_001`, where the 14 m false closure is reproducible in both front ends.

Fill the real-robot slots in the order the simulation results suggest. Exercise the fork's initialisation switches on a `realv7` recording and re-run VINS-Fusion with the corrected gravity norm; measure the marker-to-IMU lever arm; replace the fitted wheel-speed scale by a measured one and record the steering angle as a measurement so that the EKF can be formed with the 0.60 m wheelbase; annotate a frame subset of the real recordings for the detector; and build a first real-imagery database for Experiment 4.

Tune the visual systems to the platform. The cheapest remaining improvement is a measured inertial noise model for the visual estimators: Experiment 1 shows the noise model is worth more than the estimator.

Start mapping after the inertial transient, and reconsider the November configuration's range and ground-height parameters, which account for the realistic maps' false obstacles; score any map against a recording that did not build it.

Record every behaviour-changing switch in the run manifest, refuse an output directory whose name disagrees with its dataset, repeat every nondeterministic configuration at least three times, and retain the artefacts of failed attempts.

Future direction. Integrated relocalisation against a saved map, global pose-graph optimisation with a bounded loop closer, map reconstruction after objects have been moved, and alignment and merging of maps across sessions remain future work; path planning stays outside the project.

REFERENCES

[1] Saputra, M.R.U., Markham, A. and Trigoni, N., 2018, "Visual SLAM and Structure from Motion in Dynamic Environments: A Survey," **ACM Computing Surveys**, Vol. 51, No. 2, pp. 1–36.

[2] Cadena, C., Carlone, L., Carrillo, H., Latif, Y., Scaramuzza, D., Neira, J., Reid, I. and Leonard, J.J., 2016, "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age," **IEEE Transactions on Robotics**, Vol. 32, No. 6, pp. 1309–1332.

[3] BMW Group, 2024, **Autonomous Driverless Transport Vehicle Navigates Precisely through BMW Group Plant Regensburg Press Plant** [Online], Available: https://www.press.bmwgroup.com/global/video/detail/PF0009726/ [2026, September 25].

[4] Campos, C., Elvira, R., Gómez Rodríguez, J.J., Montiel, J.M.M. and Tardós, J.D., 2021, "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multi-Map SLAM," **IEEE Transactions on Robotics**, Vol. 37, No. 6, pp. 1874–1890.

[5] Qin, T., Li, P. and Shen, S., 2018, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," **IEEE Transactions on Robotics**, Vol. 34, No. 4, pp. 1004–1020.

[6] Qin, T. and Shen, S., 2018, "Online Temporal Calibration for Monocular Visual-Inertial Systems," **2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)**, 1–5 October 2018, Madrid, pp. 3662–3669.

[7] Labbé, M. and Michaud, F., 2019, "RTAB-Map as an Open-Source Lidar and Visual Simultaneous Localization and Mapping Library for Large-Scale and Long-Term Online Operation," **Journal of Field Robotics**, Vol. 36, No. 2, pp. 416–446.

[8] Chen, X., Li, S., Ren, S. and Wang, J., 2025, "RY-SLAM: A Robust YOLO-Based Real-Time Visual SLAM Solution for Dynamic Environments," **2025 International Conference on Virtual Reality and Visualization (ICVRV)**, IEEE.

[9] Macenski, S., Martín, F., White, R. and Clavero, J.G., 2020, "The Marathon 2: A Navigation System," **Proceedings of the 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)**, pp. 2718–2725.

[10] Dolgov, D., Thrun, S., Montemerlo, M. and Diebel, J., 2010, "Path Planning for Autonomous Vehicles in Unknown Semi-structured Environments," **The International Journal of Robotics Research**, Vol. 29, No. 5, pp. 485–501.

[11] Dubins, L.E., 1957, "On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents," **American Journal of Mathematics**, Vol. 79, No. 3, pp. 497–516.

[12] Macenski, S., Singh, S., Martín, F. and Ginés, J., 2023, "Regulated Pure Pursuit for Robot Path Tracking," **Autonomous Robots**, Vol. 47, No. 6, pp. 685–694.

[13] ASEAN Secretariat and AFFA Logistics Institute, 2019, "Chapter 3: Warehouse Layout, Storage, Handling & MHE," **Development of the Common Standard Curricula on International Transport and Logistics Basic Training for ASEAN Member States**, training material, 57 slides.

[14] Mecalux, n.d., **Warehouse Layout: Warehouse Design** [Online], Available: https://www.mecalux.com/warehouse-manual/warehouse-design/warehouse-layout [2026, October 6].

[15] Mobile Industrial Robots, n.d., **Eaton Case Study with MiR AMRs** [Online], Available: https://mobile-industrial-robots.com/cases/eaton [2026, October 8].

[16] Kanso, H., Singh, A., El Zarif, E., Almohammed, N., Mounsef, J., Maalouf, N. and Arain, B., 2025, "Semantic SLAM: A Comprehensive Survey of Methods and Applications," **Intelligent Systems with Applications**, Vol. 28, 200591.

[17] Keith, R. and La, H.M., 2024, "Review of Autonomous Mobile Robots for the Warehouse Environment."

CURRICULUM VITAE

NAME	Mr. Bolton Athit Davies
Date of Birth	[to be completed]
EDUCATIONAL RECORD	
HIGH SCHOOL	[school, year — to be completed]
BACHELOR'S DEGREE	Bachelor of Engineering (Robotics and Automation Engineering), Institute of Field Robotics, King Mongkut's University of Technology Thonburi, 2026
SCHOLARSHIP/ RESEARCH GRANT	[to be completed, or "—"]
EMPLOYMENT RECORD	Work-integrated learning placement, Gensurv Robotics, 2026
PUBLICATION	—